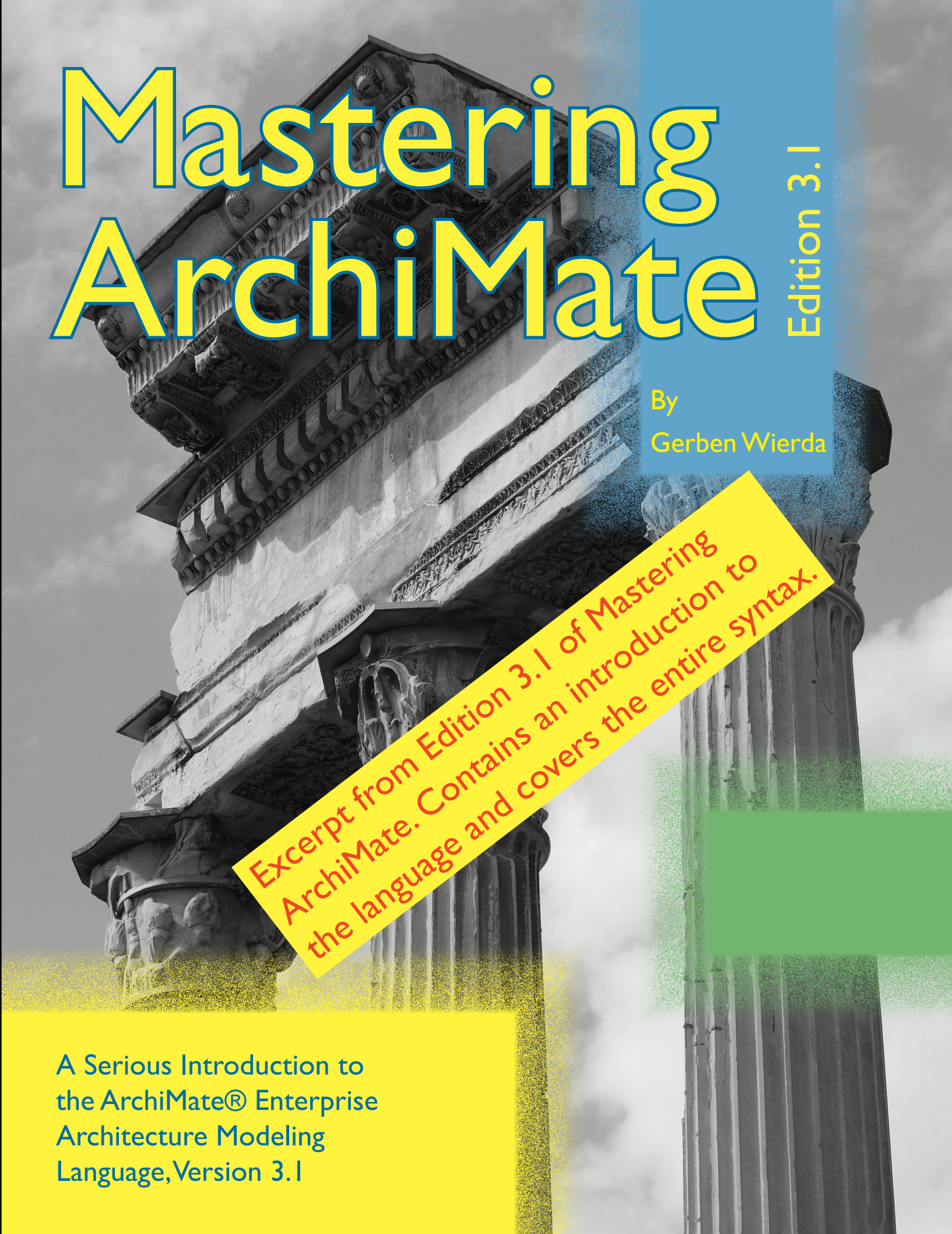

Mastering ArchiMate
Edition 3.1
By
Gerben Wierda
Excerpt from Edition 3.1 of Mastering ArchiMate. Contains an introduction to the language and covers the entire syntax.
A Serious Introduction to the ArchiMate® Enterprise Architecture Modeling Language, Version 3.1

Copyright © 2021 by Gerben Wierda, The Netherlands

Published by R&A, The Netherlands. Front cover photo by Jens Peter Olesen

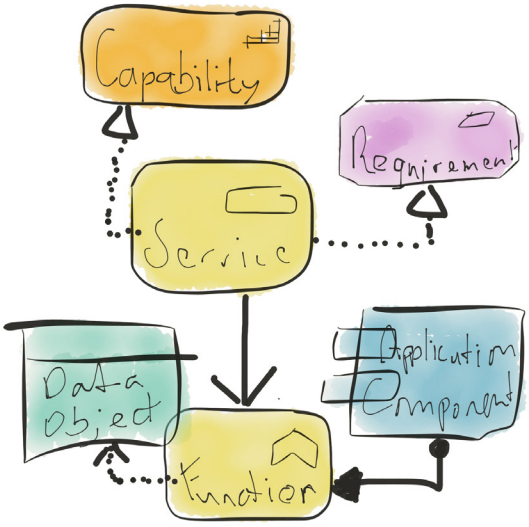
Edition 3.1 (v2021-04-19). Second Printing. This is the Free Syntax Excerpt of the Screen Edition. Please read Section 4 "License & Legal" on page 15.

Neither the author, nor the publisher, nor the printer, nor the distributor, nor the seller accept any responsibility or liability for loss or damage occasioned to any person or property through using the material, instructions, methods or ideas contained in or distributed with this book, or acting or refraining from acting as a result of such use. The author, publisher, printer, distributor, and seller expressly disclaim all implied warranties, including merchantability of fitness for any particular purpose.

ISBN PDF: 978-90-831434-0-8
ISBN hardcover: 978-90-831434-1-5
License Info: see "License & Legal" on page 15

Contact info: info@masteringarchimate.com

Mastering ArchiMate



Edition 3.1

Excerpt from Edition 3.1 of Mastering ArchiMate. Contains an introduction to the language and covers the entire syntax.

Praise

Praise for *Mastering ArchiMate Edition II and Edition III.TC1*

Mastering ArchiMate Edition II and *Edition III.TC1* received almost uniquely 4 and (mostly) 5 star reviews on Amazon:

This is a superb way to learn ArchiMate, don't think you are going to read it once and get everything out of it. This is a read and re read. I rarely give reviews but this is an outstanding read for anyone who is trying to model Enterprise Architecture

This is an excellent book and I was able to grasp concepts faster when compared to [the] ArchiMate Spec doc.

So many others are all theory, but this book provides detail rational on how to model in ArchiMate using real life practical examples

I very much liked the author's style of combining both theory and practice in an explicit and honest way, discussing also the limits of ArchiMate and the pros and cons of proposed patterns

If you want to apply ArchiMate in practice, I know of no better resource.

A thorough and practical book on ArchiMate that does exactly what its title promises: it introduces you to the basics concepts and builds on that foundation to more complicated stuff and always using real life examples, hence enabling you to truly grasp the intricacies of the modeling stuff.

Excellent book to learn the formalism of ArchiMate. Many concrete examples. Best practices, pitfalls to avoid. An essential complement to the standard document of The Open Group.

Whilst I'm not new to architecture or Enterprise Architecture I am new to ArchiMate and this book has been a fantastic primer on the subject. [...] I've been using this [...] on a complex project and found it extremely helpful!

I've found the book to be well written and very thorough, [...] not only great for beginners in ArchiMate, but will also serve as an excellent reference for more experienced modellers.

[...] goes beyond the trivial examples that most instructional books provide [...] If you are learning ArchiMate, you should absolutely buy this book. It is a bargain at twice the price!

This book is a masterpiece. [...] On top of modeling, it indirectly also educates on design principles. [...] boosted my productivity [...] The best bang for the buck in EA!

Mastering ArchiMate should be required reading for accelerating the knowledge and skills of serious Enterprise Architects.

A really clear handbook that describes both the notation but gives examples of how it can be applied to real life situations. My organisation has just moved to using ArchiMate and almost all of my colleagues have looked at the book, skimmed its contents and then asked where I got it.

Praise for *Chess and the Art of Enterprise Architecture*

Chess and the Art of Enterprise Architecture has received ~5 out of 5 stars on Amazon so far. Here are some quotes:

I read this as part of a work book club last year and the conversations it generated, as well as the observations that have stuck with me, have been invaluable. For me this book was packed with points I keep coming back to. It was also enjoyable to read and thankfully short. [...] I would recommend the book to any senior architect or senior IT leader.

This book captures the challenges faced by Enterprise Architecture in a clearly articulated and entertaining style based on pragmatic experience from the author. It is easily the best book I have read on the topic. It may not have all the answers but explains very clearly why there are problems in this area and gives good guidance on how to navigate around them and avoid the pitfalls of the past. It is easily understandable for a non technical audience and I would strongly recommend it for any one in IT or Business Change management.

Anyone working in architecture should have copies of this on their shelves to hand out to stakeholders so that they understand what architecture is about. Students could read this and pick up invaluable insights

And when Grady Booch was asked on Twitter for tips on good architecture books, guess which EA book was on his list...

Forewords

In the 15 years since its inception, the ArchiMate language for enterprise architecture modeling has come a long way. I have had the privilege of being part of its development from the start, first as project manager of the R&D project that developed the language, a collaboration of the Dutch Telematica Instituut and its partners, and later, after it was transferred to The Open Group in 2008, by leading the team that further developed and extended the standard.

Over the years, the popularity of ArchiMate has grown tremendously, supported by an increasing number of useful resources. Gerben's book has been one such resource and is of great value to anyone who wants to use ArchiMate in practice. The book gives an in-depth analysis and explanation of the language, complemented by many useful examples, modeling patterns and working practices. It goes beyond a mere explanation of ArchiMate, but also provides excellent insights in more general considerations of architecture and design.

The discipline of Enterprise Architecture is evolving, and ArchiMate has evolved with it. Since version 1, we have added concepts for expressing the rationale behind architectures, for describing how their implementation is planned and executed, for relating architectures to business strategy, and for modeling the physical world (something often overlooked by architects with an IT background). This has also served to align ArchiMate with other architecture approaches such as TOGAF and BIZBOK.

Now this book of course offers Gerben's personal perspective on ArchiMate, and we don't always agree. But the ArchiMate community is a big tent and the language can be used for many more things than it was designed for. As a self-confessed 'ArchiMate fanboy', he is nonetheless critical of what he sees as weaknesses in the language, and rightly so. Several of his past suggestions for improvement have been included in the current version of the language.

Next to the team who originally developed ArchiMate, Gerben is probably the most knowledgeable expert on the language, and the book you are now reading condenses much of this expertise. Where the *ArchiMate Specification* contains the formal definition of the language and the book *Enterprise Architecture at Work* describes the background of its design, *Mastering ArchiMate* provides you with the perspective of what I would call a 'rigorous practitioner'. All self-respecting enterprise architects should have these three next to each other on their bookshelves.

Marc Lankhorst

Managing Consultant & Chief Technology Evangelist
BiZZdesign
Enschede, The Netherlands, August 2017

I discovered ArchiMate in mid 2011 while trying to find a standard notation for architecture that would go beyond UML. I decided to read the official standard, which (in my opinion) is much easier to digest than the UML one. Despite being easy to understand, the ArchiMate standard itself wasn't of any help to start practicing (it contains descriptions of elements and relationships but it doesn't provide guidance). So I started a quest for information and examples...

I must admit that the small number of articles or blog posts about ArchiMate worried me. But, one day, I discovered Gerben's blog and my quest ended. In a short amount of time, thanks to Gerben's insightful posts, I was able to really understand key concepts, and I finally had examples of real ArchiMate use. Almost a year after having discovered ArchiMate, I was finally able to understand and use ArchiMate, just in time for a big and exciting company project for which architecture description was a key success factor.

Then the first edition of this book was published. Based on my experience with the blog, I expected much from it, and I was not disappointed: the book was presenting ArchiMate the right way, allowing people without any prior knowledge to understand it in a few days. It enabled me to quickly train my colleagues. Plenty of examples were already included, helping me and my team to quickly provide value, thanks to ArchiMate.

The title — *Mastering* ArchiMate — sounds exaggerated. How many books are there that promise you to 'master' something, while at the end you've barely learnt anything? But in this case it is real.

Together with *Enterprise Architecture at Work* by Marc Lankhorst, *Mastering ArchiMate* is the book that any serious Enterprise Architect should possess, to learn ArchiMate fast. This clearly is the kind of book that changes your practice as it also provides insightful comments about the standard itself, its limitations and how to overcome them. In this third edition, *Mastering ArchiMate* covers all new concepts and domains of ArchiMate 3. It also provides numerous new or improved examples that will certainly echo several of your needs (virtualization, DevOps...).

This is the book that has had the largest influence on my career. Without it I would not have learnt ArchiMate the right way, and I certainly would not have joined The Open Group ArchiMate Forum. I recommend it to everyone that joins an ArchiMate training: a training helps you for a week, but this book helps you for years.

Jean-Baptiste Sarrodie

Vice-Chair of the ArchiMate Forum
Enterprise Architect at Arismore, part of Accenture
Paris, France, August 2017

Table of Contents

Table of Contents

Not everything is available in this excerpt

Introduction 11

1. Why This Book? 11

1.1 Uncle Ludwig 11

2. Enterprise Architecture 12

2.1 Where are the principles and guidelines? 14

2.2 The Why, the How and the What 14

2.3 Disclaimer 14

3. Gratitude 14

4. License & Legal 15

5. Release Notes for Edition 3.1 15

ArchiMate Basics 17

6. An ArchiMate Map 17

7. Main Core Elements and Relations 18

7.1 Application and Business 18

7.2 The double use of the Serving relation 21

7.3 Function versus Process 21

7.4 Business Actor 22

7.5 Adding Technical Infrastructure to the Mix 22

7.6 System Software and Device 24

7.7 Composition and Aggregation 24

7.8 Nesting 25

7.9 Using a Node to encapsulate infrastructure 25

7.10 Event, Trigger and Flow 26

7.11 Modeling the Physical 27

7.12 Junctions 28

7.13 Grouping 28

7.14 Automated Processes 29

7.15 Distribution of data and matter (Path and Network) 29

7.16 Who is in Charge? 30

7.17 Abstractions 31

8. Other Core Elements and Relations 32

8.1 Product and Contract 32

8.2 (Formal) Collaboration and Interaction 33

8.3 The Association Relation 34

8.4 Specialization in the *Meta*-model 34

8.5 The Specialization Relation in an *Actual* Model 35

8.6 Representation 37

8.7 Location 37

8.8 The Complete Core 37

9. Derived Relations 39

9.1 Typical Application of Derived Structural and Dependency Relations 40

9.2 Derived Dynamic Relations 42

9.3 Are derived relations real relations? 42

9.4 How useful are derived relations? 43

10. Non-Core ArchiMate domains 43

10.1 Strategy 43

10.2 Implementation & Migration 44

10.3 Motivation 45

10.4 Closing Remarks on ArchiMate 47

11. Some Beginners' Pitfalls 47

11.1 Pitfalls of Derived Relations 47

11.2 Don't trust the language or the tool blindly: know what you are doing 48

11.3 Misunderstanding the standard meta-model diagram 48

12. Some Noteworthy Aspects of ArchiMate . 49

12.1 Licensing 49

12.2 ArchiMate, the modeling language 49

12.3 Separation of 'actor' and 'behavior' 50

12.4 One View Type 51

12.5 On the direction of structural relations under the assumption of 'worst case' 51

12.6 On the limitations of derived relations 52

12.7 Why two types of software? 53

12.8 There is more than one-to-one 54
12.9 Detailed or not? 54
12.10 Layering Ambiguity 54
12.11 The Use of the Non-core Domains 55
12.12 Why model? 55
Style & Patterns 57
13. Aesthetics (Style) 57
13.1 Why aesthetics matters 57
13.2 Arranging relations 57
13.3 Sizing and Arranging elements 59
13.4 Using color 60
13.5 Visual grouping 61
13.6 About labels 61
13.7 Don't Use the 'Children's Visuals' 62
14. Patterns 62
14.1 The Use of Patterns 62
14.2 A Basic TI Pattern: A Database 63
14.3 Modeling Spreadsheet Applications 65
14.4 Modeling an Internet Browser 67
14.5 More 'application platforms' 68
14.6 Infrastructure 'Building Blocks' 69
14.7 Deployment Pattern: Standalone PC Application 70
14.8 Deployment Pattern: Standalone PC Application with File Server based Data 70
14.9 Deployment Pattern: Classic Two-Tier Client-Server Application 71
14.10 Deployment Pattern: Two-Tier Client-Server Application with mounted Client Application and two databases 72
14.11 Deployment Pattern: Two-Tier Client-Server with a Remote Server 73
14.12 Deployment Pattern: Three-Tier Application 73
14.13 Deployment Pattern: Software as a Service (SaaS) 75
14.14 We only model what we can see/'change' 76
14.15 Deployment Pattern: Providing a local SaaS 76
14.16 The Use of Patterns (reprise) 76
14.17 Infrastructure Pattern: Database Replication 78
14.18 Infrastructure Pattern: High-Availability Database Cluster 78
14.19 Infrastructure Pattern: High-Availability Server 80
14.20 Using Collections and Abstraction in a model 81
14.21 External Data Services 81
14.22 Multiple Realizers 82
14.23 Other Parallel Relations 83
14.24 Summary of Basic Patterns 83
15. Some Anti-patterns 84
15.1 Application Collaboration is an Anthropomorphism 84
15.2 Using the Association Relation 86
15.3 Using properties too much 86
16. An Example from the Real World 87
17. Business Function and Business Process 91
17.1 On ArchiMate's divide between behavior and structure 91
17.2 Business Function or Business Process? 92
17.3 Good Old Fashioned Business Function 95
17.4 The 'End-to-end' Business Process 95
17.5 Business Process Modeling versus Business Layer Architecture 96
17.6 Dividing your enterprise's behavior into Business Functions 97
17.7 Concurrent Functional Landscapes 100
17.8 Application/Technology Process or Function 103
18. Secondary and Tertiary Architecture 104
18.1 Introduction 104
18.2 Primary Architecture 104
18.3 Secondary Architecture: Development 104
18.4 Intermezzo 105
18.5 Secondary Architecture: System Exploitation 105
18.6 Secondary Architecture: Application Maintenance 106
18.7 Tertiary Architecture: Application Owner 106
18.8 Tertiary Architecture: other roles 107
18.9 Making it Practical: shortcuts 108
18.10 Exploiting ArchiMate 3's improved flexibility 110
19. Modeling Risk & Security with Motivation Elements 111
19.1 Security Architecture 111
19.2 Risk 111
19.3 Modeling Risks and Controls 111
20. Using the Implementation and Migration Extension 114
21. Organizational Structure 115
21.1 Basic Organizational Structure 115
21.2 Using an ArchiMate landscape to design an object model 116
22. Virtual Machines (Parallelization) 118
23. Modeling Information Aspects 123
23.1 Messages and File Copies 123
23.2 Informational Structure 124
23.3 Status Change of Informational Objects 125
23.4 Access passive element from internal or external behavior? 126
23.5 Low Level Data Entry 126

24. Integration 128
24.1 Service-Oriented Architecture / The API Economy 128
24.2 Who is Serving Whom Anyway? 128
24.3 Associating with Trigger and Flow 129
24.4 Routing and Orchestrating Middleware (ESB and APIs) 129
25. Additional elements for model use 133
25.1 Using extra elements to facilitate analysis 133
25.2 Keeping in sync with other administrations 134
25.3 Reducing clutter through Aggregation 136
26. Complex Environments 138
26.1 Server Farm 139
26.2 Citrix 140
26.3 Complex Software Stacks 141
26.4 A completely alternative platform approach 143
26.5 Complex Application Components 144
26.6 Business logic at the infrastructure level 147
26.7 Bitcoin (Blockchain) 148
27. The Data Center 150
27.1 Networking 150
27.2 DevOps Ready Data Center (Puppet) 152
BPMN and ArchiMate 159
28. A Very Short BPMN Primer 159
29. A Possible Linking of BPMN and ArchiMate 164
29.1 An ArchiMate–BPMN Linkage Pattern 165
29.2 BPMN/ArchiMate: Multiple Roles performing a Single Process 169
29.3 BPMN/ArchiMate: A Process calling another Process 172
29.4 BPMN/ArchiMate: BPMN has no Architecture 'Layers' 173
29.5 The Process Landscape in BPMN 174
29.6 The Process Landscape in ArchiMate 179
29.7 Processes communicating via an ESB 181
29.8 Business Process Modeling in ArchiMate 184
Model Use & Maintenance 187
30. Construction & Use Views 187
30.1 Exceptional Construction Views 191
31. (Meta-)Model Maintenance 193
31.1 The Satellite Model Approach 193
31.2 Extending the Metamodel 195
Reflections 197
32. The Role of ArchiMate in Enterprise Architecture 197
33. Why Model? (And for Whom?) 198
33.1 Two modeling purposes 198
33.2 Better Insight 199
33.3 Overrated and Underrated 199
34. Using Abstractions 200
34.1 The price one pays 200
34.2 Too much detail? 201
Discussing ArchiMate 205
35. Proposed Improvements for ArchiMate 3.1 205
35.1 Fragmentation of element types 205
35.2 Remove historical baggage 205
35.3 Issues with abstractions 206
35.4 Issue with Metamodel-Specialization 207
35.5 Service as Composite Part of Function/Process .. 207
35.6 Automated Processes 209
35.7 Changing the Strength of Assignment and Realization 210
35.8 Make Access multi-directional 211
35.9 Allow two-way Flows and Triggers 211
35.10 Allow Serving to other relations 211
35.11 Why change ArchiMate? 212
Tooling 215
36. Multiple Models of a Single Reality 215
37. Tool Requirements 216
Index 219
List of Figures 226
Finger 233

Introduction

Introduction

1. Why This Book?

This book grew out of the desire to share the things I learned over the course of several years with respect to seriously employing the ArchiMate® Enterprise Architecture Modeling language. As Lead Enterprise Architect, working for the Asset Management unit of one of the largest Fiduciary Managers in the world, I introduced the use of ArchiMate in 2010, because it seemed to me — for a variety of reasons — the only reasonable choice for our modeling in the line of our Enterprise Architecture work. The choice was based on an estimate of the practicality of the language, and I grew very satisfied with how it worked in daily practice. This practice included — next to models for projects and target architectures — the maintenance of a single very large (tens of thousands of objects and relations) 'current state' model of our enterprise, built along strict guidelines and used for analysis, reporting and as source for other systems, as well as use for future state and project architectures. The scope and detail of our modeling taught us very valuable lessons about ArchiMate modeling, which I have shared in the various editions of this book: Edition I in 2012 (ArchiMate 1), Edition II in 2014 (ArchiMate 2), and Edition III.TC1 (ArchiMate 3.0.1 or 'ArchiMate 3 Technical Corrigendum 1') in 2017. Now that ArchiMate 3.1 (a relatively minor release of version 3.0.1) has been released, it is time for a new Edition, which is based on ArchiMate 3.1.

When we started using ArchiMate, I decided to stick almost religiously to the official definitions and not think or talk about diverging until we had a reasonable body of experience. After all, only when you have enough experience are you capable of really estimating the effect of the choices you have when changing the language. I still stay very close to what ArchiMate prescribes to this day, even if the depth of our experience has led me to some criticism and improvement proposals (see Chapter "Discussing ArchiMate" on page 205).

This book also grew out of the desire to provide a better introduction than what was available at the time. What was available was often pretty limited in scope and in my opinion sometimes even damaging in its explanations, as it would not teach the things to become a *good* modeler in ArchiMate. At best they would teach you some random patterns without for instance learning the pitfalls of those patterns. No introduction I had seen explained the language well enough and I did not see introductions that actually prepared you for sizable modeling work. This was certainly true also of the official certifying courses and training, which generally taught you only to reproduce the standard.

This book intends to do three things:

- Give a decent initial introduction in the concepts and relations that make up the language;
- Present a number of patterns and uses that may be useful when modeling in the language;
- Give enough content so you can develop a feel for the language.

The latter means that you will find some pretty deep and arcane discussions and examples in this book, here and there. These are not meant as practical examples to follow, but they are there because thinking about at the 'edges of practicality' improves your choice and understanding of practical solutions.

In the end, understanding is about being able to *apply* what you know. Understanding is know *how*, not know *what*. If you master the use of something, you understand something. Hence: *Mastering* ArchiMate.

1.1 Uncle Ludwig

Here and there, partly for fun, you will find references to an 'Uncle Ludwig'. Uncle Ludwig stands for the twentieth century philosopher Ludwig Wittgenstein, whom some characterize as the most important analytic philosopher (as opposed to, say, moral philosopher) to date. Wittgenstein in his entire philosophical life concentrated on 'meaning'. His result came in two parts. The first part in his youth, where he tried to build meaning on top of logic. The results were limited (but of all of his work sadly had the most influence on the computer science discipline). Later in life

he tried to answer what meaning then was for all the rest of what we say, where logic does not give you the definitive answer. He came up with the solution 'meaning lies hidden in correct use'. Given that ArchiMate is in part a service- (and thus use-) oriented language, this maxim is useful here and there. Actually, I find the maxim extremely useful for work even beyond modeling, e.g. wondering about the actual use of documents sheds light about their meaning.

If you want to know more about Wittgenstein, I suggest "Wittgenstein's Place in 20th Century Analytical Philosophy" by P.M.S. Hacker. Wittgenstein has been misinterpreted by many (e.g. when people misinterpret him as having stated that "meaning is *equivalent* to use"). Hacker not only understands Wittgenstein (he has written extensive and insightful analyses) but he also can explain it rather well. Wittgenstein often sounds daunting to people, but in my experience it can be pretty practical.

2. Enterprise Architecture

So you're working in an organization. The organization consists of many people, organized in organizational structure, in groups – including Agile/DevOps structures such as product teams or value streams –, departments and sections, and – if large enough– business units or even separate companies. If you look at what these people do, you look at business functions – say, 'after-sales' – or business processes – say 'handling a warranty claim from a customer'. These business functions and processes are a way to look at the behavior of your organization.

Now, these days, you have computers to support your work. You might not look at them anymore as computers (e.g. the iPhone or iPad you are reading a book on), but they are. And if your work is not shoe repair, plumbing, building, etc., your work will involve handling information. And even if it *is* shoe repair, the informational aspects of your work (planning, billing, accounting, etc.) are supported by IT. You *use* IT in the line of your work. The IT that supports people also has structure and behavior, just like the business itself. You use *applications*, and these 'running' applications have *functions* (i.e. behavior) and deliver *services* (support) to the business process. The applications themselves need an IT infrastructure to 'run', maybe large servers, maybe just your laptop or iPhone (where the applications are called 'Apps'), and there have to be all kinds of networks so all these systems can communicate with each other.

And not only is there IT that supports people, some of it operates *independently*. Software that runs robots in a factory. Web sites that completely handle the customer's interactions.

For about half a century now, the information revolution has moved most data from paper and other non-electronic media to IT systems, even the data that eventually is printed on paper. Not a multimedia, but a *unimedia* revolution has taken place: from all kinds of different storage and transport media, the information has been digitized and been moved to being small electrically charged or magnetized spots, each of these spots representing a single yes/no choice: a bit. We hardly think about that level, but we all know what a *file* is these days, and generally we do not think of the paper original that the term originally stood for.

And next to much of the *stuff* of business, the *behavior* of business has also become more and more digitized. And even our physical products have sometimes felt the result. Mass customization (the fact that you can have both scale and variation in production) is possible because we have computers handling all the diversity.

In large organizations, all these applications and files that support (or even are) the business have become an almost impossible to control, complex landscape, where many things can and do go wrong and where change is fraught with peril. Because changing something here will crash something somewhere else over there, in a landscape that is one big web of dependencies of business and IT. And even if that was not the case, translating business strategy and requirements to the right IT-support, or using IT-innovations to improve your business are difficult. Because, contrary to popular belief and partly as a result of that inertia-building web of dependencies, IT does *not* change fast. Mostly because its rigid logic lacks the flexibility of human compensating behavior. Building a new office is generally a process that takes less time than implementing (let alone building and/or implementing) a new core IT system.

This web of objects (physical and virtual products, bank accounts, bills, roles and actors, applications, data, servers, files, networks, etc.) and relations between them is what Enterprise Architecture is about. It is about the design of your business and the IT that supports it. It is about having the right business organization, having the right IT for your business and letting the business innovate, now and in the future. Especially, it is meant to lead to better IT choices, because, as stated above, (the complex landscape made possible by) IT is often more difficult to change than the stuff humans do. Humans are flexible. Digital computers have enabled a landscape full of brittle dependencies that shows many forms of inertia: resistance to change. But change is what we want and need.

The appearance of Enterprise Architects in this field is relatively recent. Not too long ago, if you would try to find that role you would end up looking at an organizations management. 'The' Architect of an 'enterprise' is its manager. He or she finally decides on how the business is organized, how it is run and what IT is implemented. But the field has become complex enough that a special function has appeared: the (Enterprise) Architect. The management has in fact outsourced the (rough) design of its solutions to a specialized function, whose task it is to handle all that complexity. Here at least is already one important point: Enterprise Architecture should be the responsibility of (organizational *management* of) the business, not of the IT provider. It is meant to help management make fundamen-

tal decisions, not leave them to someone else with some requirements and then say "make it so".

Now, the Enterprise Architecture function has proliferated and also fragmented. There are now business architects, security architects, application architects, data architects, information architects, integration architects, enterprise architects, infrastructure architects, domain architects, IT architects, solution architects, software architects, value stream architects, the list seems endless. And to make matters worse: the same job name may mean quite something different depending on whom and where you ask for the definition. What one company calls a business architect, the other company calls an enterprise architect or a lead architect and what one company calls an enterprise architect another may call information architect, etc.

So, I am going to lay out my own definition and fragmentation of Enterprise Architecture. First, in line with the Enterprise Architecture modeling language ArchiMate, I divide Enterprise Architecture into the following (ad hoc) practical *perspectives*:

- Business Perspective (processes, roles, abstract business objects such as 'account', etc.)
- IT ('virtual') Perspective (IT systems, data, programs, IT integration, data transport, etc.)
- Physical Perspective (physical goods, tools, transport, hardware, etc.)

These perspectives are tightly linked.

Enterprise Architecture for me has nothing to do with the organizational unit (department, business unit, project) that the architect has as his *domain*, but everything with the fact that he or she is architect on all *perspectives*: business, information, application, data, physical, infrastructure. *Enterprise Architecture is about the coherent design and modeling of all perspectives*. For me, a Solution Architecture is also Enterprise Architecture, because the subject is also a domain, and thus an 'enterprise' in itself.

Today, these perspective are often called layers, as if the business layer sits on top of an application layer which sits on top of a physical world. This BDAT stack has long permeated Enterprise Architecture thinking, because Enterprise Architecture was born from the problem of managing the complexity of many IT systems getting interconnected, and at that time, the stack was real enough: people used applications, applications used 'hardware'. Those simple days have gone.

I do recognize specialization of architects from a perspective: a Business Architect is concerned with the business perspective and an Infrastructure Architect with the IT-infrastructure perspective. Some call these layers 'domains' as well, but I find that confusing. I reserve domains for recognizable divisions of the organization, such as departments, business functions or projects. So, for me, a Domain Architect is an Enterprise Architect within a certain domain.

If enterprise in 'Enterprise Architect' does not denote organizational level, how do we then call the chief enterprise architect of the organization? My favorite job name for such a function is 'Lead Enterprise Architect' (and he or she should fulfill (amongst other things) a role comparable to that of the 'Lead Legal Counsel').

You can forget all of this, except for one thing: in the context of ArchiMate, 'Enterprise Architecture' says nothing about being in the top of the organization, but about the fact that an 'enterprise' is a coherent landscape that can be divided in business, application and technology layers (or very roughly: people, software and hardware) and Enterprise Architecture is about *all* of them.

A second division, is often made in our field: a division between 'architects' and 'designers'. For me, there is no fundamental difference between the two: both are forms of design and the only difference is the level of detail they are concerned with. Leaving out details is not to be taken lightly, though. It is one of the most difficult aspects of Enterprise Architecture. In my view, an Enterprise Architect is concerned with all details, but sparingly goes into those details. Architecture is (in part) "the art of leaving out *irrelevant* details". Leaving out details, sadly, often derails into religiously ignoring details. The key word, however, is 'irrelevant': as the Chinese proverb says: people stumble over molehills, not over mountains. An architect consciously leaves out details that he or she has decided are irrelevant to the decisions to be made.

In the early 90's, I had to follow a basic course on safety when working as a contractor for Shell. Here I learned a very valuable lesson: working safely is not about *avoiding* risks, it is about *consciously taking acceptable* risks. There is no such thing as *not* taking risks. For me, the same applies for abstraction in design. *Abstraction* is not about *ignoring* detail, it is about *consciously leaving out* detail. And luckily, as we will see later, ArchiMate is equipped for that, as it has a mechanism that supports having coherent detailed and non-detailed views of the same model.

There is a third division one can make in Enterprise Architecture. Basically, an organization can use architecture in the following three settings:

The Current-State (or As-Is or IST) architecture. This is a descriptive model of how the current landscape of business and IT is. It can be used for reporting (e.g. to regulators) and analysis;

The Future-State (or To-Be or SOLL) architecture. This is a (rough) prescription on how the future landscape should be. It generally consists of both high level models and guidelines or requirements for the more detailed work done in the line of moving towards the intended state;

Change architectures. These are the descriptions of what Change initiatives like projects will produce. A common form is a Project (Start) Architecture or a Solution Architecture (this is also true when working in an Agile methodology, both 'up-front design' and 'agile' require (a good documentation of) a good architecture). Like the Future-State, these are a combination of models and requirements, but the detail should generally be comparable to the Cur-

rent-State as the Change initiative actually results in a very specific change of that Current-State.

Enterprise Architecture is in the end about:

- Making good *choices* in the light of the strategic goals and positions of your enterprise;
- Making coherent *choices* across your enterprise
- Making good *choices* in themselves (e.g. in sense of total cost of ownership, etc.)

In all of these, modeling your existing state and your choices in ArchiMate can be really helpful.

2.1 Where are the principles and guidelines?

You might wonder, with all this talk about modeling: where are my architecture principles and guidelines that architecture is all about and that guide development for instance? I can say this about it here:

There are many definitions of what Enterprise Architecture is. The widely quoted ISO/IEC/IEEE 42010 standard, for instance, defines (system) architecture as

> *fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution*

There are two aspects: the *design itself* (elements, relationships) and the *principles of design*. This book is about the first aspect. I know many Enterprise Architects who are of the opinion that it is all about the second aspect. I could not agree less, and hence my approach to Enterprise Architecture is above all about the *actual* Business-IT landscape decisions that have to be made and the design of the next step in the perpetual change. Whether the use of principles and guidelines is a good way to come to such decisions is a question that is outside the scope of this book. It is one of the subjects of my other book: Chess and the Art of Enterprise Architecture.

2.2 The Why, the How and the What

There is another issue that needs clarification, related to the previous ones. As remarked above, many Enterprise Architects are of the opinion that Enterprise Architecture is all about the *why* of choices (strategy, translated to what and how), or in other words: about *intentions*. Now, I consider the why important, but for me – though the context of a design is very important – it is not part of that design, and the idea that it makes sense to model the why is questionable. Modeling your strategy and intentions (in ArchiMate called motivation) may be useful, but it is also naive: the complex, contradictory almost quantum-like nature of our intentions cannot practically be caught in a couple of simple models with a few boxes and arrows depicting influences. That is not to say that it cannot be useful to create a couple of visualizations that support a certain message.

Still, this book is mostly about using ArchiMate to model the actual enterprise, not its intentions, though some attention is paid to that subject and the basics are explained. As thus it is about using ArchiMate for Solution Architecture and Enterprise Architecture, and the discussion about what the difference is will be left to the audience.

And importantly: if at any point while reading this book you get the feeling "why am I looking at diagrams of arcane (technological) details of some sort of deployment or some sort of business?", remember: the book is not about teaching you those (maybe for you irrelevant) details, it is about teaching you ArchiMate (even if in the course of that it might also need to address context, such as explaining blockchain before showing diagrams about Bitcoin (Section 26.7)).

2.3 Disclaimer

I have to warn you: in case you did not know: you almost can't learn anything from a book (the exception being do-books like Bobby Fisher's *Chess Lessons*). The only way to learn something is by *doing* it (which is why Bobby Fisher's book is an exception as it centers around doing all the time). Knowledge is about 'know how' not about 'know what'. It's funny that – while being convinced of this – I have written a *book* to teach ArchiMate. But anyway, *you won't really learn ArchiMate from this book*, you need to seriously *use* the language to really learn it. But if you do that, this book is meant to help you out and speed you up. Writing the book, by the way, is almost (but not quite) as much fun as teaching a class (and educational in itself).

3. Gratitude

I am very grateful for the assistance of the following people:

- First and foremost: Jos Knoops, colleague at APG, who has been an invaluable sparring partner from 2010 to 2014 when discussing how we would be using ArchiMate, e.g. patterns;
- Former APG colleagues Joost Melsen, Floris Hack, and APG colleague Paul Hamers for more real world pattern discussions between 2011 and 2014;
- Leon Joosten, former colleague at APG and my then-time manager, who supported me at APG when I wanted to introduce ArchiMate to the organization in 2009. Without that opportunity I would not have been able to gather the experience to be able to fill this book;
- Peter Spiers of Adobe Forums, who helped me get started with Adobe InDesign and helped me solve serious issues in my first attempts at the document in 2012 and later in 2014, and without whose help, I would not have had a tool to produce this book at all;

- Marc Lankorst, the 'father of ArchiMate' and Jean-Baptiste Sarrodie, the Vice-Chair of the ArchiMate Forum, for their blush-inducing forewords on page 5.
- Steven Bradley for providing me with around a thousand linguistic fixes, improvements and suggestions.
- And of course, last, but certainly not least: my family, who — while I was writing — have carried the burden of the fact that I was writing and thus had less time to do my chores and be a sunny, active part of family life.

4. License & Legal

This book is © Gerben Wierda 2021. All photographs used are public domain. All diagrams have been created by Gerben Wierda unless indicated otherwise. ArchiMate® is a registered Trademark of X/Open Ltd. working under the name The Open Group. BPMN is a trademark of the Object Management Group.

This book comes in three different versions:

- A hardcover version distributed via normal channels for printed books (ISBN 978-90-831434-1-5);
- A full electronic version (PDF) that is restricted (printing not allowed) and stamped (ISBN 978-90-831434-0-8);
- An syntax excerpt electronic version (PDF) containing only the basic introduction to the language.

You are now reading the PDF edition.

Edition I was distributed as unprotected PDF. It was successful, but only a very, very small (but appreciated) minority donated or bought a license. Please do not give this document to people you did not buy a license for, and please do not remove the stamping or the printing restriction. It might feel like a victimless crime, but it isn't, because in the end it costs *me*. Statistics on *Mastering ArchiMate Edition I* have shown that when a free copy is available, less than 1 in 100 still pays. *Even* if 90 of those 100 would not have purchased the book anyway, a freely available copy in the end still costs me 9 out of 10 sales, and I worked very hard to create this book. Do you really want to reward me that way?

You can contact me at the following e-mail address:

info@masteringarchimate.com

Heerlen, The Netherlands,
Gerben Wierda

5. Release Notes for Edition 3.1

March 2021:

- In a few places I've used Donald Knuth's 'dangerous bend' image from The TEX Book (here shown on the right) to warn you that it gets a bit complicated. It's a way to show my respect for Knuth's work and to express my deep felt wish* it would have been feasible to do this book in TEX (if only because in my experience InDesign is slow and corrupts documents easily, i.e. has been poorly engineered).



- ArchiMate 3.1 was released in November 2019 It has taken a lot of time to update Edition III.TC1 to Edition 3.1. There were some personal reasons (this is done in my spare time, of which I had little); I write from actual experience, and I needed to build that experience; I had some aesthetical goals as well which led me astray (Knuth-like, but not as good or as serious). But being a designer at heart, I spent too much time on the looks, especially the diagrams;
- The numbering of diagrams is imperfect. Often higher numbers appear before lower ones. I have not found a way to do this properly in InDesign.
- In several places I have left the term 'infrastructure' for the IT part of what is since ArchiMate 3 called the 'technology' layer (and which supports physical technology next to digital technology) in place (though all elements are named properly). This is intentional, as infrastructure is a common term for that part of IT.
- The first edition of this book was written in 2012. Technology has changed, but I've not adapted all diagrams and text over the years, so some examples will be 'new' ArchiMate, but refer to outdated technology. Sorry.
- Really, look closely at the diagrams. I might write a blog post about the way these were produced, but they look like no diagrams from a tool you know. Guaranteed.
- Given that I have some room to spare: this book could not have been produced in the time that took without the music via/of King Crimson, (early) Gentle Giant, Camille Saint-Saëns, Edvard Grieg, Reinhard Mey, Jethro Tull, Studio Brussel, Pjotr Ilitsch Tschaikovsky, Georg Friedrich Händel, Simeon Ten Holt, Braak, (early) Allan Parsons Project, Pink Floyd, Alquin, Count Basie Orchestra, Philip Glass, (later) The Beatles, (early) De-Wolff, ELP, Frank Zappa, Jean-Michel Jarre, Joe Satriani, Johann Sebastian Bach, Mike Oldfield, Muse, Sergei Prokofiev, The Police, Red Hot Chili Peppers, Renato Carrosone, Stromae, Alfred Brendel, several soul songs, Gojira, Miles Davis, GOJIRA, Reinhard Mey, and many/much more...

* And not just because I received a TUG Award once. I truly love TEX's output quality, stability (boy, do I miss that), and logical writing through a macro package

ArchiMate Basics

Basics ArchiMate

ArchiMate Basics

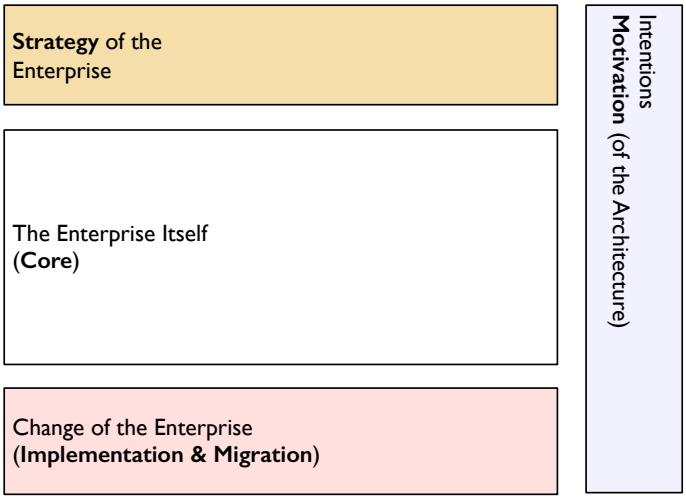
6. An ArchiMate Map

ArchiMate has over the years become a rich environment that covers many aspects of modeling in Enterprise Architecture. It started out as a simple language to model information-heavy enterprises, focused on modeling the relation between 'the Business' and 'IT'. In version 3 it covers a lot more. From modeling the strategy of an enterprise to the modeling of physical processes. The old core of ArchiMate 1 is still there, and it is appropriately called the 'core language'. It is what you use to model an actual enterprise: its processes, roles, actors, IT, physical equipment and facilities and so forth. This part of ArchiMate is about modeling *the enterprise itself*, as it is, as it should be, as it could be, whatever you need to express in your model.

As of version 3 of the language, ArchiMate now also has a part that is meant to model *the strategy of the enterprise*. It contains elements such as Capability and Resource. The language sees strategy as something that can be realized by 'the enterprise itself', it is thus seen as some sort of abstraction. We will get to the finer points about thinking about abstraction (and recursion) later.

As of version 2 of the language, ArchiMate has a part that is about modeling *the change of the enterprise*. Here we find elements to express (project) work packages, (architectural) plateaus, identified gaps that need to be filled, deliverables (of projects) and so forth. Though meant to model change of 'the enterprise itself', it can easily also be used to model changes of the 'strategy of the enterprise', e.g. we might model a deliverable that realizes a Capability.

Finally, also as of version 2, ArchiMate has a part that is about *intentions* which consists of elements such as requirement, goal, driver, stakeholder, et cetera. Though originally meant as a part that is used to model the intentions of the *architecture* of 'the enterprise itself', it can as easily be used to model intentions of 'the enterprise itself' or the intentions of 'the strategy of the enterprise' or the intentions of the 'change of the enterprise'. In fact, Mastering ArchiMate Edition II, the previous version of this book, only used them to model risk & security in relation to 'the enterprise itself'. All of these parts are shown in View 1.



View 1. *The ArchiMate Map — Rough*

This book focuses on the ArchiMate Core, *the enterprise itself*. The main reason for this is that it is there that we encounter the complexities that make modeling actually very useful. Another reason is that I have my doubts that modeling intentions and strategy are actually very useful. Modeling strategy cannot be much more than illustrative for what in reality is a narrative that has many aspects that practically can't be modeled at all in the same way that intelligent behavior cannot be caught in rules. Both intentions and strategy are domains that are far from logical in the real world and trying to map them onto a logical structure (which is what we do when modeling in ArchiMate) will have serious limitations, limitations which turn into 'bewitching ourselves' when we ignore them. Both also tend to invoke with me doubts that stem from being a bit acquainted with analytic philosophy. For the 'change of the enterprise' part of ArchiMate, I doubt if simply adding these to a model is usable and scalable and, again, their practical use might be only illustrative. Here, in my estimate, the main problem is that we do not have tooling yet that is capable of actually using this in a scalable way. For all three

parts, small toy-size examples, such as used by consultancy, tool and training providers can be easily created. But having these used in a large-scale complex setup in a way that is actually useful (and thus meaningful) is quite something else. I will pay attention to these parts, enough to give you a good introduction, but I will mainly focus on using ArchiMate to model 'the enterprise itself', using the other parts to strengthen that goal.

7. Main Core Elements and Relations

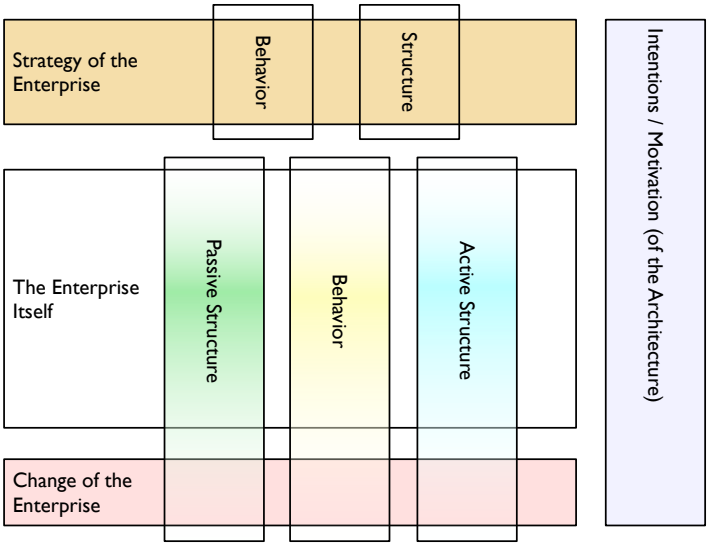
When modeling Enterprise Architecture, we need a language that knows about the concepts of 'the enterprise itself', and ArchiMate does a good job. To start with, you need to know that almost all of ArchiMate is built from three types of *elements*:

- elements that act: active (structural) elements
- elements that represent the behavior of those 'elements that act': behavioral elements
- elements that cannot act and which are acted upon by that behavior: passive (structural) elements

The three element types, connected by *relations*, can form sentences of sorts. A pickpocket (the application) steals (the application function) a wallet (the data). This is what makes ArchiMate a grammar, although people generally call it a language. The structure of the ArchiMate grammar is partly based on the subject-verb-object pattern from natural language.

Your model is therefore a story of sorts. It tells the reader the basic structure of the story: who acts on what.

Going back to our rough ArchiMate Map, we might add this internal structure as seen in View 3:

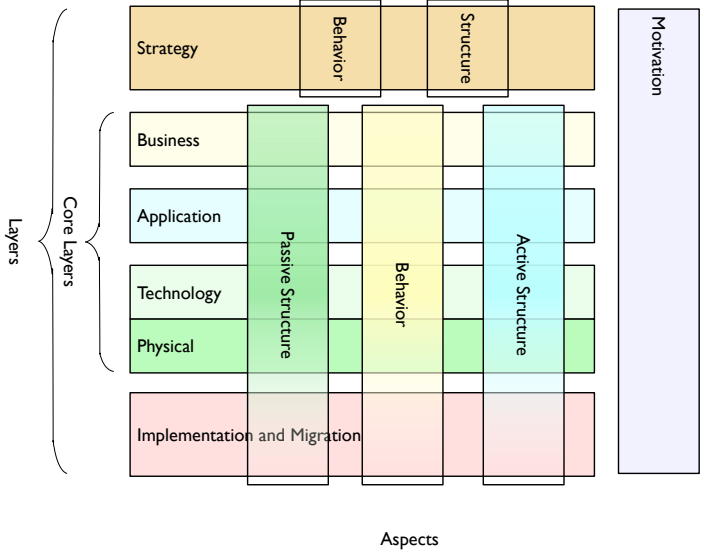


View 3. *The ArchiMate Map - With Aspects*

As you see, there is a slight anomaly here. The *Strategy* part of ArchiMate doesn't have active and passive structure, it just has structure (and only one element in that: Resource (but we'll get to that later).

We're not entirely done yet with mapping ArchiMate. The core of ArchiMate, the part about modeling 'the enterprise itself' has the classic (some would say 'orthodox') B(D)AT layering common to most enterprise architecture approaches. There are layers, from *Business* (processes, actors, roles and so forth) 'down' to *Application* (application components, application functions, etc.) and *Technology* (computing and other physical infrastructure and objects). So, a complete map would look like this:

The *Motivation* and *Implementation & Migrations* parts were extensions to the Core that first appeared in ArchiMate version 2. With respect to version 2 of ArchiMate, ArchiMate 3 has gotten an extension of the technology layer to model the physical side of the enterprise. This is why I (and others) prefer not to show the physical side as a wholly separate layer.



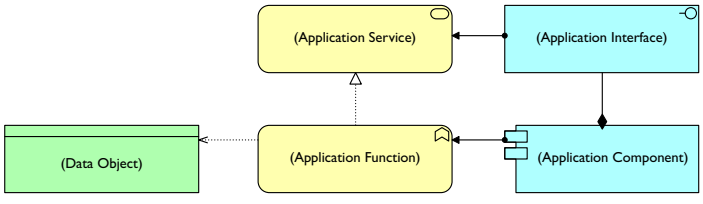
View 4. *The ArchiMate Map — Complete*

With all that said, we will now begin to start explaining the elements and relations in all these parts. We will start somewhere in the middle of 'the enterprise itself', specifically at the application & data perspective of classic IT-oriented Enterprise Architecture.

7.1 Application and Business

An application is fully modeled in ArchiMate as seen in View 2.

This is possibly the first snippet of ArchiMate you have ever encountered, and it already has five element types and four relation types, so we are going to take our time to describe it.



View 2. *The Basic Application Pattern*

Roughly, the two yellow and two blue elements in the image together make up the 'application' and the green element is the data the application operates on; i.e. think of it as the blue and yellow elements representing the word processing application and the green element representing the document being edited*. The lower three elements represent the *internals* of the application (blue & yellow) and the data (green). The two upper elements represent how the application can be used by and is visible for (exposed to) an application 'user', the *externals*.

One of the most essential aspects of ArchiMate is that modeling the behavior is separated from modeling the structure. The blue elements in the figure represent the active structure ('who') and the yellow elements represent the behavioral aspects of the 'who' elements — they are two sides of the same coin: in a certain sense one can't exist without the other.

It seems rather excessive that you need four elements to model a single application. We will later see that you can simplify this, even to a single one of these elements, but for the moment it is very important to understand what the underlying structure looks like. Actually, the lack of addressing this in documents and courses I have seen, has been a major reason for writing this book. An understanding of the foundation is required to model *well* in ArchiMate.

Having said that, here is a short explanation of the 5 element types in View 2:

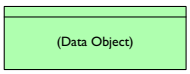


This is an Application Component. It stands for the (digital) 'actor' that an application in your Enterprise Architecture landscape actually is. It is one side of the coin of which the Application Function (its behavior) is the other.



This is an Application Function. It stands for the behavior of the Application Component, how the application can act. It is one side of the coin of which Application Component is the other.

Application Component and its behavior are, in a way, inseparable. You cannot have an actor that does not act unless the actor is dead, and in that case it should not appear in your architecture. And without magic, you cannot have an act without an actor. Again, later we will see how to leave things out of our views and models, but for now we stick to the details.



This is a Data Object. It is what the Application Function acts upon. The Application Function might create, read, write, update or delete the Data Object. Conceivably, the Data Object might not be needed; you can imagine behavior that does not access a passive element. You can also imagine that you do not model Data Objects that only exist *inside* your application. Not every variable in the application code is modeled. But as soon as the Data Object is visible to other parts of your landscape, or when it is persistent, it could be there. Generally, that means we generally only model (semi-)persistent Data Objects. Note: this is not the file or the database itself — those are represented on a lower level: the Technology level. We are still one abstraction level up. To illustrate the difference: an RTF file (technology) can be both an MS Word Data Object or an Apple TextEdit.app Data Object, depending on which application is used to access it.

Two elements in the image have not been explained yet. They have to do with how the application is used/seen (by the business or by other applications):



This is an Application Interface. It stands for the route via which the application offers itself to humans or to other applications. Note: both separate concepts (used by people and used by other applications) are supported by this one element. One example would be a Graphical User Interface (GUI), but it can as well be an Application Programming Interface (API), a Web Service or one of the many other ways an application offers itself to other 'actors' in your landscape. Given that difference in use (and thus, as Uncle Ludwig would say, meaning), it is unlikely that the same interface will be used by both a human or another application. But it can be; e.g., in the case of a Command Line Interface (CLI) used by a scheduler system, or Robotic Process Automation (RPA) using a GUI. Application Interface is a 'handle' of the 'actor' that is the Application Component. It is one side of a coin of which Application Service is the other.



This is an Application Service. It stands for the 'visible' behavior of the Application, how the Application Interface can act *for a 'user'*. It is one side of the coin of which Application Interface is the other side. Here again, the service may be 'technical' in that it is offered by one application to other applications or it may be part of your Business-IT integration: services offered to business processes (behavior of humans). The same type of element is used for both. This, by the way is a common first hurdle for architects coming from the engineering side, where it is common to define 'application service' as 'service provided by an application and *used by another application*' and define 'business service' as a 'service provided by an application that is *used by the business*'. In ArchiMate, a business service is something different, which we will see below.

Now, apart from the elements, there are relations between the elements. There are four in the initial image:

This is the Access relation. The access relation basically depicts a behavioral element accessing a passive element. Here it depicts the behavior of the application (the Application Function) accessing a passive

* ArchiMate is color-neutral. Color has no grammatical meaning. Most tooling use a 'layered' default color setup. I use a matrix based on the original ArchiMate 1 default coloring which stresses the difference between active/passive structure and behavior. See Section 13.4 "Using color" on page 60. I've found this choice to be easier for communication (and thus education, i.e. this book) but your mileage may vary.

data element (the Data Object – something that in the end generally resides in a file or a database). An arrowhead (on either side) is optional and it may depict read access or write access (e.g. two for explicit read/write).

This is the Composition relation. It means that the element at the end with the diamond is the *parent* of the element on the other end and that the child *cannot exist independently* from the parent. The relation depicts the composition of larger wholes out of smaller parts, but it does not mean the set of children modeled must be necessarily complete in your model: there may be parts that are not modeled. This relation could also for instance be used to show that an Application Component has various subcomponents.

This is the Realization relation. This has two types of use in ArchiMate. Here it means that the element at the end without the arrowhead is the element that 'creates' the element at the end with an arrowhead: the application's internal functionality realizes a service, which is the externally usable functionality of the application.

This is the Assignment relation. This also has more than one meaning in ArchiMate. Here, it means that the side with the dot (the active element) *performs* the behavior that is the behavioral element on the side with the arrow head.

Important and possibly initially confusing aspects of ArchiMate are thus that – while we have multiple elements (structural and behavioral) representing what is in the mind of many a single thing (an application or an application interface) – we also have single elements (and as we will later see, relations) that can be used with multiple meanings. When you get the hang of it, everything becomes pretty natural just like it is with natural language, but if you are looking for a strictly disjunct (made of independent concepts) approach to a modeling language (e.g., like a programming language or like UML), it might be confusing in the beginning.

Having modeled an application, we can turn to modeling the way this application is used by the business. And before that, we need to look at the way the business & information layer of Enterprise Architecture is modeled in ArchiMate. Luckily, it looks a lot like what we saw in the application layer so it is easy to understand now and it can be seen in View 6.

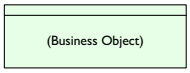
I have left out the actual human actor – the one that fulfills the Business Role – for now, to stress the equality between this pattern and the one about the application in View 2 on page 18. It looks quite the same and that is not a coincidence. The relations are the same as with the application image above. The new element types are:



This is the Business Role. The Business Role is an 'actor' in ArchiMate, but it is slightly more complicated than that, because it is an abstract sort of actor based on 'being responsible for certain behavior'. The real actors are people and departments and such. ArchiMate has an element type for those as well, but we leave that for later. Business Roles can perform Business Processes, just like Application Components can perform Application Functions.

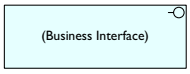


This is the Business Process. It stands for a set of causally-related activities that together realize services or create something. Roles can be assigned to a Business Process, they perform the process, just as the Application Component performs the Application Function. Just as in the application layer, a Business Process cannot exist without a Business Role (which does not mean you must model both, I am talking about the reality you are modeling), they are two sides of the same coin. We also have Business Function, but we leave that for later.

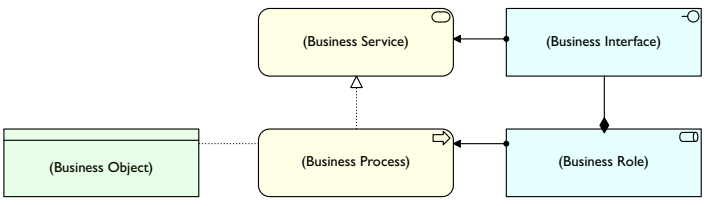


This is the Business Object: the (generally) abstract element that is created or used by a Business Process. Think of objects like 'a payment' or 'a bank account' or 'a bill'. Though it is named Business *Object*, it is more like a *Concept*. It may represent non-informational objects as well, e.g. if your company produces steel beams, you may have a 'Steel Beam' Business Object representing the beam itself and not information about the beam.

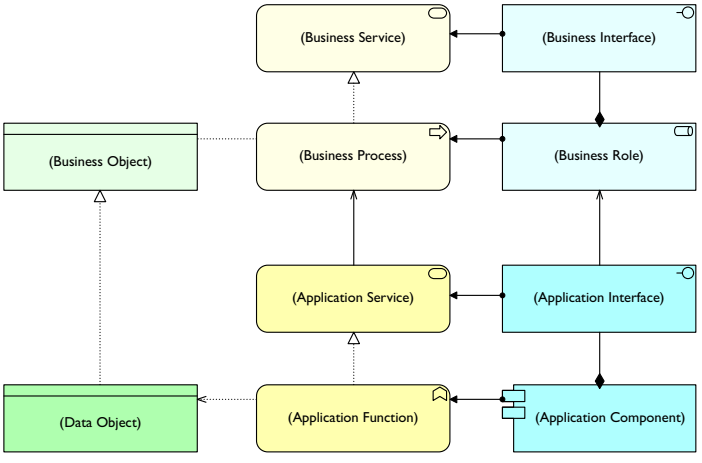
Again, just as with the application, these elements can make sentences of sorts. The proverbial 'second hand car sales' role performs the 'sell second hand car' process, which creates a 'bill'. Criminal, crime and – in this case, possibly – proof of crime.



This is the Business Interface: the way the role interacts with others. You can think of it as a 'channel'; e.g., phone, mail, meeting, etc. The interface is the visible manifestation of a



View 6. *Basic Business Process Pattern*



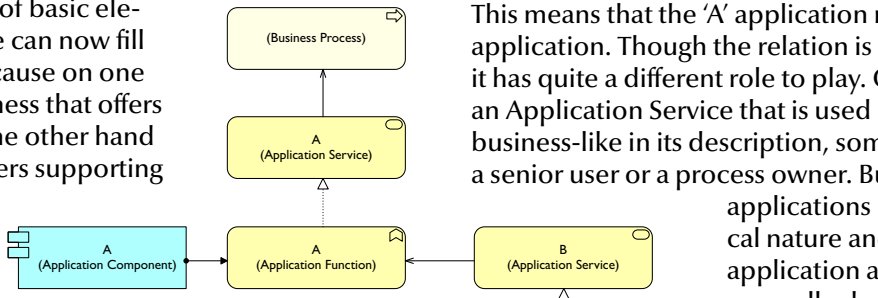
View 5. *Basic Application is used by Basic Business Pattern*

role. E.g. if the role is 'help desk agent' the interface might be 'telephone', 'Twitter' or 'web site chat'.



And this is the Business Service: more or less what it is always about in an organization. This is the reason for the existence of the process. This is the service it offers to (and thus can be used by) others (either inside or outside the company). A company might offer many services; e.g., a bank offers at the most abstract level a 'savings' service or a 'loan' service.

Having set up a couple of basic elements and relations, we can now fill in the missing links. Because on one hand we have the business that offers services to others, on the other hand we may have IT that offers supporting services to the business. Together they look like View 5 on page 20.



View 7. *An Application Using Another Application*

The application level is connected to the business level by three relations. On the left we see the already familiar Realization relation (). Here it means that the Data Object realizes the Business Object. In a concrete example: the 'bank account' Business Object may be data (a Data Object) of an accounting application; it is the same item's representation in a different architectural 'layer'. Note: we will get back to 'concrete versus abstract' modeling later.

In the middle and on the right we see a new relation:

 This is the Serving relation. It means that the element at the end without the arrowhead *serves* the element at the end with the arrow head. In previous versions of ArchiMate this relation was called Used By and you will still see this name in many places. The Application Service, for instance, Serves (is used by) the Business Process. The Application Interface (e.g. the Graphical User Interface) Serves the Business Role.

The fact that an Application Interface can serve a Business Role is the 'other side of the coin' of the same Serving relation between Application Service and Business Process. They are twins. Both illustrate the 'service oriented' way that ArchiMate relates one layer to the next as far as actors and their behavior goes.

With what has been explained so far, you can already do much of the modeling you need in terms of Business-IT focused 'Current State' or 'Change/Project' Enterprise Architecture. There are two more things that need to be explained before the basic setup is complete: applications using other applications, and business processes/roles using other business processes/roles. Here again, what happens at business level and application level is identical, so we are going to illustrate only one.

7.2 The double use of the Serving relation

So far, our example has only shown Serving as a relation *between* levels in your architecture. But the same relation type can also be used *within* a level. The business may use an application, but an application can also be used by another application.

In View 7 you see the same Serving relation () twice. Once between the 'A' Application Service and the Business Process that uses the 'A' application, and once between the 'B' Application Service and the 'A' Application Function. This means that the 'A' application makes use of the 'B' application. Though the relation is Serving in both cases, it has quite a different role to play. Often, the definition of an Application Service that is used by the Business is pretty business-like in its description, something you discuss with a senior user or a process owner. But the relation between applications is more of a technical nature and you discuss it with application architects. The difference generally shows itself clearly in the types of names and descriptions of the service. It is hard to truly imagine an Application Service that is both used by another application and a Business Process. After all, both uses will be pretty different, unless we are talking about a fully automated business process, which we will handle later.

It is good to mention here that 'serving' (or 'used-by' as it was called before) is not unambiguous. If Application A calls a service (API, web service, etc.) of Application B, but it does this to deliver information that Application B needs, who serves whom? Arguments can be made for either direction. We'll get back to this in Section 24.2 "Who is Serving Whom Anyway?" on page 128 when we will discuss the choice of pattern in some more depth. For most of our patterns we will not need a deep discussion.

7.3 Function versus Process

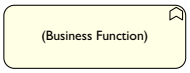
So far, we have modeled business behavior as a Business *Process* and application behavior as an Application *Function*. But ArchiMate actually has two main work horses for behavior in all layers: process *and* function. In terms of the grammar, they are completely identical. They allow for exactly the same relations with other elements. In fact, you could see them as 'convenience types' for a generic 'behavior' element in each layer. For now, as we are doing the superficial introduction, it is best to use the following guidelines:

- You use a process if you are thinking of a causally-related set of behaviors ('activities') that in the end *produce* something, normally a service or an object. Business Process is thus an outside-in way of dividing up business behavior, based on what the behavior actually *produces*.

- You use a function if you are thinking of a grouping of related behavior based on – for instance – same tools, same skills or same role that performs it. Business Function can thus be seen as an inside-out way of dividing up business behavior, based on what it *is capable of*.

In section 17 "Business Function and Business Process" on page 91 we will look into the difference in the business layer in more depth and see that there is a nice way of combining them both when modeling your business layer.

Business Function looks like this:



And Application Process, as you might expect, looks like this:



We will get into the use of colors later, but as you can see I use colors to separate the aspects (structure, behavior) and intensity to separate the layers (business, application). It suffices to repeat the earlier footnote on page 19 here that ArchiMate is officially colorless. I use these colors because in my experience they really work well.

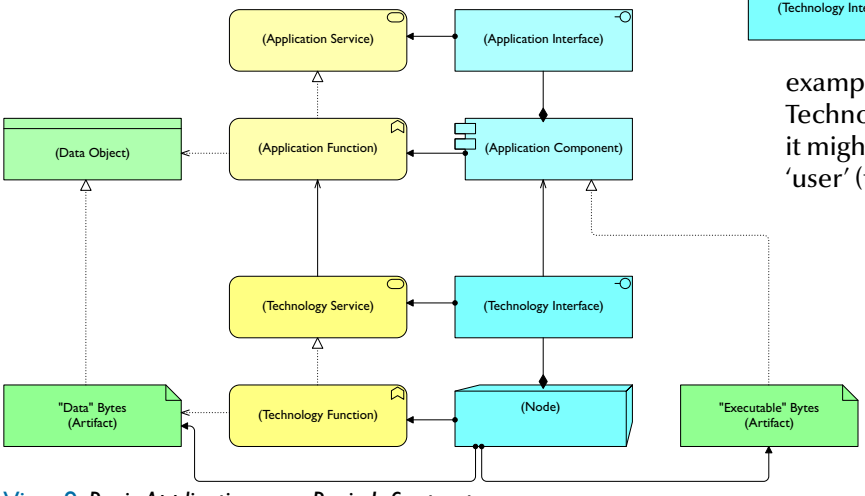
7.4 Business Actor

Earlier we encountered the Business Role. The Business Role is an abstract sort of actor. But in a business architecture there are of course *real* actors: people, departments, or even business units or companies or maybe regulators. ArchiMate has an element for that: the Business Actor, as seen in context in View 8.



View 8. *Business Actor in Context*

On the left we see the already familiar Business Process to which a Business Role is Assigned. The Business Role *performs* the Business Process. On the right we see the new element type Business Actor, which is Assigned-To the Business Role. This must be read as the Business Actor *fulfills* the (responsibilities of the) Business Role. ArchiMate was designed to be economical with relation types, so it re-uses the relation for a slightly different meaning.

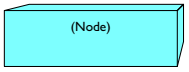


View 9. *Basic Application uses Basic Infrastructure*

7.5 Adding Technical Infrastructure to the Mix

Just as the Business Process in our current scenario needs the Application Service to be able to be performed, the Application Function needs (IT) infrastructure to run. If we add the infrastructure to the application layer, we get View 9.

There are no new types of relations here, but there are new element types:



This is the Node. This is a slightly complicated concept in ArchiMate and more details follow later. For now, think of it as a generic piece of IT infrastructure: hardware combined with its system software, where files are stored or applications can run. We'll see the details later.



This is the Technology Function. Just as with the Application Function and the Application Component, the Technology Function stands for the behavior of the Node. They are two sides of the same coin. As you might guess, there also is a Technology Process, which is the same, just with an 'arrow' icon in its standard visualization.

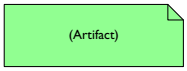


This is the Technology Service, the visible behavior of the Node. In many ways, this is what the infrastructure is all about, what it can do for the applications, its reason for existence. This is what the applications need to function. Typical examples of Technology Services are for instance a 'file share' or 'application execution' or a 'database service'. The latter may cause confusion, because you might wonder why that is a T*echnology* Service and not for instance something at the *application* level. We'll discuss that in Section 12.7 "Why two types of software?" on page 53, but for now, it is enough to say that the Node comprises both the hardware and the system software. A database system is generally modeled as 'system software' and the database as an Artifact (see below).



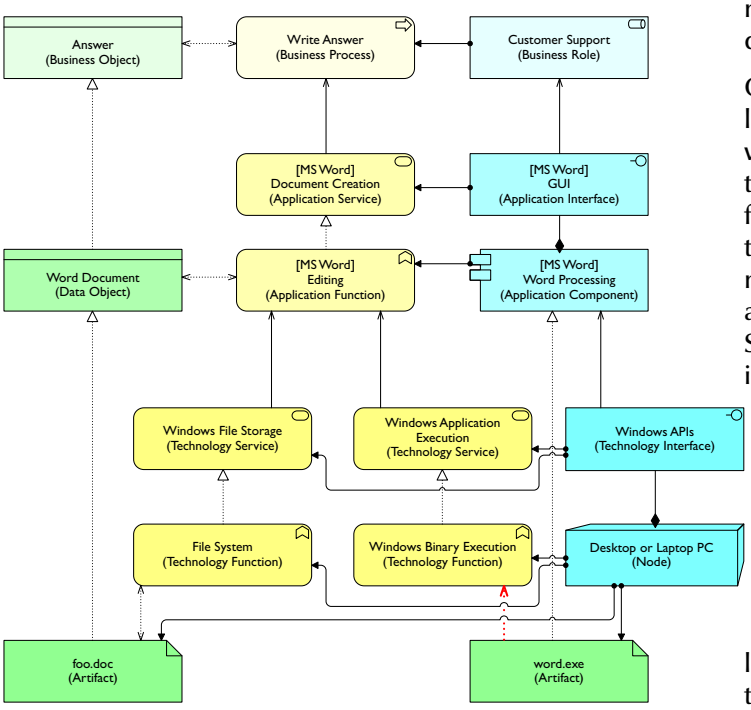
This is the Technology Interface. This is not that easy to explain. For Application Interface, one can easily dream up an easy example: the (graphical) user interface. But for the Technology interface, ArchiMate is not very clear; it says it might be best thought of as a kind of contract that the 'user' (the application) has to fulfill. An example would be a protocol, like the SMB or NFS protocol for file sharing and the TCP/IP or UDP/IP ports where the service is offered. Unless you are a dedicated infrastructure architect, you can generally do without this one. I am just mentioning it here to be complete and because leaving it

out here would introduce the concept of 'pattern' before it is wise to do so.



The last new element (for now) is the Artifact. This one is pretty simple. The best example is a file. Another often-used example is the actual database where your application's data resides. Another example is the actual 'executable' (file) also known as 'the binary': what your application is when you look at the 'byte' level. The 'data bytes' Artifact in the model above is the one that realizes the Data Object that is accessed by the Application Function. The 'executable bytes' Artifact forms the bytes (a file, or a set of files often called a 'software distribution') that the system can read and interpret as a program. On the infrastructure level, 'a byte is a byte' in the sense that both passive (Data) elements and active (Application) elements are in the end nothing but a collection of bytes. Deep down, we get to the basic level of zeros and ones (as we should).

The relations are mostly pretty straightforward. The Assignment relations (•——►) between Node and Artifacts stand for the fact that the Artifacts *resides* on the Node. In other words, it depicts where in your infrastructure you can find a file. Also pretty simple are the Serving (——►) relations between Technology Service and Application Function and between Technology Interface and Application Component. If an Application Function needs access to a file that resides on a file system, the file system is a Technology Service that Serves the Application Function. And its mirror is the Serving relation from Technology Interface to Application Component. This mirroring is depicted by the Assignment relation (•——►) between Technology Interface and Technology Service, exactly as happens in the levels above between the 'interface' and the 'service'.



View 10. *Write Answer Process, supported by MS Word and a Standalone PC*

And lastly, there are the Realization relations (┈┈►) between an Artifact and a Data Object or between an Artifact and an Application Component. This one is also pretty simple and easiest to explain by example. Suppose your application is Microsoft Word, then the file you are editing could be called "foo.doc". And if the application is MS Word, the Artifact realizing the Application Component could be "word.exe".

> *Note: we are still in an IT-focused scenario. The Technology Layer of ArchiMate's Core has (since version 3) elements to model the physical side of an enterprise, e.g. a physical production process. We will get to that below. For those that are already acquainted with earlier versions of ArchiMate: the name of the 'bottom' layer of ArchiMate has changed from (IT) 'Infrastructure' to the more generic name of 'Technology' so it better fits the non-IT oriented use.*

I have a detailed example model for you in View 10 to see everything in a single context. The model shows someone writing a letter to a customer that contains an answer, supposedly to a question the customer has asked. Two infrastructural services are needed for this to work. The application should run and the document must be stored. In this example, everything happens on a standalone PC.

If you are an 'enterprise' architect you may think that all this detail is irrelevant and that the language looks like a language for detailed design only. Don't worry: using ArchiMate does not mean you absolutely must model all these details, the language can handle both: a roughly sketched Business Operating Model down to the nitty-gritty details you have to confront when you are in a project. I am simply using an example everybody knows, to illustrate how it works in ArchiMate. And even in View 10, I have left some things out, and I have combined the Technology Interfaces into one element. I could also have combined the Technology Services into one element. I will address handling details later when I am discussing 'modeling patterns'.

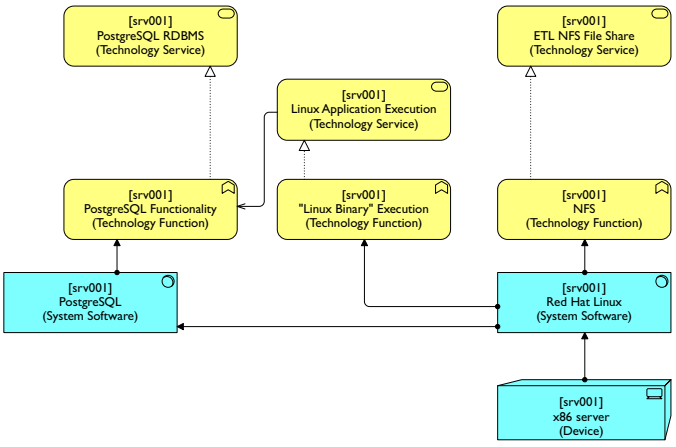
One more comment, before we go on with the rest of the language. As you might have noticed, the initial example with infrastructure in View 9 had no Access relation between the Technology Function and the 'executable' Artifact. But the 'write answer' example of View 10 does have that kind of a relation (shown in red). Here an explicit Technology Function for application execution was modeled and that function needs access to the 'executable' artifact. So which one is correct? The answer is: whatever your brief is and what you want to show. ArchiMate is a grammar, so it is *your* choice what you want to put in that grammar and how you put it. You have considerable freedom and you will certainly develop your own style. You can certainly model incorrectly in ArchiMate, just as you can write false statements or make grammatical errors in any language. So, yes, there are wrong models, but I will be unable to show you a way to model 'the right way'. The syntax of the ArchiMate language does not by definition lead to correct statements, but there are also many ways to say the same thing, just as with other languages. Later,

we will address choosing your style and patterns, the latter being like 'common phrases in the language'.

7.6 System Software and Device

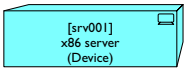
Our first use of IT infrastructure elements was pretty limited. We added the Technology Function, the Technology Service, the Node, the Technology Interface and the Artifact. ArchiMate adds two useful active structure concepts to model the actual infrastructure level of your IT architecture: Device and System Software. The best way to explain them is by giving an example of how they can be used.

In View 11 we see a model of a database server in our landscape. The name of the server is 'srv001' and that could be the name it carries in a CMDB for instance.



View 11. *Device and System Software*

The new element types are:



This is the Device, the actual hardware, the 'iron', the equipment of our infrastructure. In this example, two elements of the type System Software have been deployed on it (Assigned-To it). The little logo of the element is an image of a keyboard/screen.



This is the System Software element. It stands for software that we consider to be part of our Technology layer and not our application layer. Common uses are operating systems or database systems, but also runtime environments are generally modeled as System Software (the name of which is a bit quaint, given its wider use). In our example both are available: the Red Hat Linux operating system and the PostgreSQL database system. Modeled too is that the PostgreSQL software uses the Red Hat software. (If we have our existing landscape modeled in such detail, we could by analysis find all PostgreSQL databases that run on Red Hat, handy if we are planning an update and there is something about PostgreSQL running on Red Hat that merits extra attention).

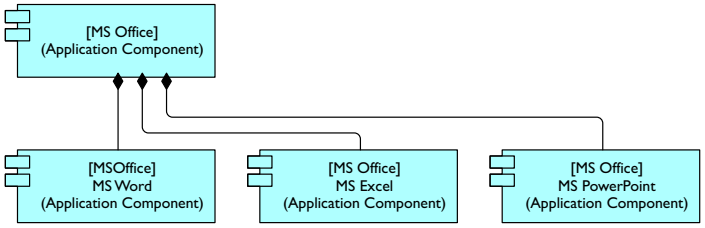
What really sets System Software apart from Application Component is that System Software is often a sort of *platform*. It is an environment where other software can run. So, for instance, Java on your computer is System Software. It can read and execute JAR files which contain Java instructions. See also section 12.7 "Why two types of software?" on page 53.

7.7 Composition and Aggregation

We already met the Composition relation (◆——) earlier. The composition represents a whole-part relation. It is best to look at the ArchiMate version of this common relation as the relation between a tree (a real one from your garden, not the computational concept) and its branches. A branch is a branch of a single tree and cannot be a branch of multiple trees. If the tree is destroyed, all of its branches are destroyed as well.

Generally, you may always create a Composition relation between two elements *of the same type*. View 12 contains an example.

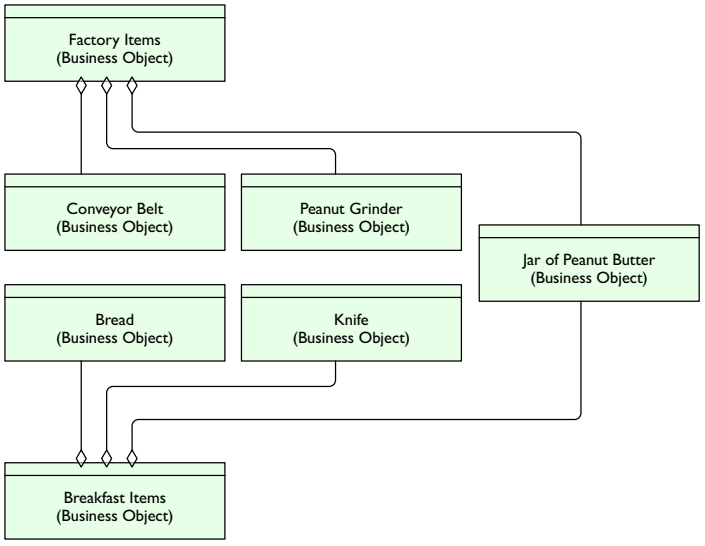
The Aggregation relation (◇——) is a variation on the



View 12. *Composition Example*

theme. It is shown in View 13.

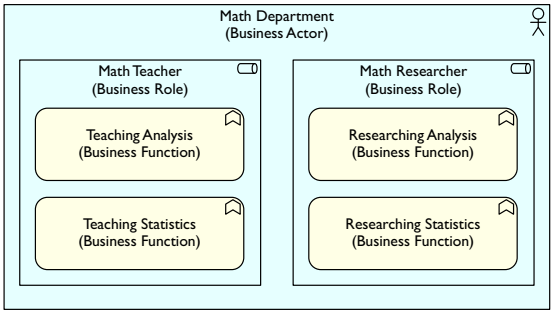
It is best to look at the Aggregation relation as a kind of



View 13. *Aggregation Example*

grouping (note: in Section 7.13 "Grouping" on page 28 we will see an official way to 'group' various elements of *different* types). The 'parent' (on the end with the diamond) represents a 'collection' of the children. But unlike Composition, the children may be a member of multiple Aggregations as you can also see in View 13: The 'Jar of Peanut Butter' is both part of the 'Factory Items' of the peanut butter factory and the 'Breakfast Items' of a consumer's home. It's like the number 4 being both part of the collection of squares of integers and the collection of even numbers.

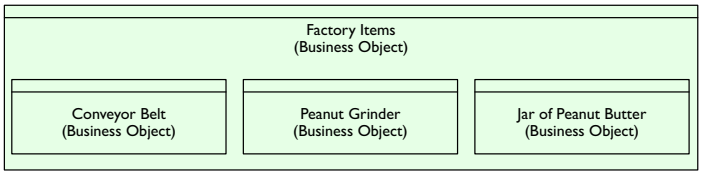
Composition versus Aggregation is sometimes described as 'has-a' versus 'part-of', the difference being not directly clear from the terms. Also, composition is sometimes referred to 'strong ownership', whereas aggregation is sometimes referred to as 'weak ownership', again somewhat problematic as the aggregation does not 'own' the child at all.



View 17. *Assignment Nesting, Two Levels Deep*

7.8 Nesting

There are four relation types that may be drawn by nesting an element inside another element: Composition, Aggregation, Assignment and Realization. These are the so-called *structural relationships*. Note: tooling often allows more than the language does, especially if you use tooling that is nothing more than a good drawing application such as Visio for Windows or OmniGraffle for Mac OS. Anyway, let's take the 'Factory Items' from View 13 as an example. Nested, it looks like View 15, which looks a lot cleaner and that is why many modelers like it. But we can already see a disadvantage: you no longer see anymore what the relation

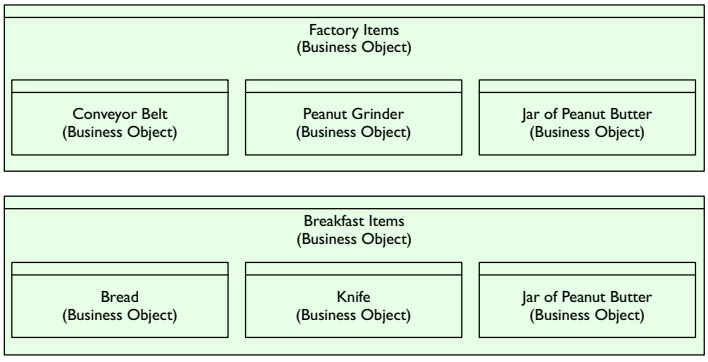


View 15. *Nested Aggregation*

is between parts and whole: Composition? Aggregation?

It gets worse, when you want to model both Aggregations from View 13 in Nested form in one view as in View 16.

Not only are you unable to see the actual relations, it has



View 16. *Two Nested Aggregations with Shared Object*

now become necessary to include the 'Jar of Peanut Butter' element twice. And though the name is the same, there is nothing that will tell you if behind-the-scenes it is the same element. You can have two different elements with the same name in ArchiMate, after all, the label has no meaning inside the language (as the language' is in fact a *grammar*) even if it has a meaning in the world of an architect. Besides, even if your modeling guidelines force different names for different elements, think of a very large view with the same element twice. Are you going to spot that the same element occurs twice? Probably not. So are you going to see the dependencies in full? Again: probably not.

View 17 contains another Assignment example of Nesting, now two levels deep.

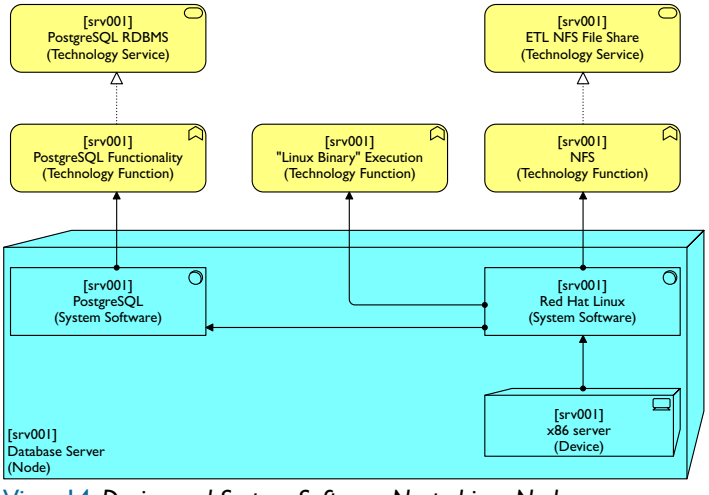
In summary: Nesting makes for nice views on your model, views that are easy on the eye. But don't think about them too lightly, because *constructing* your model this way comes with risks. And even bigger risks than you think, because some tools will let you create nestings without actually creating a relation between the nested elements (which is in conflict with the ArchiMate standard), or they may create a default relation which was not the relation you were thinking of. I've seen one tool that was unable to produce View 16 unless the shared object was not shared at all, but created twice in the model; this because nesting was used to show the tool's administrative model structure.

7.9 Using a Node to encapsulate infrastructure

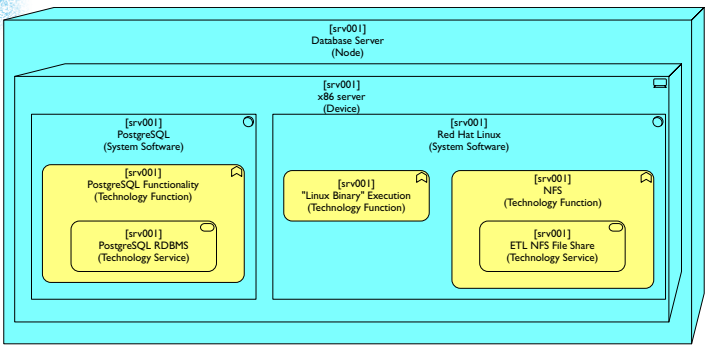
Now that we have seen Composition and Nesting, we can look at View 11 on page 24 that introduced System Software and Device and show in View 14 and View 22 on page 26 how it might be modeled.

In this example, the System Software and Device elements are children (Composition) of an abstract Node element (the Composition relations are not explicitly shown here, but instead modeled as Nesting). This is kind of a nice grouping of an infrastructure element as the composition's children have no independent existence or use. We can also go further: in View 22 on page 26, everything has been Nested.

In this introductory section, we only look at the element types and how they relate to each other. Later, we will look into finding a sweet spot for using a Node for encapsulating infrastructure details when we discuss patterns. E.g. in



View 14. *Device and System Software Nested in a Node*



View 22. *A maximally Nested Node*

Section 14.2 "A Basic TI Pattern: A Database" on page 63, we will see that it might be handy to restrict ourselves here, to make analysis of the model easier. You have the choice of course whether you want to model these details (and you can go as far and deep as you like). You can Nest or not. It all depends on the use you want to make of your model (in other words: Uncle Ludwig always wins).

7.10 Event, Trigger and Flow

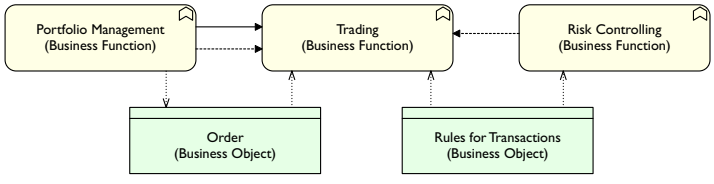
So far, we have handled the structure of and dependencies in your architecture, and all relations so far were so-called 'structural' and 'dependency' relations (explained below). ArchiMate is not big on the dynamics of an architecture, but it does have two 'dynamic relations': Trigger* and Flow.

In View 18 we find Trigger (——→) and Flow (┈┈→) relations in a business layer example. We see three Business Functions here from the asset management world: 'Portfolio Management' is taking investment decisions, which result in orders for the 'Trading' function. So, 'Trading' starts trading when it receives an order from 'Portfolio Management'. Triggering means there is a causal relation between the two functions. The flow between 'Portfolio Management' and 'Trading' says information flows from one to the other. In this case it is the 'Order'.

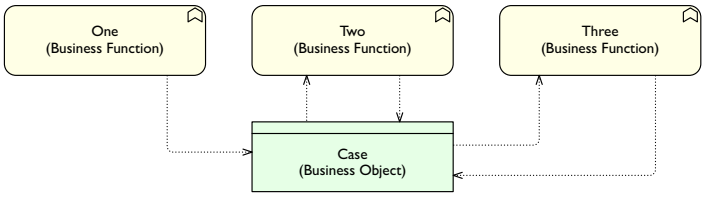


View 18. *Trigger and Flow*

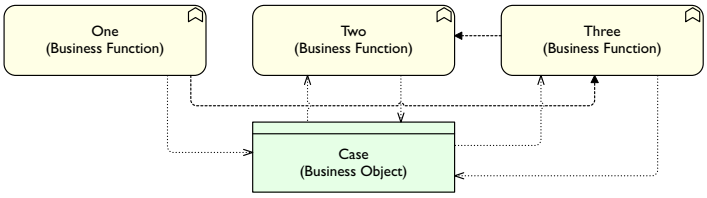
But the Trading function also regularly receives a list of allowed counterparties, countries and currencies from the 'Risk Controlling' function. This information Flows from one to the other, but it does not Trigger a trade. A Flow relation between two Business Functions could also be modeled as Business Objects written (Accessed) by one function and read (Accessed) by another. If we add those, it looks like View 19.



View 19. *Trigger, Flow and Access to Objects*



View 20. *Business Functions sharing Access to a Business Object, without Flows*



View 21. *Business Functions sharing Access to a Business Object, with Flows*

So, how useful is the Flow relation if you can also use the Business Objects and the Access relation? Well, it has a few advantages:

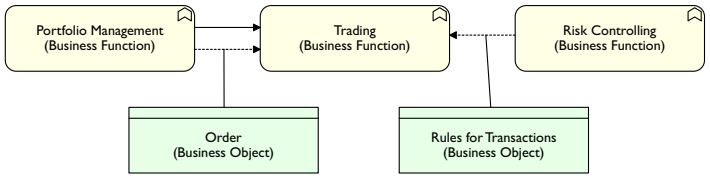
- You can make simpler views by leaving the Business Objects out and for instance label the Flow relations. The problem, though, is that such a label generally cannot be used to analyze what goes on your landscape, I'll say it already here: watch out for relying too much on labels and properties of elements, as they often live outside the 'analyzable' structure of your model.
- Technically, both Access relations do not guarantee a Flow. After all, if two functions access a warehouse, can you say that which one has put in the other takes out?
- But, most importantly, your dependencies can become clearer with a Flow. Take, for instance, the example of a 'Case' flowing though your business from Business Function to Business Function. Having read/write relations from these Business Functions to that single Business Object tells you nothing about how information flows. Take the example in View 20.

  This does not tell you how the 'Case' Flows through your organization. What do you think? Look at View 21 and it becomes clear what the flow of information is.

  Without the Flow relations, would you have known? Could you have drawn the wrong conclusion? Certainly. Is that a bad thing? After all they all depend on that Business Object. Well, take this example: without the Flow relation, you might think that the roles behind function One, Two and Three may have to agree concurrently on all the contents of the Case Business Object. But, in reality, you might only need to set up talks between One and Two on the one hand and Three and Two on the other and depending on the issues at hand, that might be simpler.

The above discussion of Flow is valid for ArchiMate 2 and earlier (I have left this in for didactic reasons). ArchiMate 3 has bridged the gap between Flow and Access, however, by

* Formally it is called the Triggering relation in ArchiMate, but I use the noun Trigger. Flow isn't called 'the Flowing relationship' either.



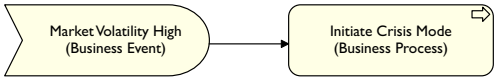
View 24. *Flow with Payload*

allowing relations to relations, which is specifically useful for Flow.

This is shown in View 24. ArchiMate allows us to draw relations to relations. A main reason for this is this construction. Here we have used an Association relation (simple line) between a Business Object and a Flow relation, thus signifying that the Business Object is the payload of the Flow.

Trigger and Flow relations may generally be drawn between behavioral elements within a layer; e.g., from Application Function to Application Function or from Business Process to Business Process.

ArchiMate also has a special element in each layer that depicts an event, 'something that happens' or a 'state'. Business Events can trigger business behavior and they can be 'raised' by business behavior, which is also depicted with a Trigger relation. An example can be seen in View 23. Application Event and Technology Event are likewise available in the other core layers.

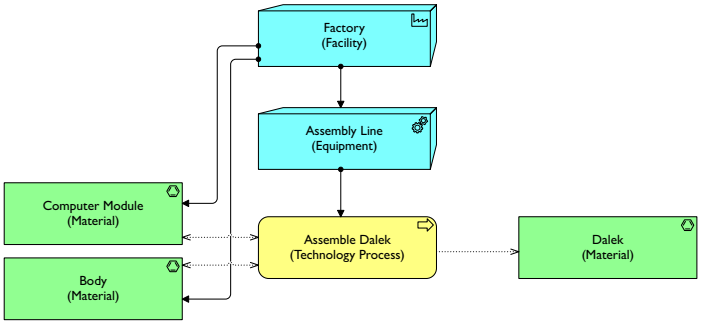


View 23. *Business Event Triggers Business Process*

ArchiMate does not fully support detailed process modeling, it only has a limited support for the dynamics of behavior. In section 29 "A Possible Linking of BPMN and ArchiMate" on page 164, I will present a method to use ArchiMate for EA while using BPMN for process modeling and I will show a bit of actual process modelling in ArchiMate in 29.8 "Business Process Modeling in ArchiMate" on page 184.

7.11 Modeling the Physical

As of version 3, ArchiMate has special elements to model physical aspects of the enterprise. This is a major improvement as it was until this version an enterprise architecture language that was rather narrowly information/IT-focused. But enterprises are not just about information, even if enterprise architecture as a discipline was invented (and still mostly needed) because of the complexity of the overall landscape of information systems.

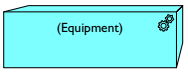


View 25. *Modeling the Physical — Assembling Daleks*

In View 25 a small example of the use of ArchiMate's elements for modeling the physical is shown. What we see here is a Factory that contains an assembly line that builds Daleks (evil cyborgs from the Dr. Who TV series) based on the pre-assembled parts body and brain (computer module). Several new element types are introduced here.

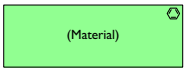


This is a Facility. It is meant to model places where you can deploy technology. Mostly, they will be used for physical production facilities, e.g. a factory. But if, for instance, your process is growing a specific fungus that can be added to raw sheep milk cheese and you need a specific kind of cave for that, such a cave would also be a facility. A facility is something that in the end houses a physical process of sorts.



This is Equipment. Equipment is meant as an active element that can perform a physical process. In this case, we have an assembly line, that is housed in the facility which, we assume, provides power, security, etc., etc.

As you can see, the behavior of the Equipment is modeled as a (generic) Technology Process, which is an element we already encountered before. This means that there is in ArchiMate a separation between physical and computational *structural* elements (subjects and objects in our 'sentences') in the Technology layer, but the behavior (the verbs in our sentences) is not so separated. This and the way these active structural elements are positioned in the language have some interesting effects on modeling, which we will see later. Note: much can be said about the choices that the ArchiMate designers have made. At the end of the book I will discuss these and I will concentrate my criticism there. In this chapter I will — apart from a remark here and there — just report how the language is set up.

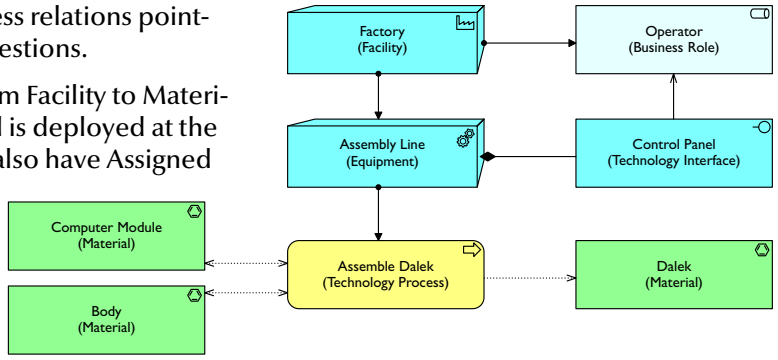


This is Material. Material is the physical sibling of the computational passive element Artifact. It is meant to model any physical material. But unlike Artifact, which can realize active software elements (the Artifact foo.exe realizes the 'foo' System Software for instance), it cannot Realize a physical active element, e.g. it cannot Realize Equipment or Facility (which I think is an oversight, but I will keep my discussion of and critique on the language largely to chapter "Discussing ArchiMate" on page 205).

The Access relations from Technology Process to the Material elements show that the process takes the parts and creates the whole. To model the fact that the assembly process 'destroys' the parts, I've chosen to model those Access relations as read/write. Here we encounter a bit of pure-IT-origin of ArchiMate, because in IT of course, reading is not the same as exhausting something, whereas in physical processes generally the input disappears when the output is created. For those new to ArchiMate (including those that will get exposed to your views without having learned the

language), seeing those Access relations pointing both ways might raise questions.

The Assignment relations from Facility to Material represent that the Material is deployed at the Facility. We could of course also have Assigned them to (deploy them on) the Equipment. Here we encounter already an important modeling issue: is one approach better that the other? The grammar itself is neutral on the issue and while learning the grammar (through certifying or through studying this chapter) is a good thing, it doesn't teach you how to model *well*, how to model for the purpose you are modeling *for*. This, then, is of course the main subject of the other chapters of this book.



View 30. *Assembly Line with Interface and Operator*

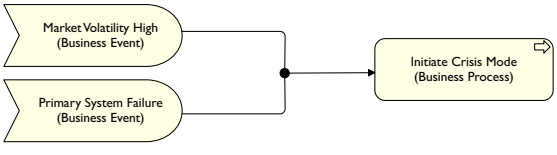
In View 30 I have added the operator and the interface he or she uses to operate the Equipment. I've modeled the operator as a Business Role and the control panel as a Technology Interface. Here too, we see that both the informational/computational and the physical infrastructure use the same element for their interface: Technology Interface. So, here too, the separation between physical and informational/computational does not exist. In section 8.5 on page 35 we will see how everything in the technology layer hangs together.

One final remark: the Technology layer can Serve the Business layer directly. No need for an intermediate application.

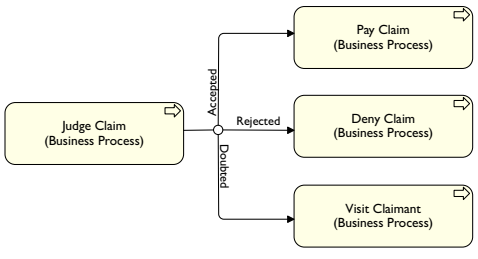
7.12 Junctions

ArchiMate 3's Junction (a 'relationship connector') has been greatly improved with respect to previous versions of ArchiMate. Up to and including ArchiMate 2, it was restricted to connecting Flows and Triggers. Now, it can also be used to logically combine Assignment, Realization, Association, Influence (see page 45), Access, and Serving (all relation types to a Junction must be the same type, though), and it comes standard in two forms: AND and OR. Note that in ArchiMate the OR must be understood as XOR, the *exclusive* OR, meaning that only one of the possible connections is valid at the same time. An example is shown in View 28.

Shown here is an asset management organization's rule that if a system failure happens in times of market volatility, the *combination* of these events trigger the initiation of the crisis mode processes. An example of the XOR Junction can be seen in View 29. Shown here is an insurance firm's rule that a claim is evaluated, after which it *either* goes to the payment process, *or* the 'deny claim' process, *or* evaluation is extended with a 'visit claimant' process.

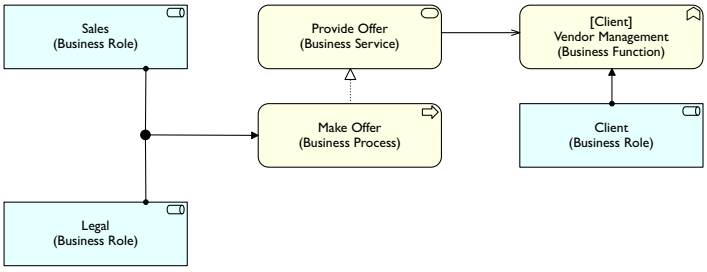


View 28. *AND Junction*



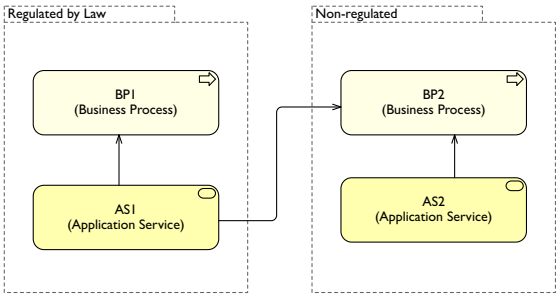
View 29. *XOR Junction*

One practical use of Junction is to model collaborations. As we will see below, ArchiMate has special elements for this, but using a Junction is possible since ArchiMate 3 and this makes modeling collaboration a lot simpler. Have a look at View 26. Here we see modeled that Sales and Legal (both Business Roles) *together* perform the 'make order' Business Process that Realizes the Provide Offer Business Service



View 26. *Collaboration through the use of Junction.*

which is used by the Client's vendor management Business Function. ArchiMate 3 allows the arrow heads at the end points of relations that attach to a Junction, but this is optional, so your tool may not support it.

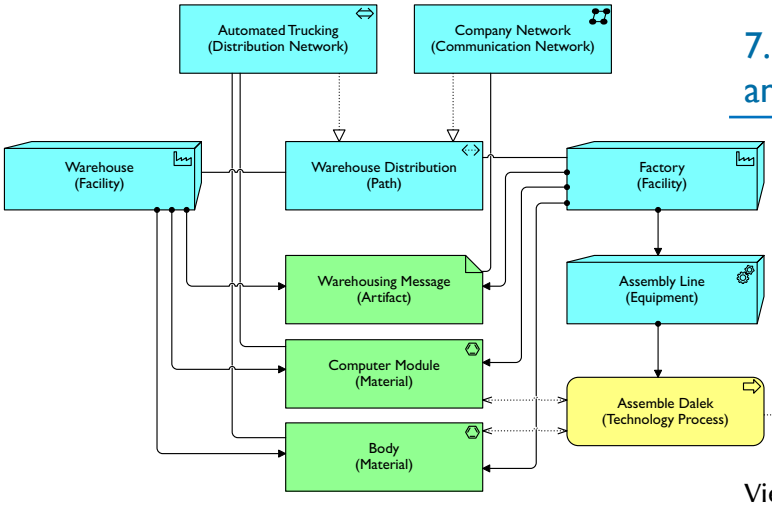


View 27. *A Grouping*

7.13 Grouping

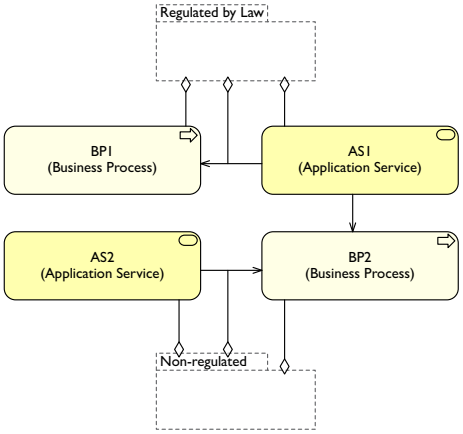
Another element of ArchiMate that has been greatly improved in ArchiMate 3 is Grouping. For those already acquainted with earlier versions of ArchiMate: it *was* just a graphical construct (misnamed as a relation), it now *is* a true element that can Aggregate everything else. And it shows as a grouping because you normally present it as a Nesting (see section 7.8 on page 25). An exam-

ple is shown in View 27 on page 28. Here we see two Business Processes that each use an Application service. BP1 and AS1 are regulated (e.g. by the government) and BP2 and AS2 are not. What really is in the model if we un-Nest is



View 32. *Assembling Daleks — with Communication and Distribution*

shown in View 33. Note specifically that not just the elements are (can be) Aggregated by a Grouping, relations can also be Aggregated.

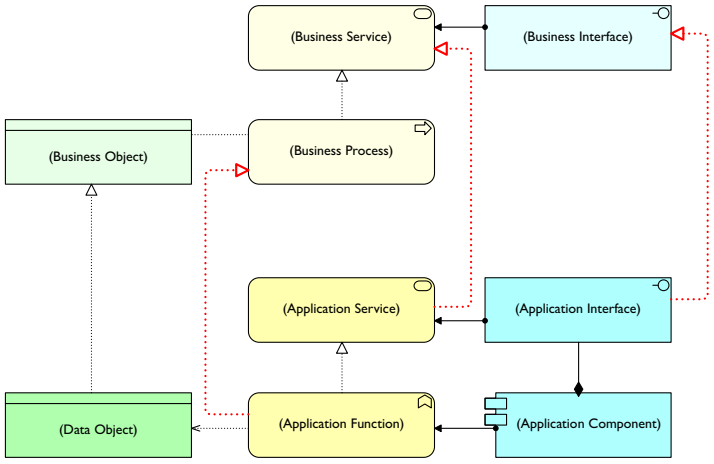


View 33. *A Grouping Expanded*

Just to be precise (so you won't fail an exam on my account on this subject): Grouping is not formally a part of the official ArchiMate Core Framework, it is part of a separate domain 'Composite Elements' which was not shown in the overview in View 4 "The ArchiMate Map – Complete" on page 18, or the two diagrams before it. In my defense of not showing them there: neither does the standard...

7.14 Automated Processes

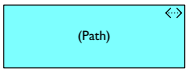
So far, our landscape was based on a business process performed by people (actors fulfilling a role). But what if a process is fully automated? ArchiMate has the following solution: If a process is run by people, a Business Role is Assigned-To the Business Process and a Business Interface is Assigned-To a Business Service. The Business Role *performs* the Business Process. If a Business Process is *performed* by an application, ArchiMate allows us to use the Realization relation to say that the Business layer elements are abstractions of the application (or technology) layer elements. View 31 shows them (in red, only application to business layer). This way, we can model an application that by itself performs a process that realizes a service*.



View 31. *Automated Process*

7.15 Distribution of data and matter (Path and Network)

We have already seen that distribution of information between behavioral elements can be modeled with Flow and/or with Access to passive elements. ArchiMate supports three concepts to model the infrastructure that is required for distribution to happen. One of these is Path, a more abstract element type that can be used to logically model any sort of distribution. The other two are Communication Network and Distribution Network, Extending the example from View 25 on page 27 to View 32 on page 29, we can show all three.



This is a Path. A Path is a kind of abstract element that stands for any form of distribution, physical, information or both. In the example it is used to model the distribution of parts and information about the parts from the warehouse to the production facility. The (logical) Path is Realized by two networks:



This is a Distribution Network. A Distribution Network stands for any setup to handle *physical* transport. It could be a fleet of trucks, a railway setup on the organization's premises, people running around as couriers with packages, and so forth. In this case, the warehouse-factory complex has a fleet of small autonomous trucks that automatically drive between warehouse and factory delivering the parts from which a Dalek will be assembled.

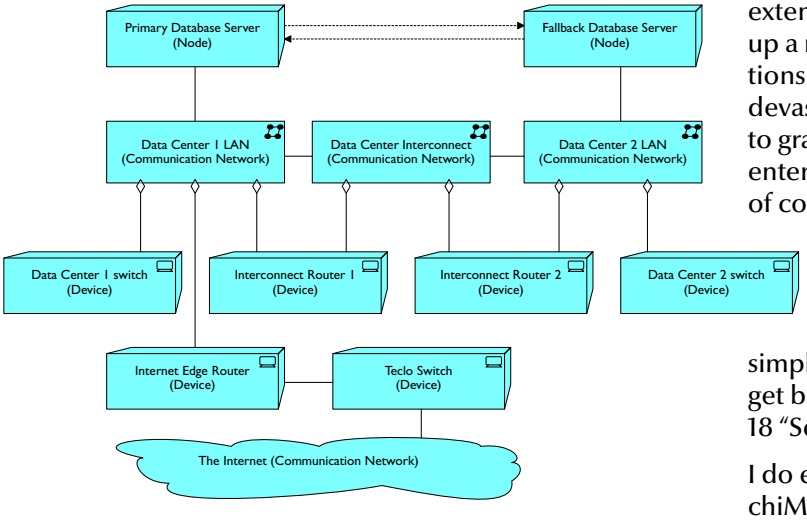


This is a Communication Network. A Communication Network stands for any

* This is fundamentally different from the way it was done in previous versions of ArchiMate, which allowed direct Assignments of Application Components to business behavior. These are no longer allowed.

setup to handle *communication* transport. Most commonly this will be a digital data network, According to the standard, it represents 'the physical communication infrastructure' (with Path being the element that represents the 'logical' (communication) infrastructure. There is some vagueness here, because what is 'physical'? Communication networking is in reality a stack of technologies, with physical cabling (or the ether) as the only really physical part, in network engineering terms 'layer 1'. Your organization's LAN will generally have on top of that (switched) ethernet (layer 2), (routed) Internet Protocol (layer 3), and more.

One new relation is shown, that is the Association relation – a simple line – about which we will say more below. Here, it is used to associate Facilities with Paths, and Networks with passive elements that are transported by them.



View 34. *Using Aggregations to Model Networking Devices*

Another use (not shown) is associating a Device or System Software with a Communication Network.

Networking components may have two types of relations with their environment. As mentioned above, Association is used to show which elements *make use of* the network, and Aggregation is used to model the elements that *create* the network. An example is shown in View 34. Here, two database servers in two data centers form a cluster of sorts, but only Data Center 1 is connected to the Internet. The example is rather old-fashioned, as these days we might see multiple data centers configured as one large 'stretched fabric' (not routed at all, but switched).

As an aside: the example also shows something else: I've changed the graphic shape of the 'Internet' Communications Network. ArchiMate doesn't forbid that, I can do graphically what I want, the visual forms are just 'practice' or 'default', it is the *model* structure that counts.

Anyway, I seldom saw these networking concepts used before ArchiMate 3. For the new elements, this is no surprise, obviously. A reason for the lack of use of the Path/Network elements that have been in ArchiMate from the start, I think, is the following: For the application people, it's all just 'networking', or 'infrastructure', but for the networking people there are a lot of software systems involved to make all that transport happen. The elements are thus not very usable in the reality of this part of the business. For the infrastructure people, the actual devices and systems that make up networking (and the management of said networks) are more useful to model. So, the language may position 'networking' as 'technology/infrastructure' and not 'business' or 'application' (for the business), the infrastructure people will rightfully disagree. The Networking element itself then becomes some sort of abstraction/decoupling point in a model.

Behind this discussion are views on the way we should practice enterprise architecture. Are we focused on the business and is 'technology/infrastructure' an unimportant low-level aspect of it? Or does enterprise architecture extend all the way down into all the complexities that make up a modern enterprise? Are we making simple abstractions for strategic business purposes, ignoring all (possibly devastating) detail? Or are we using the modeling language to grapple with all the complex dependencies in a modern enterprise? My personal perspective is the latter and that of course permeates this book. This handling of different perspectives/uses is something that requires attention when we model, because modeling an enterprise from just the 'primary business' perspective is something that is in my view too simplistic for the goals of enterprise architecture. We will get back to this aspect of modeling enterprises in Section 18 "Secondary and Tertiary Architecture" on page 104.

I do expect, though, that – thanks to the changes in ArchiMate 3, such as the physical, the big influence of innovations in IT infrastructure and the rise of the 'Internet of Things' – the Network and Path elements might start to play a role in ArchiMate modeling, though it is a question whether they are actually needed to model networking at all. After all, we could model any network as a Technology Service/Interface used by other elements of the physical or communications infrastructure. This will be shown in Section 27.1 "Networking" on page 150.

7.16 Who is in Charge?

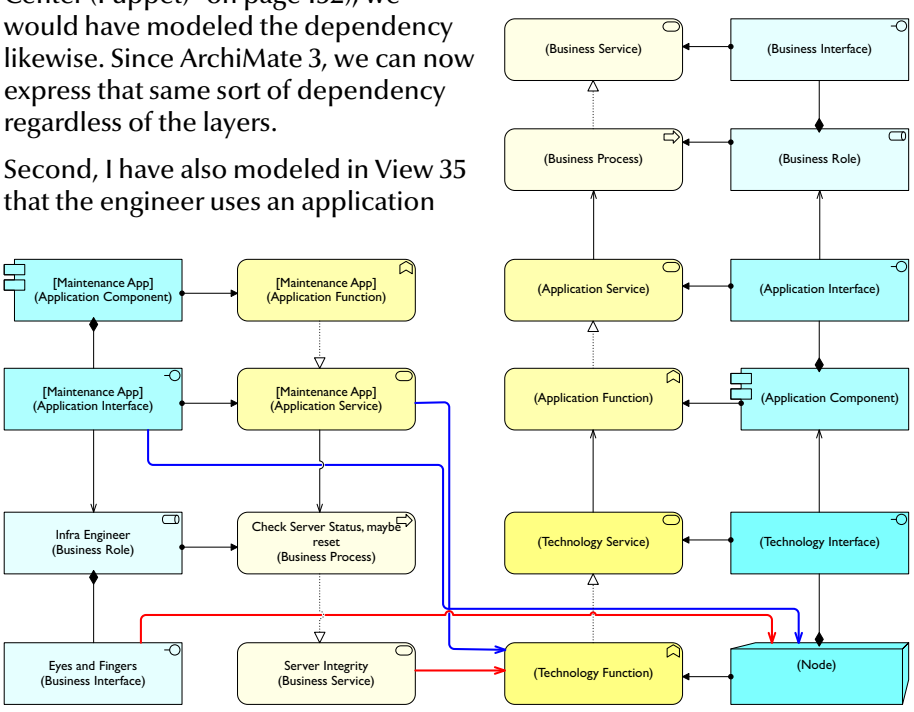
So far, ArchiMate looks like a very orthodox IT-oriented enterprise architecture modeling language. It can show how the business is supported by applications which are supported by infrastructure. Physical processes and materials have been added to 'infrastructure' (which has been renamed to 'technology') but we still are in effect talking about a classic BAT-stack (Business–Application–Technology). From ArchiMate 3 on however, this strict layering has been loosened in an important way. This is an important improvement, because it was always a rather difficult puzzle to model complex stacks, because of the strict one-way layering.

I've illustrated this in View 35 on page 31. On the right hand side, we see our classic BAT-stack. There is a Business Process performed by a Business Role, the process and the role are supported by an Application Service and Applica-

tion Interface which are the visible parts of the application. The application itself is supported by a Technology Service and Interface (runtime environment, file shares, database, etc.). Everything is as we already saw in View 5 on page 20, View 9 on page 22, and View 10 on page 23 but, in View 35, something is added on the left hand side.

Here, two things have been added. First, there is a Business Process that checks the infrastructure every day and that restarts the server if certain criteria are met. The scenario here is that we do not want to restart if we can avoid it but sometimes we have to. This is done by an infrastructure engineer. Now, instead of modeling that the infrastructure engineer also uses the infrastructure, we have modeled that the infrastructure depends on the engineer's process directly (to perform well). This is shown in red. Let's stress this: instead of a human – the infrastructure engineer – using the infrastructure, it has been modeled as the infrastructure using the engineer to remain operational. If the engineer would have been some automated infrastructure (automated infrastructure will return in Section 27.2 "DevOps Ready Data Center (Puppet)" on page 152), we would have modeled the dependency likewise. Since ArchiMate 3, we can now express that same sort of dependency regardless of the layers.

Second, I have also modeled in View 35 that the engineer uses an application



View 35. *Technology uses Application and Business*

to manage the infrastructure. The engineer's application is used by (is Serving) the infrastructure (in blue).

In View 36, you see the blue route from View 35, and it illustrates that the stack in ArchiMate 3 is not only BAT, but also TAB (it is also TBA, TA, BT, BATABAT, etc.). And it nicely shows how, via applications and technology) the primary business process depends on the infrastructure maintenance process. While *technically* it all may depend in a different direction (the IT interface is Serving the human), *logically*, the human is Serving the IT. ArchiMate can do both, what is modeled is *your* choice depending on *your* needs.

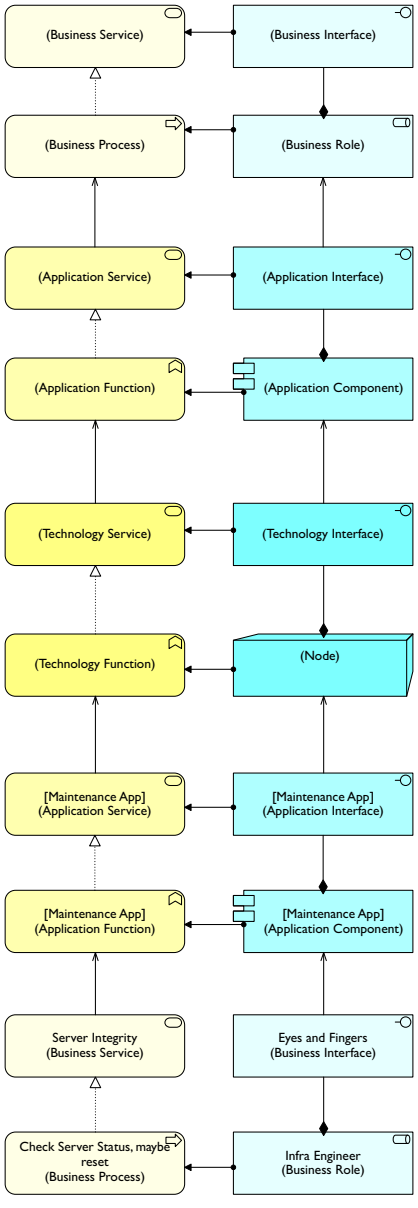
7.17 Abstractions

Abstraction is a favorite tool in the enterprise architect's tool chest. ArchiMate employs and enables abstraction in several ways and for several purposes. Here we will describe some ways ArchiMate 3 supports abstraction.

The first abstraction we already encountered in the first sections and it was shown in View 10 on page 23. It is the abstraction in the 'passive aspect' of ArchiMate's Core. A file (Artifact, e.g. 'answer.doc') may Realize a Data Object (MS Word data) which Realizes a Business Object (e.g. 'Answer Letter'). All three elements stand for the same thing, but at different abstraction levels (which come from data modeling).

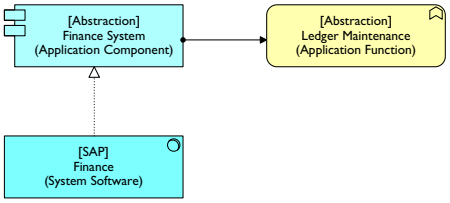
Related to this is the abstraction that goes from Artifact to Application Component (or System Software). The Artifact stands for the *deployment* of software, the bits and bytes, the Application Component (or System Software) for its role as active element in the landscape. This was illustrated in View 9 on page 22 and View 10 on page 23.

The third abstraction is one we encountered previously in section 7.14 on page 29 (View 31): modeling that our business is (partly) automated. There might be something we would like to call a Business Service, but in fact it is provided by automation. A good example is for instance internet

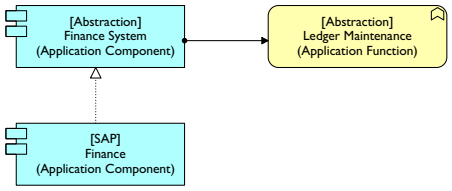


View 36. *Who is in Charge?*

banking. At the business level, the bank provides an 'internet banking' Business Service via a 'web' Business Interface, but in fact all this is IT: applications and infrastructure. ArchiMate 3 allows us to model this by modeling both the application (or technology) elements (such as Application Services or Technology Interfaces) as well as the business layer elements (such as Business Service or Business Interface) and use Realization relations to let the IT elements realize the business elements, thus turning the business layer elements into abstract representations of the IT layer elements.



View 38. *Abstraction Generic-Specific: Application abstracted from Technology*

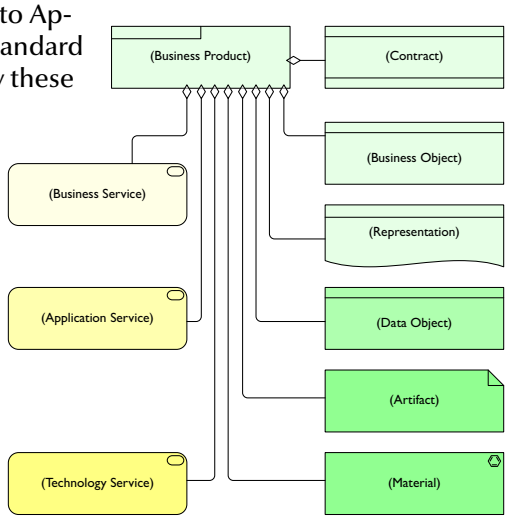


View 39. *Abstraction Generic-Specific: á la TOGAF*

A fourth abstraction runs from the Technology layer to the Application layer. It is possible to create Realization relations from Technology Function/Process to Application Function/Process, from Technology Service to Application Service, and from Technology Interface to Application Interface. Sadly, though, the standard does not really explain what is meant by these abstractions.

An abstraction that is often employed in enterprise architecture modeling is that of generic versus specific IT. E.g. you can have a generic application, such as "the accounting system", but in the real world of your enterprise this is a real application, say the "MainAccount" application from the firm "Pro-IT". ArchiMate 3 supports several ways you could model this*:

- Using the application layer for the 'generic' application while using the technology layer for the specific application. An example is shown in View 38. This might be the intended use of the fourth abstraction;
- Using a Realization relation from respectively Application Component to Application Component or Node to Node. This has been put in at the behest of the TOGAF people (ArchiMate is after all owned by TOG), but apparently The ArchiMate people don't like it enough to fully document it. In TOGAF, by the way, this is confusingly called 'physical versus logical' applications (or technology). 'Actual' (or 'real') versus 'logical' (or 'conceptual') would have been better, I think. Anyway, an example is shown in View 39. Note true 'physical' versus logical is in ArchiMate supported by the deployment scenario from Artifact above.
- Using Specialization within a layer. We will describe this below, when describing the Specialization relation.



View 37. *Product, Contract, Value and a Product's Constituents*

The first one should be avoided, I think. Though the relation is allowed in ArchiMate you should not use it this way, as it means quite something different from what we have in mind now. It is an example of an ArchiMate pitfall which we will discuss later. The latter two have advantages and disadvantages, which we will see later in the book when we are discussing modeling patterns.

The standard also says that an active element (e.g. Application Component) being assigned-to behavior (e.g. Application Function) might be seen as an abstraction, as — in this example — the Application Component is 'implementation' and the Application Function is 'implementation independent'. I think (again) that this is questionable, and we'll discuss this in chapter "Discussing ArchiMate" on page 205.

8. Other Core Elements and Relations

With the elements and relations of the previous section (21 of ArchiMate Core's 39 element types), you can probably do much of an enterprise architect's work, if that work is focused on 'Current State' and 'Change/Project' Architectures. Here is the rest of the Core:

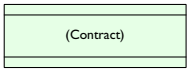
8.1 Product and Contract

If you want to model the offerings of your organization to the outside world (anyone who uses what you produce, be it clients or regulators), a handy element is Product. A Product is a simple enough concept. It is an Aggregation of one or more services and/or passive elements, and (also optionally) a Contract. It is often Associated with a Value. It looks like View 37.

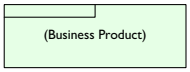
As you can see, the Product element breaks the active-passive-behavior pattern. It Aggregates behavioral elements (the actions themselves) *and* a passive element. It also is a business-layer element, but it may Aggregate an application-layer or technology-layer element. This is because it is a *Composite Element*. ArchiMate 3 has introduced this category, and it consists of Grouping (see section 7.13), Product,

* One is reminded by Andrew Tanenbaum's remark that "the nice thing about standards is that there are so many to choose from".

Location (section 8.7), and Plateau (section 10.2). The new element types are:



This is the Contract. It represents the formal or informal agreement that covers the delivery of the service provided. This might be a specific Service Level Agreement or General Terms & Conditions.



This is the Product. It is what you offer to the outside world. For a department of your organization, the Product may be something offered 'internally'. It can be handy to make the link between your Enterprise Architecture and how management looks at the organization.

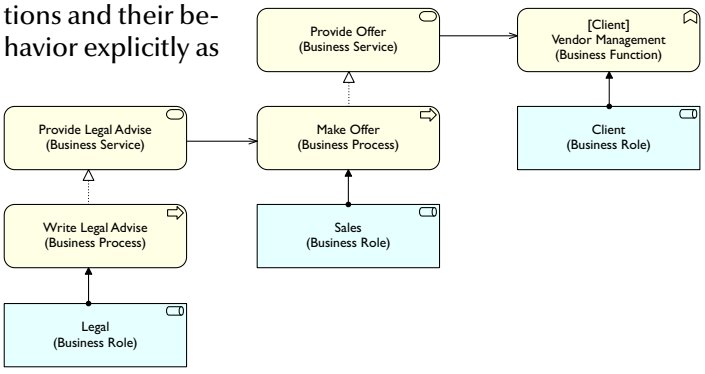
8.2 (Formal) Collaboration and Interaction

Suppose you have two Business Roles in your organization that need to work together to get some specific work done. For instance, the 'sales' and the 'legal' Business Roles need to work together to create the offer for the prospective client. Within ArchiMate there are generally two ways of modeling this.

Using the elements and relations we already described, you can put one of them in the lead and let the second provide a service. In our example: 'sales' produces the offer, but it uses the (internal) services realized by (the processes of) 'legal' in its processes (see View 40).

In this set-up, it is clear who is in charge. 'Sales' decides to send the offer out, not legal. 'Legal' must provide something (and for instance withholding approval means 'sales' cannot proceed), but the service to the client is realized by the process of 'Sales'.

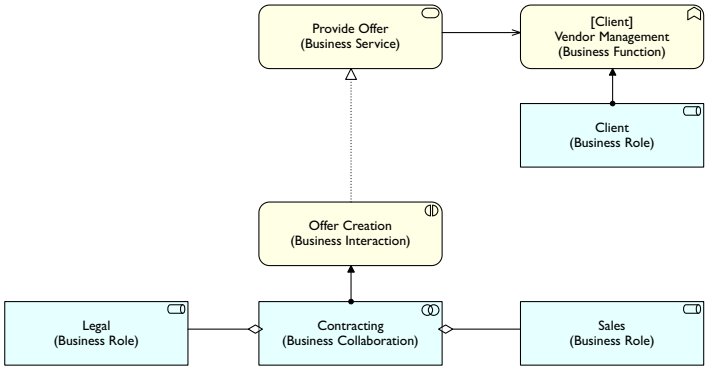
ArchiMate offers a second way to model such collaborations and their behavior explicitly as



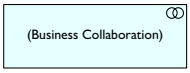
View 40. *Collaboration: Sales Uses Legal on Order Creation*

elements. You can create a Business Collaboration element that consists of (Aggregates) the Business Roles that are part of that collaboration. You can then assign a Business Interaction to that collaboration, the interaction being the behavior of that collaboration. It looks like View 41.

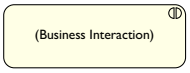
The whole 'offer creation' 'process' is now modeled not as *one* of the parties 'owning' it, but *both*. There are a few new elements:



View 41. *Collaboration: Sales and Legal Collaborate on Order Creation*



This is the Business Collaboration. It is a special kind of Business Role that Aggregates multiple other Business Roles (or Business Collaborations, of course, as these are also types of roles, see Section 8.5 "The Specialization Relation in an Actual Model" on page 35). It is the concept that defines multiple roles forming a (specific) single 'collective' role to do something together.



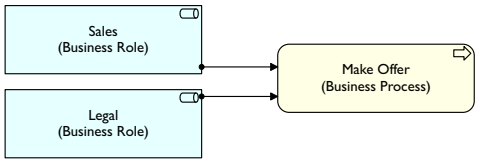
This is the Business Interaction. It is the behavior of that Business Collaboration, the activities they do together. Note that, while the Business Collaboration Aggregates two or more Business Roles, the Business Interaction does *not* formally Aggregate Business Functions or Business Processes. What is not clear in ArchiMate is whether we must see a Business Interaction as functional or process-like. Given that it generally produces a result, it is probably best to see this as process-like.

One consequence of using a Collaboration is that it is not clear who is in charge. This might be more acceptable to the organization in terms of sensitivities, but sometimes it is just true: nobody really is in charge. In Section 17.2 "Business Function or Business Process?" on page 92, I will present a 'modeling pattern' of the business that uses Collaboration to show the loosely coupled nature of some of the organization's 'end-to-end' processes.

Both methods (using a service and using an interaction) are correct; it is a matter of style what you want to use and both approaches have some consequences. If you think about it, you could even model the sales-client interaction as a collaboration, after all the transaction requires decisions of both sides. For this introduction, it suffices that you have seen this.

Let's not forget we already saw collaboration in the enterprise modeled in View 26 on page 28. Here we used the Junction to let two Business Roles perform some behavior together. That way, we also showed that the 'Make Offer' process is assigned to two roles, in fact forming a 'de facto' collaboration.

There is of course a still simpler way. We could just Assign two Business Roles to a single Business Process or Function. This is shown in View 43. There are issues, though. Why is this a *collaboration*? Maybe *either* of the two performs it. The standard generally (but not everywhere) says that it is preferable to look at parallel relations to an element as that either one is 'enough'. But it does not demand this. In chapter "Style & Patterns", when we are discussing modeling patterns, we will discuss this more thoroughly.



View 43. *Informal Collaboration*

In the application layer (and in the technology layer), something similar exists in ArchiMate, as is shown in View 42 on page 34 for the application layer.

The Application Collaboration stands for the collaboration of two Application Components. The behavior of that collaboration is the Application Interaction and such a behavior may realize an Application Service. Here too, it is not clear who is in charge, though as we saw in section 7.2 on page 21, the Serving-pattern is also ambiguous. This is all a matter of patterns and style, and we'll get back to that later in the book. The Technology Collaboration and Technology Interaction are similar, but in the Technology layer.

Certainly in the Application and Technology layers, ArchiMate's Collaboration and Interaction elements are generally abstractions of what is technical reality. See Section 15.1 "Application Collaboration is an Anthropomorphism" on page 84).

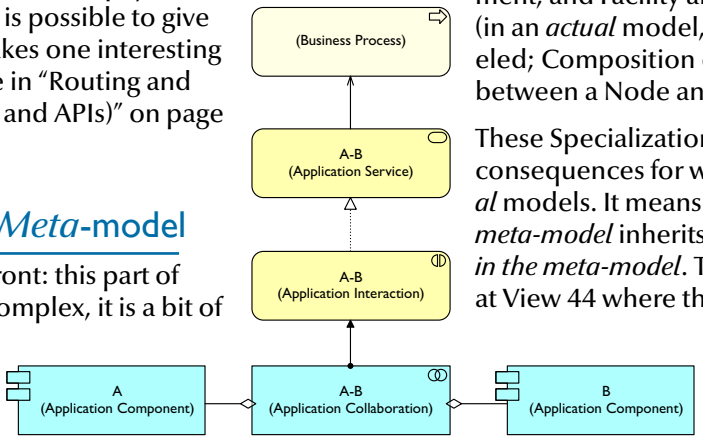
8.3 The Association Relation

The weakest relation, the 'catch-all' relation of ArchiMate, is the Association relation which normally is depicted as a simple line (———). It has a couple of formal roles in ArchiMate, but it also has the role of 'relation of last resort'. If you want to model the relation between two elements, if you know they are related somehow but you cannot model how, you can use Association. It is, therefore, often a sign of lack of knowledge or effort, so it is a matter of style if you want to use it (we'll talk about style later). For now, know that it exists and that it has a few official uses, such as we saw in the relations between Nodes and Paths or Distribution Networks and Material or between 'payload' and Flow. Since ArchiMate 3.1 it is possible to give it a direction (———>) which makes one interesting use more elegant, as we will see in "Routing and Orchestrating Middleware (ESB and APIs)" on page 129.

8.4 Specialization in the *Meta*-model



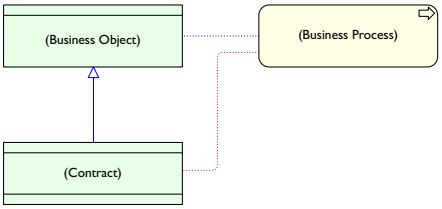
I have to warn you up front: this part of ArchiMate is not only complex, it is a bit of a mess. I'm going to explain this, but you should not give up if this part initially gets too convoluted or complex. In the end, the actual use becomes not that difficult and the language remains of practical use.



View 42. *Application Collaboration*

The Specialization relation (———▷) is probably the most complicated (some would say, 'confused') relation in ArchiMate, because it is used in two fundamentally different ways that influence each other.

Before going on, let me first clearly describe the difference between an ArchiMate *model* and ArchiMate's *meta*-model. A *model* is what you create using ArchiMate, e.g. a model of your organization's current landscape, or a model that describes the outcome of a transformation such as a project. The ArchiMate *meta*-model is a model of how the different element types and relations together form the (grammar of) the ArchiMate language *itself*. This book (this chapter above all) explains the *meta*-model so you can use it to make *actual* models.



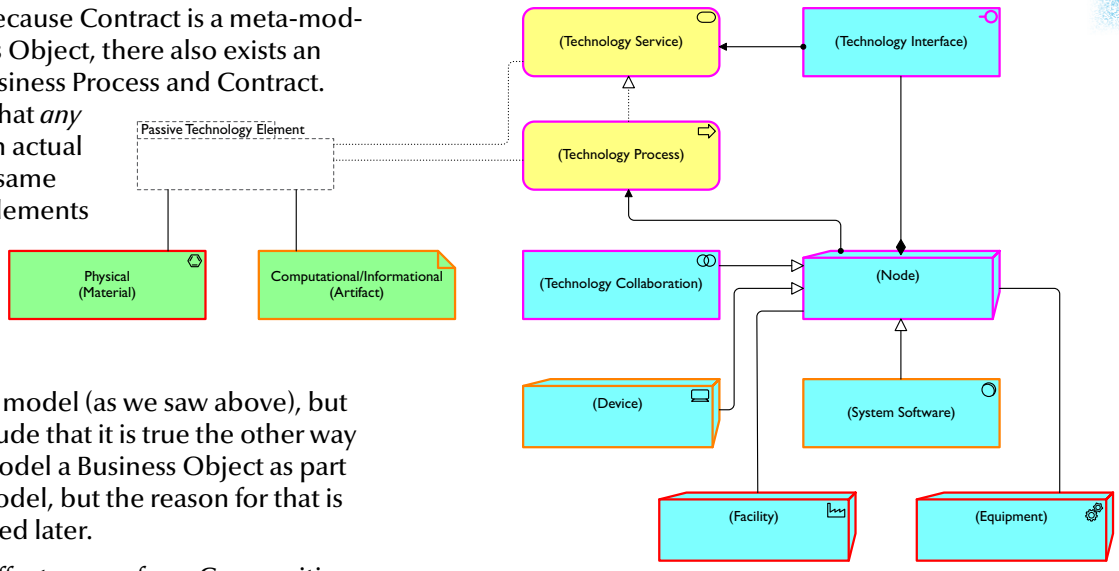
View 44. *Specialization of Contract in the Meta-model*

Returning to Specialization, Specialization plays several roles inside the ArchiMate *meta*-model:

- In the *meta*-model, Contract is a Specialization of Business Object (in an *actual* model, Contract plays an independent role);
- In the *meta*-model, Business Collaboration is a Specialization of Business Role (and in an *actual* model *Aggregates* Business Roles)
- In the *meta*-model, Application Collaboration is a Specialization of Application Component (and in an *actual* model *Aggregates* Application Components)
- In the *meta*-model, Technology Collaboration is a Specialization of Node (and in an *actual* model *Aggregates* Nodes)
- In the *meta*-model, Device, System Software, Equipment, and Facility are all Specializations of Node (in an *actual* model, *this* Specialization is not modeled; Composition or Aggregation are often used between a Node and the 'IT subtypes').

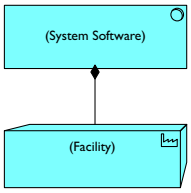
These Specializations in the *meta*-model have consequences for what can be modeled in *actual* models. It means that the child concept *in the meta-model* inherits all the relations of its parent *in the meta-model*. To illustrate this, have a look at View 44 where the place of Contract in the meta-model is illustrated. The blue relations are in the meta-model. Because an Access relation exists between Business Process

and Business Object, and because Contract is a meta-model Specialization of Business Object, there also exists an Access relation between Business Process and Contract. It also means, for instance, that *any* Business Collaboration in an actual model can have *exactly* the same relations to other types of elements that a Business Role can, e.g. being Assigned-To a Business Process. Note that the opposite is not true. A Contract may be Aggregated by a Product in an actual model (as we saw above), but from that we may not conclude that it is true the other way around. Actually, we may model a Business Object as part of a Contract in an actual model, but the reason for that is different and will be discussed later.



View 45. *How the Node ties informational and physical together in the meta-model*

One important secondary effect comes from Composition and Aggregation. As *any* element type in the meta-model can Compose or Aggregate itself and because *any* child type is also a parent, the parent type can Compose or Aggregate each child type. We already employed this earlier when, since Device is a subtype of Node, we modeled a Device as Composite part of a Node (as in Section 7.9 "Using a Node to encapsulate infrastructure" on page 25.



View 46. *Silly Composition*

But as any child type is *also* a parent type, any child type can Aggregate or Compose *any* member in the whole 'family tree', the parent type and all its children, grand-children and so forth. This leads to weird consequences such as: (non-physical) System Software may Compose (physical) Facility, as shown in View 46. Grammatically, you are allowed to do this in ArchiMate. There is no constraint but it is probably unwise to do so.

Another *seemingly* weird effect of this also takes place in the Technology Layer. In View 45 all the child types of Node, both physical and computational/informational are shown. I have outlined the physical elements in red, the informational/computational ones in orange and the neutral ones in violet. Look at the right hand side first. Here you see that performing behavior by active technology elements has been made possible by the Assignment between Node and Technology Process. The same is true for the interface. Technology Interface is defined as a Composite child of Node. The result of the Assignment between Node and Technology Process is that *any* child type of Node may be modeled to perform *any* Technology Process. This in itself is not problematic: it is after all *up to us* not to model apparently crazy things such as "System Software saws wood". If we take the left hand side into account, there is an additional effect. On that side I had to show a 'hidden' element, one from ArchiMate's meta-meta-model, an abstract one (one that never gets actually used in a real model).

In the meta-model, Material can reside on (Assignment) Equipment and Artifact can reside on (Assignment) System Software. But through Technology Process, System Software can Access Material. So, the language does not stop us to model (informational) System Software or Device *accessing* (physical) Material while it does stop us from (physical) Material being *deployed* on a computational/informational system (below we will see it actually doesn't, but for this stage of the explanation it is true enough). This is not so weird as it may look. Suppose the tool for sawing is not a simple hand saw, but an automated 'sawing machine'. Is it that strange to say that its software 'performs' the sawing, together with the rest of the equipment? If not, then "software saws a tree into planks" is also not that strange after all, it is a modern day equivalent of "human saws a tree into planks", which also is rather impractical without the actual saw. So, this is useful: a lot of our 'things', from factory robots to washing machines have become automated and depend on software after all.

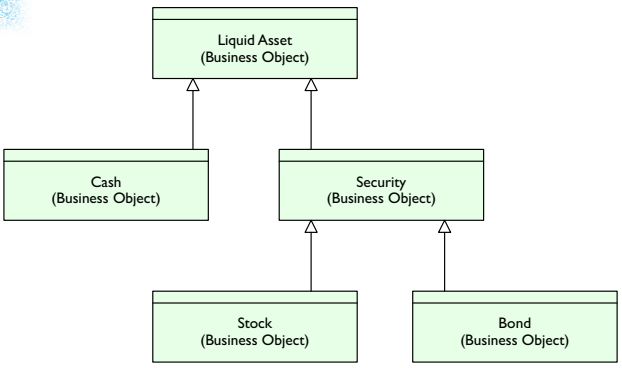
Summary: the Specialization relation *in the* meta-*model* is like a 'class-relation', which are well known from software engineering. The Specialization relation *in the* meta-*model* says that an element *type* is a kind of another element *type* and an instantiation of it can do whatever an instantiation of its parent can.

8.5 The Specialization Relation in an *Actual* Model



When you use Specialization yourself, you will be using it in actual models. In an actual model, Specialization can also be used with a sort of 'inheritance' in mind. For instance, a 'car insurance' and a 'travel insurance' are both a kind of 'insurance'. Or a 'stock' and a 'bond' are both a kind of 'liquid asset'. An example is shown in View 47.

Here, we say that 'Cash' is a kind of 'Liquid Asset' (a 'liquid' asset is an asset that you can easily trade) and another kind is 'Security' which again can be specialized into 'Stock' (a deed to a partial ownership of a company) and 'Bond' (a loan to an organization or country). If an element is a

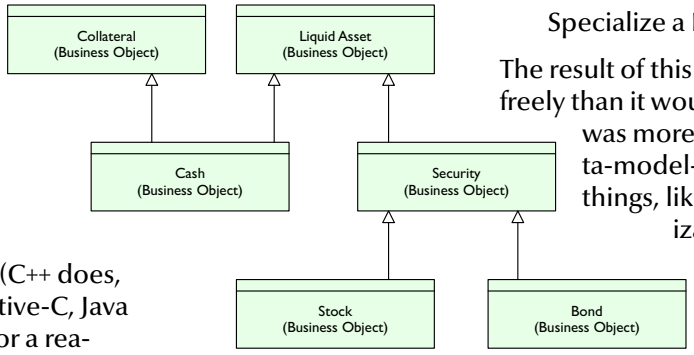


View 47. *Specialization Example: Investment Types*

specialization of another element, whatever relations are true for the 'parent' (the element at the arrowhead) must be true for the child. If a Business Process handles 'Liquid Asset' (e.g., a reporting process), it must be able to handle both 'Cash' and 'Security'. On the other hand, if a business process handles 'Security', it must be able to handle both 'Stock' and 'Bond' but it does not need to be able to handle 'Cash'. Importantly, this interpretation, is *not* forced upon us by ArchiMate itself.

In Object-Oriented (OO) design, the specialization is sometimes called the 'Is-A' relation. Its counterparts are the 'Has-A' relation, which in ArchiMate is Composition (◆——) and the 'Refers-To' relation, which in ArchiMate is Aggregation (◇——). Generally, what you can do with an 'Is-A' Sub-classing (which is what you do with the Specialization relation in the meta-model), can have complex consequences. Take for instance 'Cash' from the current example. Suppose our company says we may only use cash as collateral for securities we lend (and thus not (other) securities). We would have a new Business Object in our landscape called 'Collateral' and 'Cash' could be a kind of 'Collateral'. It looks like the example in View 48.

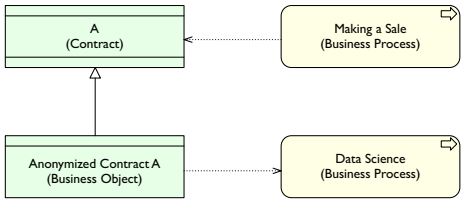
What we have here is called in OO-terms 'multiple inheritance': Cash is both a kind of collateral and a kind of liquid asset. Most OO languages do not support true multiple inheritance (C++ does, more modern Objective-C, Java and C# do not) and for a reason: it tends to become messy because the 'parents' may have conflicting rules of behavior for the 'child' to inherit (a bit like parenting in real life, true, but maybe not the best paradigm for software engineering or business architecture modeling).



View 48. *Specialization Example: Multiple Inheritance*

But ArchiMate's Specialization relation *as used in actual models* is not like that. If we have a Business Process that Accesses 'Liquid Asset', we may formally *not* conclude from the model that it Accesses 'Cash' or 'Security'. That relation we must explicitly model ourselves. Nobody does this in practice, but technically we should.

That Specialization in *actual* models is different, is driven home by the fact that it is a relation that is allowed between 'instances of the same type'.



View 50. *Model-Specialization (versus Meta-model-Specialization)*

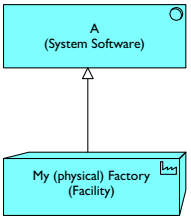
We already saw this. In View 47 on page 36 and View 48, we saw Business Objects Specialized into Business Objects. This *model*-Specialization' is inherited by child types via '*meta-model*-Specialization'. This is best illustrated with a simple example. Have a look at View 50. Here we see a model-Specialization of a Business Object that is a Specialization of a Contract, which is the reverse of what the meta-model-Specialization is between both concepts. Modeled here is a Business Object that is an anonymized version of a Contract so it can be used for analytics. To illustrate that, both Business Processes are shown. From a software engineering perspective this Specialization is problematic, for instance, it does not adhere to the 'Liskov substitution principle' (the child type can always replace the parent type in a specific situation where the parent type plays a role). In this case: we cannot use an 'Anonymized Contract' as object for the 'Making a Sale' process.

This way of modeling is possible because:

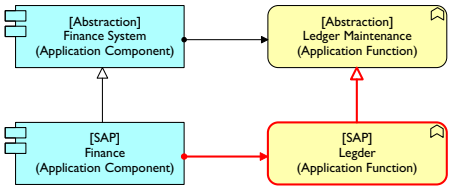
- Any element type can model-Specialize itself;
- Hence: Business Object can model-Specialize Business Object
- This model-Specialization is inherited by Contract through: Contract is a *meta-model*-Specialization of Business Object;
- Hence: Contract (is a Business Object and thus) can Specialize a Business Object.

The result of this is that ArchiMate allows us to model more freely than it would if the model-Specialization relation was more 'software engineering'-like (or more meta-model-Specialization-like). It allows us to model things, like the almost certainly silly model-Specialization in View 49. This is allowed because any Node can model-Specialize itself and because System Software and Facility are both meta-model-Specializations of Node.

None of this is properly documented in the standard (e.g. when the model-Specialization relation is defined), though it can be implicitly concluded from the way ArchiMate omits a role for any kind of Specialization in the model-'derivation' of relations (derived relations will be explained in Section 9 "Derived Relations" on page 39). Truly,



View 49. *Silly Specialization*



View 53. *Abstraction Generic-Specific via the Specialization relation*

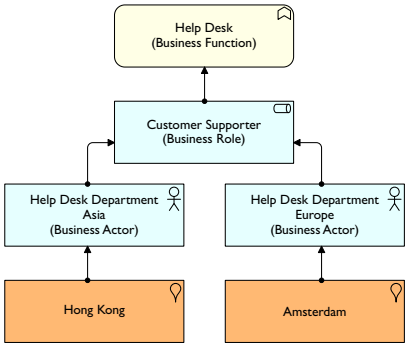
this is not ArchiMate's clearest and most elegant part.

There is another aspect to the use of model-Specialization in ArchiMate models. ArchiMate as a language is completely agnostic with respect to 'class versus instance' modeling in actual models. You can make concrete models, where you model real elements, such as how the 'Pro-Fin' financial application supports certain business processes and depends on certain infrastructure.

But you can also make abstract models, where you use elements that represent a 'type' or 'class', e.g.:

- A model valid for any finance application of which there are supposed to be several in your landscape. Or a model where you don't model every single desktop in your organization, but just a generic 'Desktop PC" or "Tablet" element. Or more specifically a "Windows Desktop" or "Apple iPad" end user device. You could of course also use Aggregation here to connect the 'class' to its 'instances').
- A model where your parent doesn't necessarily represents a collection, but where you are modeling 'conceptual' versus 'actual' elements in your landscape, as in section 7.17 "Abstractions" on page 31.

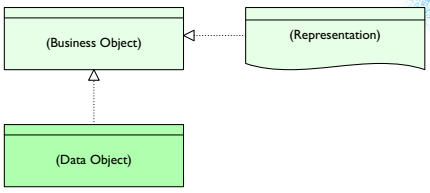
An example of the latter is shown in View 53. Here, the same generic-specific relation is shown as in View 38 and View 39 on page 32. I've added something extra: the element and relations (outlined) in red. Here is a question for you: would you add them or not? Some say you shouldn't, as the function itself is an abstraction of the implementation (in this case the Application Component). In fact, the ArchiMate standard's text suggests as much. Others would argue that the real behavior of the real system is not an abstraction at all, but more an aspect of the system. There are, after all, no two systems in the world that behave exactly the same, and ignoring that can be perilous. We're back in 'modeling pattern' territory here, and such discussions are for the rest of the book.



View 52. *Example use of Location object*

8.6 Representation

Earlier we encountered the Business Object, the work horse of passive structure in your business layer architecture. We saw how this business-level element could be Realized by an application layer Data Object. Take for instance the 'Answer' Business Object that was Realized by the 'Word Document' Data Object in View 10 on page 23. ArchiMate also has another way to Realize a Business Object, it can be realized by a Representation. A good example would be a print of the answer letter you are sending to your customer. The relation between a Business Object and its direct surroundings looks like View 51.



View 51. *Representation and Data Object Realizing a Business Object*

The new element types are:



This is the Representation. While the Data Object is the element in our application layer that represents the Business Object, the Representation is another way of representing the Business Object and it stands for a representation that can be shared with others, e.g. a print, a PDF file you send as attachment in a mail message, or a web-page.

Representation was sometimes a useful link to the physical world in pre-version-3 ArchiMate (e.g. for the actual real Steel Beam we mentioned in 7.1 "Application and Business" on page 18), but as far as I'm concerned it is a strange leftover of days where the IT/application-centric ArchiMate, designed with administrative systems in mind, needed an element to model information on paper. It has no relation to the physical domain in ArchiMate 3 and its description has always overlapped with Artifact (PDF *'file'* after all).

8.7 Location

The final Core ArchiMate element to explain is Location. This was introduced in ArchiMate 2 and an example is given in View 52 (note the informal 'collaboration').



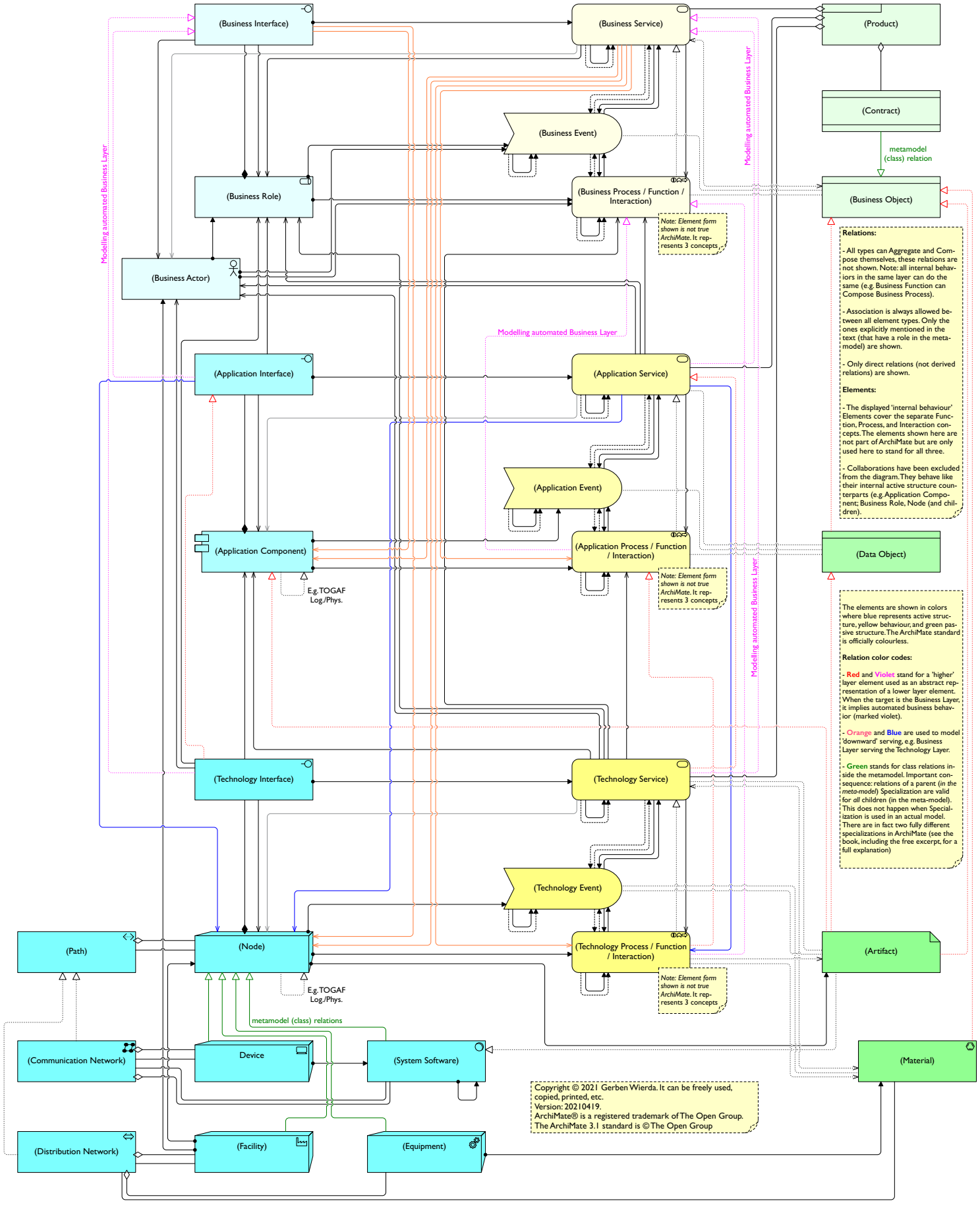
This is the Location element. This element stands for a geographical location where something resides. You can use the Aggregation relation to link it to several other element types to model where they are located.

Again, for reasons of correctness: Location is not part of the ArchiMate Core Framework, but – together with Grouping – part of the 'Composite Elements' category, the category of elements that can Aggregate many element or relation types.

8.8 The Complete Core

Without taking the rest into account, ArchiMate's Core is already rich. There are 41 element types (including the Composite Elements) and 10 relation types in Version 3.1.

In View 54 on page 38 you'll find an overall picture of a selection of ArchiMate's Core element types and their rela-



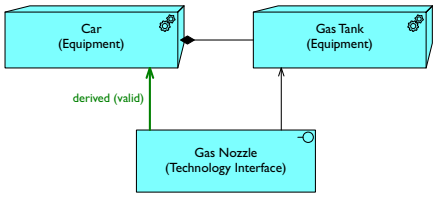
View 54. *Selection of ArchiMate 3.1 Core meta-model in 9 colors, direct relations only*

tions. This is often referred to as the ArchiMate meta-model. Note:

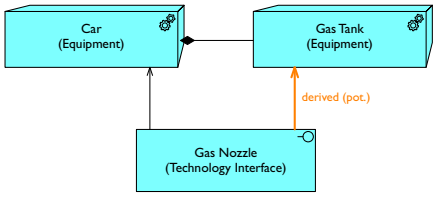
- Process, Function, and Interaction take the same place and are represented in the diagram by a formally non-existing element;
- The red relations are those that are used to model automated business layer elements. E.g. the Business Service 'web site' that is technically in reality an Application Service provided by IT. See section 7.14 "Automated Processes" on page 29.
- The orange relations are those that are used to model generic/specific abstractions, such as the generic 'finance application' versus a specific system that is actually used. See section 7.17 "Abstractions" on page 31. For a third way, see section 8.5 "The Specialization Relation in an Actual Model" on page 35.
- The green Specializations are *meta*-model relations. They show that the children are also parents (a Device is also a Node, just like a ball is also a toy). These are not modeled in actual enterprise architecture models, they are shown because you need them to interpret the meta-model correctly. See section 8.4 "Specialization in the Meta-model" on page 34.
- View 54 is repeated on page 235. During the rest of the book, you might want to look back at the meta-model to check for things. If you keep your finger between that page and the one before it, the core of the meta-model is always readily available (no such ease for the PDF users, I'm afraid).

9. Derived Relations*

So far we have included every concept, every element type and all connecting relations of the Core. ArchiMate however, offers a mechanism to create 'shortcuts'. The researchers, business users and students that created ArchiMate even offered a mathematical proof that their definition of the shortcuts are 'correct', in the sense that there is not a runaway one which in the end creates the situation that everything can be related to everything else using every kind of relation. Adding these shortcuts to the model was originally seen as extending 'ArchiMate proper' to a sort of 'ArchiMate+', but these shortcuts are so handy and useful that most practitioners (or teachers) do not really distinguish between the direct and shortcut relations anymore (which is a bad thing, as we will see). Add to that, tools generally do not support the actual difference (well) and it is easy to understand that the shortcuts are probably one of the most (implicitly) used but least (explicitly) understood aspects of ArchiMate. It did not help that for a long time (and because the text was full of omissions and ambiguities), the ArchiMate people stated that the real definition of the language was in the appendix of the standard where all possible relations are given, and that 'all relations mentioned there were equal'. ArchiMate 3.0.1 went a long way in strengthening the standard on this point and was the first version to be adequate on this point.



View 55. *'Valid' derivation*



View 56. *'Potential' derivation*

Officially, ArchiMate calls these 'shortcuts' *derived relations* and I generally explain them by saying that the derived relations are a summary of a 'dependency route' that lies between two elements in a model. All tools have to allow you to model such a 'summary' without modeling the underlying 'true' route of elements and relations, but in the end each summary relation implicitly assumes that such a route with real elements and relations is there, even if you have not (or need not have) created them in your model.

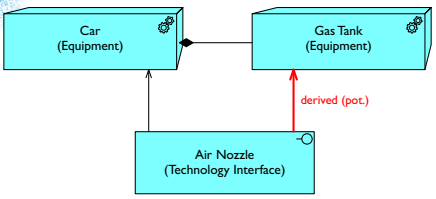
As of ArchiMate 3, the relations are divided into the following categories:

- Structural Relations (Composition, Aggregation, Assignment, Realization)
- Dependency Relations (Serving, Access, Influence — which we haven't seen yet, but we will later on —, Association)
- Dynamic Relations (Trigger, Flow)
- Other Relations (Specialization — a category by itself).
- A Junction is not a relation but a relationship *connector*.

Between ArchiMate 3.0.1 and ArchiMate 3.1 an improvement has been made of the logic of the derived relations, but at the same time the whole setup has been made much more complicated. Derivations have been divided into 'valid' and 'potential' ones. The (8) 'valid' ones can be relied upon to be valid from the model alone (*certainly* true, though confusingly elsewhere in the standard it is defined as 'with high certainty' which is a bit less absolute), the (15) others *may* be true, but require information not in the model to decide if they are.

Suppose a gas tank is modeled as a Composite child of a car. An example of a (necessary) *valid* derivation might be: If the gas tank can use the gas nozzle, then *certainly* the car can use the gas nozzle. An example of a *potential* deriva-

* Note: this section is about derivation in a *model*. ArchiMate sets out additional rules for the use of derivation in the *meta*-model. Derivation in the meta-model is used to calculate all the allowed relations from the direct relations and the rules. See appendix B of the standard



View 58. *Invalid 'Potential' Derivation*

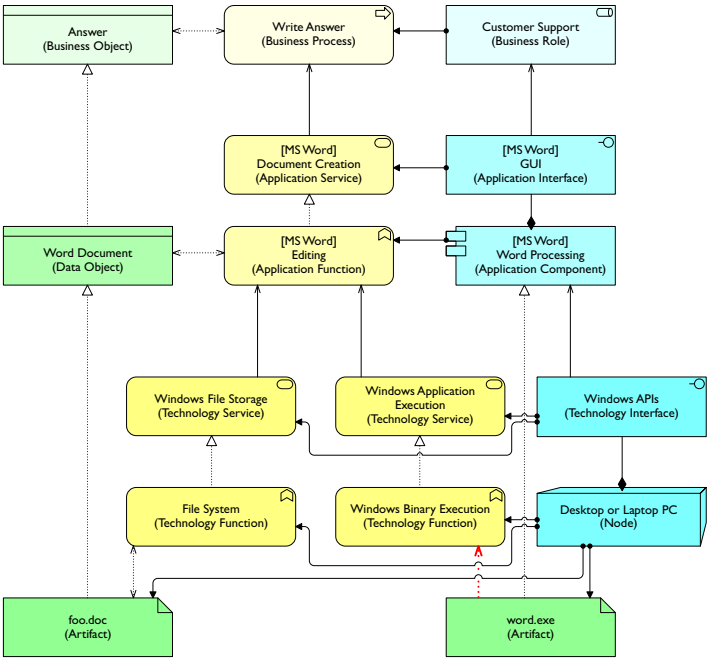
tion might be: If the car can use the gas nozzle, then *potentially* the gas tank can use the gas nozzle, which in this case is actually true. But it is of course not always true.

Have a look at View 58. Using the same 'potential' mechanism, we have derived the red Serving relation from the black relations. But we have not been modeling the gas nozzle, but the air nozzle, which connects to the wheels, not the gas tank, of course. So, while we know the car may use the air nozzle, it is only *potentially* true for the gas tank – and in this case false.

The details are in Appendix B of the standard. It is all rather complicated and it is seldom relevant during modeling (we'll get to relevance and use below). But we will illustrate the mechanism with some examples.

9.1 Typical Application of Derived Structural and Dependency Relations

The best way to illustrate the main derivation rules is by using an already familiar example, the earlier used 'write answer' example. It is shown again in View 57 for easy reference (ignore the redness of one relation this time).



View 57. *Write Answer Process, supported by MS Word and a Standalone PC*

Given this example, the questions we may want to ask are:

a. What is the relation between the 'Desktop or Laptop PC' Node and the 'Write Answer' Business Process?

b. What is the relation between the 'word.exe' Artifact and the 'Document Creation' Application Service?

c. What is the relation between the 'Desktop or Laptop PC' Node and the 'Answer' Business Object?

For this, ArchiMate comes with procedures for creating summaries that are made up from structural and dependency relations:

- Find a route from one element in the model to another following the relations between them.
- For multiple structural relations in succession: Every structural relation has a *strength*. If you have a valid (partial) route consisting solely of structural relations, *all in the same direction*, the derived relation between both ends of the (partial) route is the *weakest* relation found on the (partial) route, much like the weakest link in a chain. The strengths are (from weak to strong):
  1. Realization
  2. Assignment
  3. Aggregation (direction: from parent to child)
  4. Composition (direction: from parent to child)
- For a sequence of structural and a dependency relation, you may move the connecting endpoint of the dependency relation backwards over the structural relation to get a *valid* shortcut. In a sense, the structural 'parent' of the element inherits the dependency relation of its 'child' as in the example above. If you move and endpoint forward over the structural relation, you get *potential* shortcut.
- For multiple dependency relations in sequence, you have a *potential* shortcut that is the weakest of those. The strengths of the dependency relations are (from weak to strong):
  1. Association
  2. Influence (which we will see later)
  3. Access
  4. Serving

Using these procedures as often as we want, we can make shortcuts. Answering the questions above:

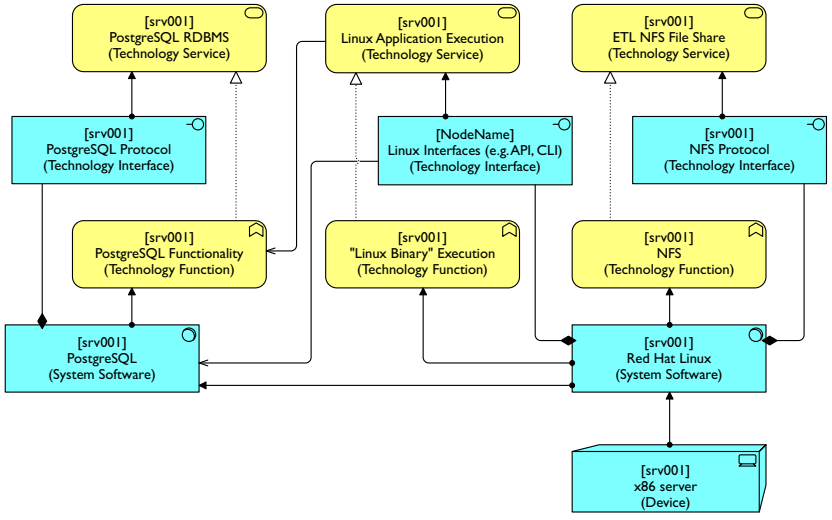
a. From the 'Desktop or Laptop PC (Node)' we can follow the Assignment and Realization to 'Windows Application Execution (Technology Service)' and replace the two structural relations by the single weakest one: Realization. The starting point of the Serving relation from 'Windows Application Execution (Technology Service)' to '[MS Word] Editing (Application Function)' can travel 'backwards' across our new Realization shortcut to 'Desktop or Laptop PC (Node)', we now have a Serving relation from the PC to the Application Function. The starting point of the Serving Relation from the '[MS Word] Document Creation (Application Service)' to the 'Write Answer (Business Process)' can likewise travel backwards to the Application Function along the Realization between Application Function and Application Service. We now have two Serving relations in sequence which we may replace with a single one.

The result is : *the PC Serves the Business Process*, which makes good sense.

b. The 'word.exe' Artifact Realizes (4) the 'Word Processing' Application Component, which is Assigned-To (5) the 'Editing' Application Function, which Realizes (4) the 'Document Creation' Application Service. The weakest of these three structural relations in sequence is Realization, so *the 'word.exe' Artifact Realizes the 'Document Creation' Application Service*. A conclusion which also makes sense.

c. If we take the route from (a) we need just one extra step added to the first answer: from the 'Write Answer' via Access to the 'Answer' Business Object. The weakest of that sequence of two dependency relations is Access, so we have a *potential* shortcut: *the PC Accesses the 'Answer' Business Object.*

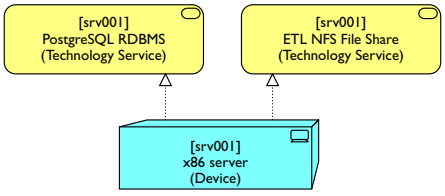
The derivation mechanism has seen quite a bit of change over the years (illustrating how troublesome this all is).



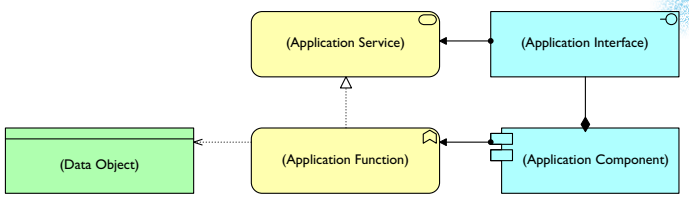
View 60. *System Software and Device with Interfaces*

E.g. before ArchiMate 3.1 the structural and dependency relations (without Association) formed a single set with strengths. That led to conclusions like: "if the wood stack is wet, the plank must be wet". The current setup has fixed this.

Before ArchiMate 2, Assignment was bi-directional. The Open Group's ArchiMate Forum's decision to make Assignment unidirectional in ArchiMate 2 solved the problem that quite a few nonsense derivations were possible. In ArchiMate 1 it would for instance enable the route from the 'foo.doc' Artifact via the PC Node to the 'Windows File Storage' Technology Service with Realization as the resulting relation, or in other words: 'foo.doc' Realizes Windows File Storage. Hmm, I think not. In ArchiMate 3, Assignment's directionality is also shown in the visualization of the relation (from dot to arrowhead).



View 61. *Simplified/derived version of View 60*



View 59. *The Basic Application Pattern*

Anyway, sometimes multiple shortcuts do still exist. For instance, In ArchiMate 3.0.1 there was an alternative answer to question (c): From the PC Node via Assignment (strength: 5) to the 'foo.doc' Artifact, then via Realization (strength: 4) to the 'Word Document' Data Object and via another Realization to the 'Answer' Business Object. The result is: the PC *Realizes* the 'Answer' Business Object. In other words: the PC *Accesses* the 'Answer' Business Object and it also *Realizes* that same 'Answer' Business Object. Here it is clear that we are looking at two aspects of this PC, it both executes the application and it stores the data. If the data were to reside on another server, we would not have both routes. So, the two routes are in fact a true aspect of the situation: the PC does both and so both relations dutifully appear. No problem. Still, ArchiMate 3.1 blocks that route by additional rule (there are 8 of these) that you can only derive Access type relations to passive structure.

So, all is well? It gets a little more complicated. Have a look at the Basic Application pattern, repeated in View 59.

If you go from Application Component to Application Service, you can follow two routes. The first route (via the Application Interface) leads to Assigned-To as the (weakest) derived 'summary' relation. But the one via the Application Function leads to Realization as the (weakest) derived 'summary' relation. ArchiMate does not tell you which one to take if two possible routes exist. That could be OK, as we saw in the previous example from ArchiMate 3.0.1, when two routes stand for two different aspects. But in this case, either derived relation says something about how you view the concept of Application Component. If you choose Assigned-To, you keep to the separation of active component and its behavior. If you choose Realization, you look at the Application Component from a more behavioral point of view. We get back to this confusing issue when we discuss the language and look at possible improvements in Chapter "Discussing ArchiMate" on page 205.

What all of this so far implies is that derived relations are useful to make summary views or abstract representations of a complex landscape, but there are risks involved in using them without (or mixed with) the underlying reality. The risks we have seen so far are still benign. But there are some pitfalls we will see later.

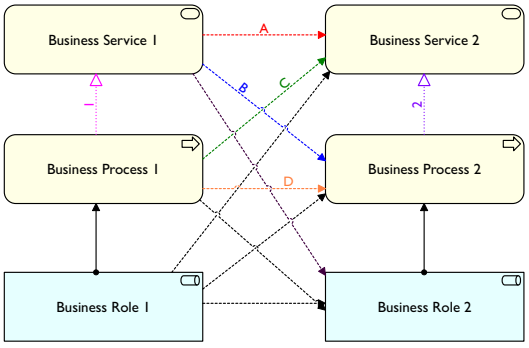
Finally, returning to View 11 on page 24 in System Software and Devices, if we add a couple of Technology Interfaces to that view we get View 60.

You might understand why I did not show this in View 11: very complicated for something as simple as a server that provides a file share and a database. But using derived relations, you can conclude that View 61 is a correct representation of the same. So, if you want to keep it simple: you can just model these. Keeping it simple and using abstractions make your models and views easier for the reader, but there are some caveats. We'll encounter several of those later in the book.

9.2 Derived Dynamic Relations

Since ArchiMate 2 there is also the possibility to create derived relations from *dynamic* relations. The procedure has changed in ArchiMate 3 and now is: you may move the start or endpoint of a Flow relation at an element to the start of a *structural* relation (Composition, Aggregation, Assignment, Realization) that ends at that element. In fact, this change was what prompted the same improved approach for structural and dependency relations in ArchiMate 3.1.

Take a look at the example in View 62 which uses Flow. If we start with Flow A (red) and we move the endpoint of that Flow to the start of Realization relation 2 (purple), we get Flow B (blue). We can also move the start of A to the element at the start of Realization 1 (violet) and get Flow C (green). We can do both in either order and get Flow D (orange). And so forth to produce all the other Flows in the diagram.



View 62. *Derived Dynamic Relations*

For Trigger the procedure is almost the same. There is a constraint: Instead of all 'moving an endpoint up' the dependency of all structural relations you may *only* move it up for Assignment.

The Trigger relation is also transitive: if *a* Triggers *b* and *b* Triggers *c*, then *a* Triggers *c*. This is a conclusion that is not always valid of course. The fact that the Legal Business Function Triggers the Sales Business Function (e.g. when there is an update in the service agreement text for customers), combined with the fact that the Sales Business Function Triggers the Finance Business Function (e.g. when an invoice needs to be sent) does not mean that the Legal Business Function Triggers the Finance Business Function. The logical nature of a rule-based grammar is never able to cover all the possible meanings in real world scenarios.

9.3 Are derived relations real relations?

The ArchiMate 3.1 standard contains a table (appendix B) with all allowed relations between element types. This table is calculated from the direct relations in the meta-model and 'meta-meta-model'* and the derivation rules. It contains all the possible direct and derived relations in ArchiMate and tools must support these to be ArchiMate compliant. All of these are, according to ArchiMate, to be seen as valid relations. Note that the table† in version 3.0 of the standard was horribly broken and this was fixed more than a year later in 3.0.1.

Most practitioners will use the relations in the table as if they are all 'independently real' in a way, that is, not bothering about a relation being indirect and what that means.

Derived relations have subtly changed over the years. The original (seductive, but naive) idea seems to have been that it should be possible to 'calculate' valid summary views from complicated models, thus providing both the designers and the business with a preferred way of looking at the landscape *automatically*, while being sure that both were just different views of the same model and thus coherent‡. Of course, this changed when in reality people started to model using derived relations without bothering about the underlying (implicit) details. As a result, the difference between derived and direct has all but disappeared in ArchiMate modeling circles and the main function for derivation today is to calculate the relationship table from the meta-model and meta-meta model. But some derived relations can only have a meaning if understood through an underlying route of direct relations. For instance, many business processes depend on the local network (The LAN); if the network goes down all hell breaks loose. But no business process uses the network directly, they use applications and these applications use infrastructure. If you draw a Serving relation from a Router to a Business Process (an approach that neglects ArchiMate's official but rather limited Networking elements to model networking, see 27.1 "Networking" on page 150), it is certainly true, but in reality it implies that the intermediary elements are there, even if they have not been modeled. The situation is even more complex because even what could be a direct relation may be derived. If Application Service A Serves Application Function C, it is still possible that in between sits Application B (and potentially many others as well).

---

* There are underlying abstract elements in the standard, such as 'Composite Element', 'Core Element' that are used in the description of the standard. The meta-model is what we actually use when modeling and what is described and used in this book.

† There were also many problems with the text and figures in the rest of the standard. And while not everything has been fixed, I strongly advise against relying on ArchiMate 3.0 and not updating to ArchiMate 3.0.1.

‡ The name ArchiMate stems from a contraction of the words 'Architecture' and 'Animate'. The original research idea was that it should be possible to animate architecture views (probably based on that idea of derived 'shortcuts/summaries' and other ways of providing different users with different viewpoints *automatically*).

9.4 How useful are derived relations?

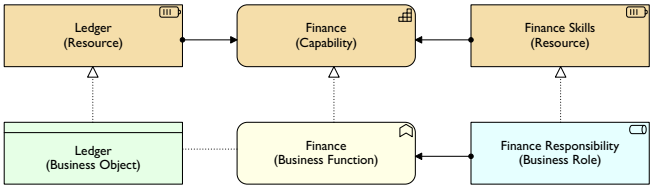
As a syntactical procedure that creates useful inferences from a detailed model, derived relations are of limited use. They will not allow all meaningful conclusions and allow for many false ones because deriving valid inferences requires more than what a syntactical/logical procedure can deliver. (We could have known given Uncle Ludwig's analysis of more than half a century ago). The derivation mechanism, in other words, is insufficiently reliable in the real world.

I tend to use them above all as a 'light' guide during the development of modeling patterns. For analysis in models, I allow more freedom than ArchiMate's derived relations allow. Every modeled relation is fair game from which to draw conclusions (across the entire model). In fact, if there is a route, there generally is a relation. It is not for nothing that our landscapes have become hard to change.

10. Non-Core ArchiMate domains

10.1 Strategy

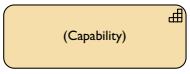
As mentioned in section 6 "An ArchiMate Map", ArchiMate sees Strategy (in part) as some sort of abstraction which is Realized by the 'enterprise itself' (the Core). It is easiest to present a small example first. One is shown in View 63.



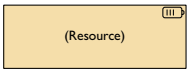
View 63. *Capability and Resource are abstractions of the 'enterprise itself'*

Here we see that the enterprise's Finance Capability is Realized by the Finance Business Function, which, you will recall, generally is the behavior performed by a Business Role, in this case the Finance Responsibility Business Role. At the Strategy level, this role represents a Resource, which I have named Finance Skills. This Resource is then Assigned to the Capability. On the left hand side, something that looks like it happens: we have a Ledger Business Object that Realizes the Ledger Resource in the Strategy domain. As you will recall, the Finance Business Function is behavior, which can Access Business Objects, in this case the Ledger Business Object. Note that there is a difference between the layers: where Business Function *Accesses* the Business Object, the Resource is *Assigned-To* the Capability.

According to ArchiMate, Strategy is about 'long-term or generic plans'. Strategy, thus, cannot be easily made up from elements of 'the enterprise itself' (which, I might stress here, is a term I have coined, it does not come from the standard). Being a more reality-oriented architect, I tend to disagree. All this abstraction makes the things we use less easy to use (and thus with a less practical meaning). In other words: I think the capability of deploying a fighting force of such and such size, with such and such equipment, and such and such skills is made up of real elements of 'the enterprise itself' (in this case: an army). However, this section of this book is devoted to describing ArchiMate as it is, so let's describe the first two new element types:



This is Capability. This is meant to model the high-level representation of the abilities of an organization (either existing or desired). Capability in ArchiMate is a 'behavioral' element, which means that it needs structure that can 'make that behavior happen'. Those are the Resources:

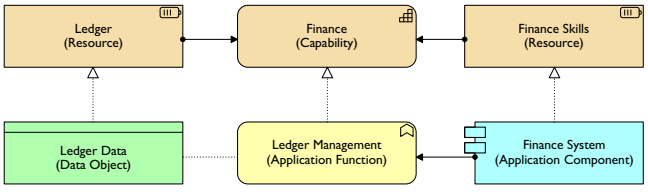


This is Resource. A Resource is an asset that an organization controls/owns. These can be tangible (facilities, equipment, cash) or intangible (rights, skills). Resources are needed for a Capability to exist, which is modeled by Assigning them to the Capability.

As mentioned before (see View 3 on page 18), the Strategy domain doesn't fully reproduce ArchiMate's standard active-behavior-passive aspect split. It is useful to quote the standard's exact definition here:

> *"A capability represents an ability that an active structure element, such as an organization, person, or system, possesses. "*

However, as you can see in the example, Capability is not positioned as 'possessed' by 'actual' active structure, only indirectly: It is positioned as an *abstraction* of the behavior (of active structure). Here (as in many other places), by the way, we encounter the fact that natural language and our human concepts are often impossible to define strictly in exact terms (Uncle Ludwig again). While active structure 'performs' behavior, we – as humans – often say some agent 'has' behavior and we mean the same thing. Along the linguistic route of 'has' comes the term 'possess', while along the term 'performs' comes the use of the term 'assignment'. In the Strategy domain, Assigned-To means 'needs' or requires'. It is best not to dwell on such 'imperfections' stemming from the essential conflict of non-logical natural language and logical modeling and just accept what the standard gives you as a 'tool'.



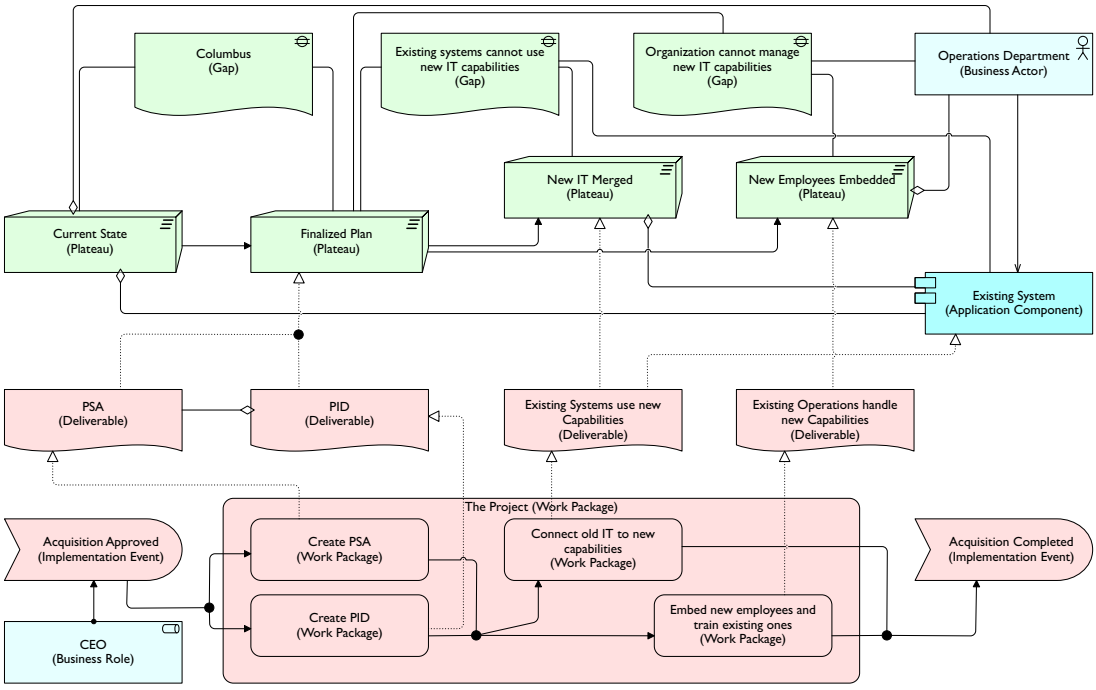
View 64. *Capability and Resource can be Realized by any 'layer' of the ArchiMate Core*

To illustrate that the ArchiMate layers are not really layers (in terms of being stacked in a single order), a variation on View 63 is shown in View 64 on page 43. Here, the Strategy-domain elements are Realized by application-layer elements.

Now, strategy is of course about defining what you are going to do. Hence, the Strategy domain of ArchiMate comes with an element that can be used to model this (high-level, long term of course).

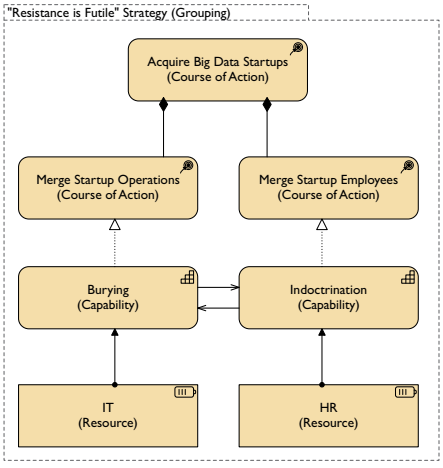


This is Course of Action. It is meant to model what the enterprise has decided to do (strategically of course). Now, as almost always, an example will explain its use best.

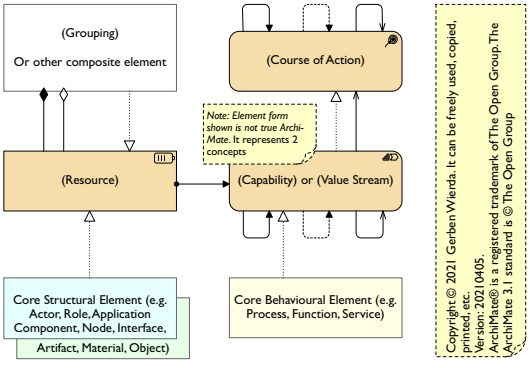


View 67. *Example of the Implementation and Migration domain with a bit of context*

In View 65, a simple example is shown. We have a dinosaur company that is being eaten alive by nimble small startups using big data analytics. To survive, the company has devised the strategy that it will acquire these startups and merge them with its own operations. For this, it has devised the "Resistance is futile, you will be assimilated" strategy. The Strategy has as its main Course of Action "Acquire Big Data Startups", which consists of two parts: merging the startup's operations with its own and merging the startup's employees with its own. Shown in the view is how this plays out in the company's *actual* Capabilities, which consist of indoctrinating new employees so they'll fit the existing company culture perfectly and burying IT innovation in non-productive set-ups. Also shown here: the use of Grouping to aggregate a set of arbitrary concepts, in this case the Strategy concepts and their relations.



View 65. *Simple Strategy Example: "Resistance is futile, you will be assimilated"*



Copyright © 2021 Gerben Wierda. It can be freely used, copied, printed, etc.
Version: 20210405.
ArchiMate® is a registered trademark of The Open Group. The ArchiMate 3.1 standard is © The Open Group

View 66. *Strategy domain of ArchiMate*

Strategy is a relatively new domain in ArchiMate. This means that — as I am writing this — there is little experience and — just as with most other changes in ArchiMate with respect to the previous version(s) — some kinks will still have to be ironed out. To give a simple example: suppose a Course of Action requires us to build a new Capability? Should not a Course of Action be able to create one in some way?

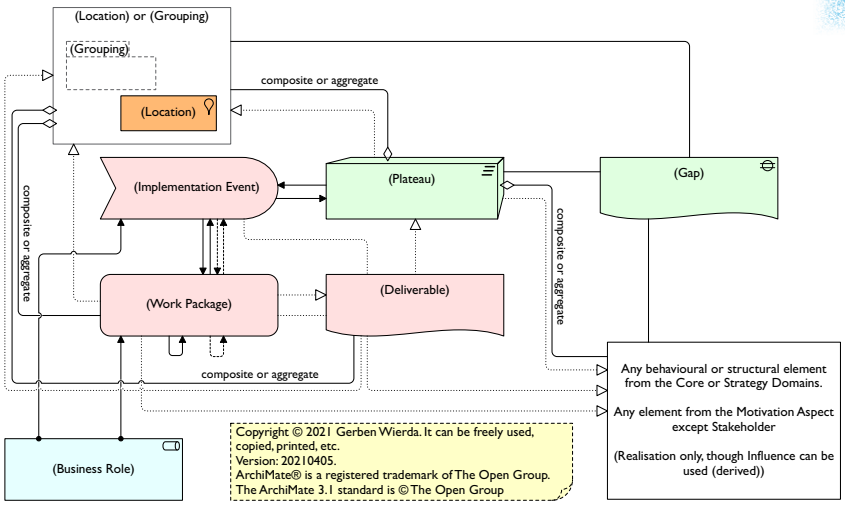
To complete this explanation, View 66 contains an overview of the Strategy domain's meta-model and its relation to the rest (the 'combined' and 'stacked' elements shown are not perfectly ArchiMate) and how it is related to the rest of ArchiMate (with the exception of the Motivation aspect, which will follow below). In section 12.11 "The Use of the Non-core Domains" on page 55, we will shortly discuss the use of this domain in enterprise architecture modeling, but I can already say here that the rest of the book will not pay extensive attention to this domain. Not that it is needed: it's simple enough.

10.2 Implementation & Migration

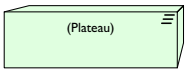
The Implementation & Migration domain has been part of ArchiMate since ArchiMate 2. The domain is meant to model everything related to change. This means it can be used to model plateaus (enterprise architecture lingo, see below), 'gaps' between plateaus, and the work needed to realize enterprise changes (go from one plateau to another) in projects programs. It can again best be explained by an example.

In View 67 on page 44, we see the Program that has come out of the strategy of the previous section. This Program consists of a couple of Work Packages that Realize Deliverables. These Deliverables Realize Plateaus, which Aggregate 'real' elements of our landscape. There is an initial Plateau that stands for the situation at the start of the Program.

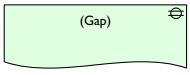
A few possible relations with Core ArchiMate elements are shown. The CEO is Assigned-To the approval to signify his importance for this event. Some Plateaus Aggregate the Existing System and the Operations Department, as these are objects of the changes. They are also related to some of the identified Gaps. Let's describe the various elements.



View 68. *Implementation and Migration domain of ArchiMate — meta-model overview*



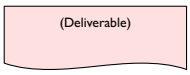
This is the Plateau. It generally stands for 'a state the enterprise 'landscape' can be in', mostly used to describe intermediate states in a larger transformation of the enterprise's landscape), but — as you can see in the example — it can also be used to model phases in a program or project. In the example, the startup phase of the project delivers a Project Initiation Document (from the Prince2 method) and a Project Start Architecture (from the DYA framework).



This is the Gap. It stands for a difference between Plateaus. It will generally be the product of a gap-analysis, so it is an assessment of sorts. It can be Associated with Plateaus or with elements from the Core (the 'enterprise itself'). In the example, the Columbus Gap (Columbus Management — as I read it originally from Jan Hoogervorst — is: "When we left, we did not know where we were going, when we arrived we did not know where we were, and everything was paid for by other people's money") is Associated with the state at the start of the project and the moment the plans are finished. We know where we are, what we are going to do, and where we will end. Or at least we have that illusion.



This is the Work Package. It stands for an amount of work to be done in a project-like setting. So, actions intended to achieve a well-defined result, within time and resource constraints.



This is the Deliverable. It stands for anything well-defined that comes out of a Work Package. In most project management methods, these need to be signed-off on by for instance a project executive, or by customers, end users, and so forth.



And finally, this is the Implementation Event. It plays the same role in the Implementation and Migration domain as the other events do in the Core language: a marker to signify the start, status or end of behavior.

In View 68, the full Implementation and Migration part of the meta-model (including all the allowed direct relations, with the exception of the general rule that any type may Compose or Aggregate itself and Association is always allowed) is shown, including how it connects to the rest of ArchiMate (the gray diamonds represent that these relations can be either Composition or Aggregation).

In section 12.11 "The Use of the Non-core Domains" on page 55 we'll shortly discuss the use of this domain in enterprise architecture modeling, but I can already say here that the rest of the book will not pay extensive attention to this domain.

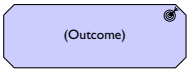
10.3 Motivation

As mentioned in section 6 "An ArchiMate Map", ArchiMate has a whole part intended to model the 'why'. It contains several concepts. It is easiest to illustrate (most of) them in an example as can be seen in View 69 on page 46.

The new elements and relations are:



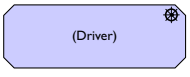
This is a Goal, which is an end state that a Stakeholder wants to achieve, or a direction a Stakeholder wants to move in. In the example, the Business wants to achieve 24/7 operations and the IT Run wants to lower IT cost.



This is an Outcome. This is like Goal, but where Goal is about a *desire*, the Outcome is an actually *achieved* result. In the example, the 24/7 Outcome Realizes the 24/7 Goal (this organization has been successful).

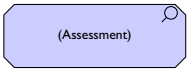


This is a Requirement, which is a obligatory aspect of what a system or process Realizes. In the example, there is a Requirement to use only standard operating systems in your infrastructure, to have high-available infrastructure and to have a certain business process fully automated.



This is a Driver, which is something that drives the organization to set goals and implement changes. These may be

external (e.g. market forces, regulation) but also internal (e.g. the mission or vision of an organization). In the example, there is one driver: 'Increased World-Wide Competition'.

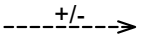


This is an Assessment, which is the outcome of an analysis of a Driver. E.g. if a driver is "customer satisfaction", a poll might result in an assessment. In the example, there are two Assessments: a new competitor has arrived on the scene and we win/lose 10% of our customers each year (because they come to/from the competition)



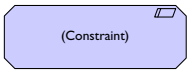
This is a Stakeholder, which is a role that is interested in achieving a Goal. In our example, there are two Stakeholder roles: Business and IT Run (the responsibility for keeping the systems operational). Note that Stakeholder is not related (or equivalent) to the Business Role element in the Core.

There is a new relation as well:

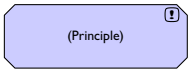


This is the Influence relationship. It is used to model the way Driver, Assessment, Goal, Principle, Requirement and Constraint can influence each other. Normally, a label on the relationship is used to denote the type of influence. In the example, the Requirement to use standardized operating systems in the infrastructure influences positively the 'Low IT Cost' Goal, but the 'High Available IT Infrastructure' influences that same Goal in a negative way. Note: The 'High Available IT Infrastructure' Requirement Realizes the '24/7 Operations' Goal but we could also have used the Influence relation there. The Realization could be interpreted as a different, more 'definitive' form of a 'positive influence'.

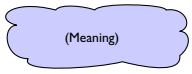
The full Motivation domain of ArchiMate is shown in View 70 on page 47. It shows the allowed direct relations, excluding the always allowed Composition and Aggregation of an element type by itself. Also not shown in that diagram is that *any* Motivational element type may Influence *any* other Motivational element type. And that – obviously, as Association is allowed between all element types in ArchiMate – any Motivational element type may be Associated with any other Motivational element type. Which leaves us at explaining the remaining Motivational element types:



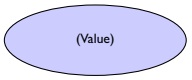
This is a Constraint (a Specialization of Requirement), represents a forbidden aspect. Examples are budgetary or time constraints for a project (Work Package), technology constraints for a system (allowed technology), or legal constraints for processes or services. We could say that Constraint is a 'convenience' type, any role it performs in a model could have been fulfilled by a Requirement.



This is a Principle, which is a sort-of Goal that is a generalized Requirement, specifically for the *architecture* of the enterprise. There is no Principle modeled in the example, but – like Constraint – it behaves more like a Requirement than a Goal, though, even if it is not a Specialization of a Requirement.



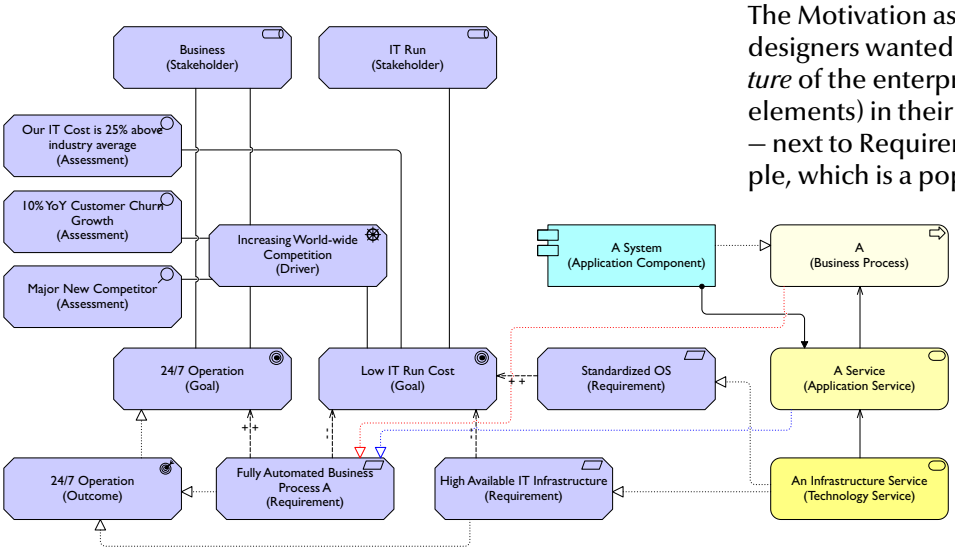
This is Meaning. In earlier versions of ArchiMate, which lacked the Motivation aspect, it used to be part of the business layer. It represents an interpretation of something (e.g. a core element) by someone (e.g. a Stakeholder). If used in Association with passive structure, it is generally used for the intention of that element.



This is Value. In those earlier versions of ArchiMate, this also used to be part of the business layer. It stands for the value of either an element from the Core or from Outcome. But you can of course Associate it to anything, as Association is the catch-all. As far as I have seen modeling with the use of valuation, it is generally done differently, with valuation as (financial) numbers in the properties of elements. In other words: I've hardly ever seen this (or Meaning, which is evermore suspect from Uncle Ludwig's perspective) used in a useful (meaningful) way.

The Motivation aspect of ArchiMate shows its origin: the designers wanted to catch the intentions for the *architecture* of the enterprise (which was modeled with the Core elements) in their modeling language. Hence, the use of – next to Requirement and Goal – the concept of Principle, which is a popular, though toxic, concept in enterprise architecture circles. See my other book *Chess and the Art of Enterprise Architecture*.

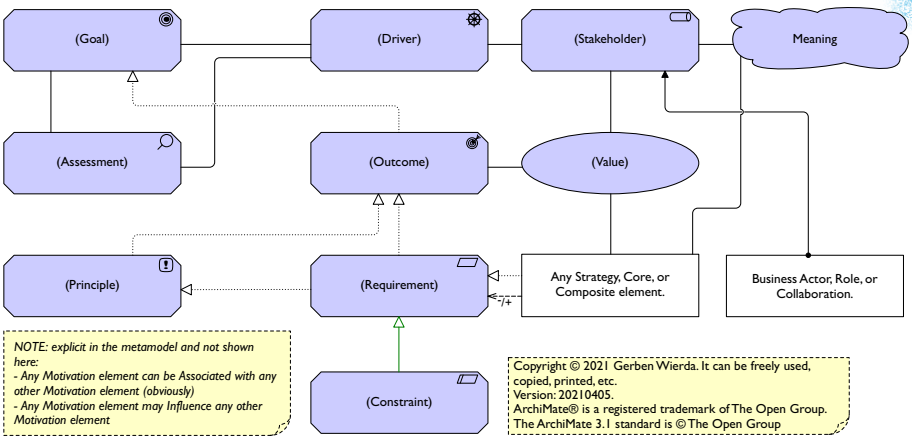
The concepts of the Motivation domain are clearly not exactly, discretely separated. It is sometimes a matter of taste if something is seen as a Goal or a Driver. Having a certain strategic Goal as an organization may count as a Driver, it seems to me. The divide between a requirement and a principle is not very clear either. It seems it



View 69. *ArchiMate Motivation aspect example*

is a Requirement that has been promoted to a Goal in a certain context. These overlaps and indistinctnesses are not really a problem (there are more issues like this in ArchiMate), it then comes down to creating your own good patterns to use them. Just don't rely on the standard to be a good guide here.

What this part of ArchiMate also shows is how different the world of human intentions and motivations is from the logical world of IT and how difficult it is to map one onto the other ("I have one negative influence and one positive. Now what?").



View 70. *Motivation domain of the meta-model — most Influence relations not shown*

The relations between the elements are generally Influences or Associations. The exceptions are that Realization is used to:

- let a Requirement or Constraint (potentially via Principle) Realize a Goal, via Outcome;
- let any ArchiMate concept from other domains Realize a Requirement. In the example, a Technology Service, an Application Service and a Business Process all Realize a Requirement. An interesting question is if you have — as is the case in the example — a Requirement like 'Fully automated Business Process'. Is it the Business Process that Realizes this Requirement (in red) or the Application Service (in blue) or both? There is quite a bit of freedom in choosing your patterns.

In section 12.11 "The Use of the Non-core Domains" on page 55 we'll shortly discuss the use of this domain in enterprise architecture modeling, but I can already say here that the rest of the book will not pay extensive attention to this domain with respect to its original goal: modeling the intentions of the *architecture* (principles and all).

Nothing stops us from using this for other practical, less navel-gazing uses, i.e. modeling intentions of the *enterprise*, however. As shown in section 19 "Modeling Risk & Security with Motivation Elements" on page 111, the Motivation domain offers us a basic mechanism to model Risks, Control Objectives (including those from Security) and Controls and in that way link ArchiMate models to the world of Risk and Security.

10.4 Closing Remarks on ArchiMate

ArchiMate has improved and matured over the years, but there are still remnants that will make you wonder. If you get the feeling that ArchiMate is not 100% clean, I cannot fault you. But even with these issues (which we will visit later) I must stress that in my experience, ArchiMate is very usable, and even more so in version 3 than in previous versions. Creating Solution Architectures based on detailed ArchiMate models has, in my experience, substantially reduced delays and unforeseen issues — having a detailed 'Current-State' model has seriously improved control over and reporting on our landscape to regulators.

When we started to use ArchiMate, we initially had many doubts about how the ArchiMate designers had set up the language. But, we did follow the rule that we would strictly stick to the meta-model and not change or adapt it in any way (even though our tool supported that). We did this for two reasons:

- Any future tool update might break our models or require a lot of work;
- Understanding something comes only after a lot of experience. Though we did sometimes dislike what we saw, we decided to distrust our (beginner) skills more than the skills of the language designers. This was wise: when our experience grew, we learned to appreciate most of the choices the designers had made.

If you focus on finding faults (something that comes naturally for architects as they are always for the lookout of something that may break) you will. But in my experience, that does not affect the usability much. ArchiMate is very, very usable.

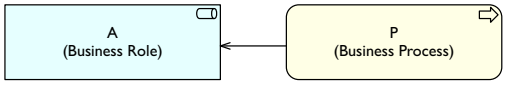
11. Some Beginners' Pitfalls

11.1 Pitfalls of Derived Relations

If you begin with ArchiMate, there are a few pitfalls that I've seen many people fall for. Previous editions of this book started out with pitfalls that came from the sloppy way derived relations had been implemented in the language, resulting in relations that were possible, but that were easy to misinterpret as a beginner. With the changes in ArchiMate 3.0.1 and 3.1, most of the craziness has been removed.

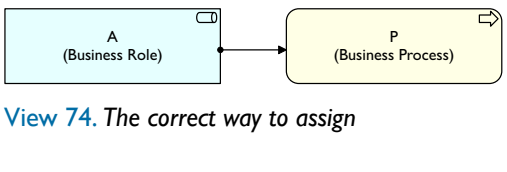
Though less so, it is still true that — even if the language allows the relation — it is not certain that the relation actually

means what you think. That shortcut leaves intermediate structure out, and sometimes that structure in ArchiMate will be something that is not what you intended to show.

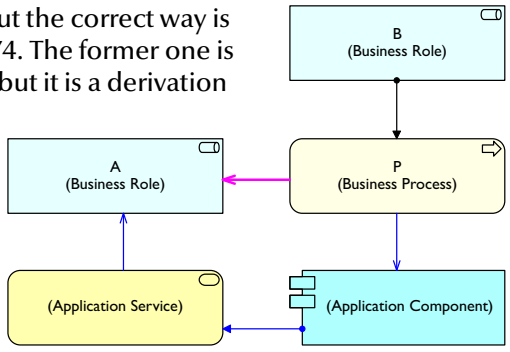


View 71. *Attempting to assign process to role*

For instance, a beginner might use the pattern in View 71 to assign a process to a role (or a role to a process — it is possible to state this in both ways). But the correct way is shown in View 74. The former one is a valid relation, but it is a derivation of some other route. A possible — though convoluted — route may be what is shown in View 73. The violet relation is a Serving (formerly "Used by") relation. It doesn't signify that role A *performs* process P, it makes *use* of it. In the scenario there is an application that needs to be restarted once in a while. Role B performs that process and as Role A uses the application (blue relations), it depends on process P instead of performing it. So, it is important to keep in mind:



View 74. *The correct way to assign*



View 73. *Potential derivation for View 71*

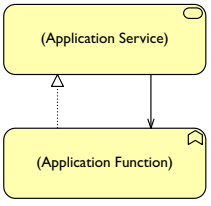
> Pitfall 1. Derived relations may not mean what you think.

This is one reason why I generally say that it is best to keep as close as possible to the core relations in the meta-model, and only use derived relations when they are part of patterns. We will discuss many patterns in the rest of the book.

Question 1. Note that there is a much simpler way to get the derivation. Can you say which one?

Answer 1. Process P Realizes a Business Service("Restart Application") which Serves Process A, straight from the meta-model.

There is another small confusion possible during derivations: that a 'read' Access relation runs from passive element to behavioral or active element. But a 'read' Access has the arrow pointing at the opposite direction of the actual grammatical direction of the relation, which always runs from behavioral/active element to passive element.



View 72. *How Function and Service are displayed in the Meta-model*

11.2 Don't trust the language or the tool blindly: know what you are doing

A tool sometimes offers to create you a 'default relation' between two elements. If you draw a 'default' relation between that Application Component and Business Process A, your tool may take for instance Realization, which means that the Role is an abstraction of the Application. But in this case, you want Serving. A basic pitfall is therefore, trusting your tool to repair your lack of knowledge. Of course, this explanation will see to it that you will never do that, but it is a pitfall I have seen often.

Hence, the second beginner pitfall is:

> Pitfall 2. Trusting your tool blindly.

Lastly, ArchiMate (and several tools) support 'viewpoints'. In ArchiMate 3, Viewpoints have become optional, which is a good thing in many ways (for one: if you want to certify, you don't need to be able to reproduce them all). Viewpoints are in fact subsets of the ArchiMate meta-model (the subset may include derived relations). A viewpoint may restrict which types of elements you may model and what relations are allowed between them. They are a sort of patterns, but not quite, as they do not so much prescribe *how* to model but only offer constraints in *what* you can show.

I use 'view templates' for modeling instead (see Section 30 "Construction & Use Views" on page 187). A tool will constrain your use of elements and relations if you happen to be working in an ArchiMate Viewpoint and not being able to create a relation may not be because it is not allowed, but because it is not allowed in the active Viewpoint.

> Pitfall 3. Using a viewpoint without knowing that you are.

Personally, I completely ignore the viewpoints and only model in a 'generic' viewpoint — the one that allows the entire metamodel and use my own patterns.

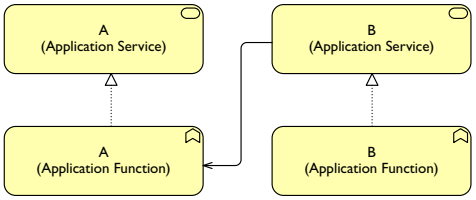
11.3 Misunderstanding the standard meta-model diagram

I have seen beginners often just copy the relations they see between element types in the basic meta-model (View 54 on page 38). This is often fine, but there are some snags.

A common snag is the way the standard meta-model shows the relation(s) between a function/process and a service. This looks like View 72.

What the view illustrates is that 'a' function can realize 'a' service and that 'a' service can be used by 'a' function. But in real modeling, we model generally not 'a' service or function. We model specific (even if they are abstract) services and functions.

So, when a beginner models something like View 72 as the relations between a specific function and the service that it provides, he actually says the function also uses the service it provides: it uses itself. Now, as a way of modeling recur-



View 75. *How the function/service relations in the meta-model are intended.*

sion, this could be OK, but that is probably not what is meant (we generally do not model this kind of technical details in Enterprise Architecture). In short: the *meta-model* pattern of View 72 should most likely *not* appear in our models.

What the above snippet illustrates is that *an* element of the *type* Application Function can Realize *an* element of the *type* Application Service, and that *an(other)* element of the *type* Application Function can use *an element* of the *type* Application Service as illustrated in View 75.

The Realize and Serving relations are between the same *types* of element, but not between the same *elements*. And of course, the same is true at the Business and Technology levels.

So, the last beginner pitfall mentioned here is:

> Pitfall 4. Copying relations in the meta-model to your model blindly.

These beginner pitfalls have a message in common: don't use a language unless you know what you are doing. It is little use to you or your environment if you as an architect are a superficial operator.

12. Some Noteworthy Aspects of ArchiMate

12.1 Licensing

Though being the property of The *Open* Group, it is debatable whether ArchiMate is an *open* standard. Most definitions of 'open standard' require the standard to be royalty-free and maintained by a not-for-profit organization. ArchiMate meets neither requirement. There are definitions of 'open standard' without these requirements, such as industry standards in a commercial domain, e.g. telecom standards (which are generally based on patents). These are licensed against FRAND (Fair, Reasonable, And Non-Discriminatory) terms. Whatever your interpretation of 'open', the short story is (Spring 2017):

- Writing about ArchiMate (blogs, books) is royalty-free;
- ArchiMate is royalty-free for academia;
- ArchiMate is royalty-free for internal use by organizations;
- ArchiMate is royalty-free for internal use by organizations that use it to do consultancy for others, mainly as long as you do not advertise that you sell 'ArchiMate consultancy';
- Selling ArchiMate tools & training requires (in practice, possibly not legally) a commercial license from TOG, especially if you want to use the ArchiMate® Registered Trademark in commercial traffic. Prices start at US$ 2500/year. Think twice before you start building a tool with a plan to sell for $10 in the App Store.

There is a more extensive treatment of the (rather toxic) legal issues on the ea.rna.nl web site. The legal reality is much more complex than the bullets above and there is no warranty that the above stands up in court. You need to read the actual licensing conditions of ArchiMate's material and the use of the ArchiMate® Registered Trademark and consult a good IPR lawyer if you do not want to pay unnecessary fees when exploiting ArchiMate commercially (or pay up, which is probably cheaper). Besides, these may change in the future while this book will not necessarily follow. I personally do not consider ArchiMate an *open* standard (but a *commercially exploited consortium* standard), but as I'm not religious about using only open standards. I do not mind.

12.2 ArchiMate, the modeling language

ArchiMate started out as a public Dutch research institute project, supported by a couple of large administrative organizations, both public and private. Its initial version was finalized in 2004. In 2009 it was published as a standard by The Open Group.

Enterprise Architecture is not new and quite a bit of modeling has been done, most of it not in ArchiMate.

People create models for projects, often just as a set of images in an 'architecture' document. Sometimes, some modeling for projects is done in UML, but most of the time, you will look at some non- or semi-standardized use of boxes, arrows, dotted lines, nesting, etc., generally some sort of free-format graphical tooling will be used like Microsoft Visio for Windows or OmniGraffle for Mac or worse: Microsoft PowerPoint. One of the 'nice' aspects of such modeling is that it is often ambiguous enough for all stakeholders to see their own preferred reality in it. A more vague and ambiguous approach enables this often 'politically' expedient modeling. Everybody is happy, that is, until it turns out there is a problem somewhere during a later phase of a project. In my experience, people do not understand detailed Visio images either, but they seldom have to understand them anyway, so everybody is happy. When ArchiMate arrives, suddenly people are confronted with views that they have to agree on as a definition of sorts, and suddenly they have to understand what the image says. And the result is that some of them will fight adoption of the language. They want to stick to the old simpler way. Other resistance comes from people who see some of ArchiMate's weaknesses (especially software architects are often in this group, as ArchiMate's 'looseness' is in conflict with their need for solid foundations).

It gets even worse with the modeling of your current state. Once every few years, the lack of insight into the existing landscape will cause an effort to create an overview. People work for months at unearthing the situation and generally in the end you will have a few large posters with boxes and lines (and a legend to explain what every box type, arrow type and color signifies). The model is never maintained and after a year, it becomes so outdated that it becomes pretty useless for anything but a general introduction. Now, with ArchiMate, this can of course also happen if you just draw your visuals in ArchiMate without the support of tooling that can keep thousands of elements and relations in a single coherent model.

The fact that a single coherent model is useful is of course clear to many. So sometimes modeling is done in Enterprise Architecture tooling. These tools often have their own internal proprietary language or meta-model: a meta-model that also comes with a certain philosophy on what Enterprise Architecture is. These days, some of these tools will have ArchiMate as an option, often (more or less successfully) grafted on top of something like UML or their own proprietary language. I'll say a bit more about tooling in Chapter "Tooling" on page 215.

Modeling in ArchiMate brings the advantage of a fixed syntax of your models with reasonably clearly defined element and relation types. Modeling with a tool brings you the possibility of having those different views on what you are modeling to be actually related 'under the hood' and it brings more possibilities of actually maintaining and using your models. Modeling in ArchiMate also brings modeling that is independent of your modeling tool. If you ever decide to migrate to another tool that supports ArchiMate, it is possible. In the worst case the migration cannot be automated and migration is a lot of hand work, but it is possible without translation to another meta-model (which in a practical sense can be considered impossible: you will largely have to redo all your work).

In Chapter "Discussing ArchiMate" on page 205, I will discuss some of the areas where I think ArchiMate could be improved. In this chapter I'll say a few things about its strengths and some other weaknesses. But, although I have had my criticisms in the past and still have some to this day, *my overall conclusion remains that the language is very practical in actual use*. I would not have written a book about it if I were not convinced that ArchiMate can be extremely useful for Enterprise Architecture work. And the most important argument is this: there is currently no better practical alternative.

In the final document of the original research project, the authors write:

> *During the initial design of the ArchiMate language no explicit attention was paid to any desired formal properties of the metamodel itself. Emphasis was put on the applicability of the language.*

And this is also what resulted: a very *usable* language, which is not perfectly logical — though later versions have slowly added a more structured foundation. As mentioned above, this sometimes offends architects who desire a formal, certain language which can be used without interpretation (or so some architects think). ArchiMate offers a 'rational' way of looking at Enterprise Architecture, but it does not offer a 'perfectly logical' way. Later versions of ArchiMate have improved and extended (where every aspect of the latter is not by definition an expression of the former) the language and some original choices leading to severe problems. The derivation mechanism could still be improved (though we know it can never be perfect). But it still remains noticeable here and there that the language never was based on a formal model. This remains true, even if ArchiMate 3 came with a meta-meta-model, giving the language (the meta-model) an improved foundation and ArchiMate 3.1 improves the definition of derivations.

12.3 Separation of 'actor' and 'behavior'

At the business level, separating process (behavior) from role (actor) is common practice in modeling. There is some confusion with respect to a business function (as will be discussed in Section 17.2 "Business Function or Business Process?" on page 92).

Some approaches see a function as behavior embedded in some sort of actor-like element, you can recognize it in the language used: a function *is* behavior (as in ArchiMate) versus a function *does* something (function *performs* behavior, it thus seems to be an actor of sorts). ArchiMate is a language where the separation is more fundamental and the separation has been replicated in all layers. This clarity in the meta-model really helps to make good models, but it is important to stress that at every layer, the 'actor' and its behavior are fundamentally inseparable. You can choose not to *model* one (and use derived relations to pass them by), but they are implied to exist even if they are not explicitly modeled.

The split often is confusing for those coming from ways of thinking that do not have that split, e.g. Process Modelers who seldom use the function concept at all (for more depth, see: 17.4 "The 'End-to-end' Business Process" on page 95) or Software Architects where (like in mathematics) a function is both structure and behavior in one.

There is another misinterpretation I've come across. If you say 'passive element' to people, they sometimes interpret that as 'not taking *initiative*', as '*re*acting only'. This I encountered for instance when a colleague first was told about the interface concepts in ArchiMate. He found it illogical to say that an interface is an active element. Because an interface does nothing by itself, it needs to be used before it does something. In his use of the words active and passive it meant an interface is passive. But ArchiMate does not mean '*re*active' by the word 'passive' and it does not mean 'taking initiative' by the word 'active'. In ArchiMate terms:

- Active means *capable* of performing behavior (either acting or reacting);
- Passive means *incapable* of *performing* behavior (or sometimes just the aspect of being incapable, e.g. an executable file of a program ("`foo.exe`") is passive. It's

just a file; but it can Realize an Application Component, an abstraction that stands for the executable file being executed and becoming an active element in your landscape).

Behavior is what links active to passive elements: Active elements are Assigned-To behavior and behavioral elements Access passive elements. The passive ArchiMate elements are philosophically speaking 'objects' and the active ArchiMate objects are philosophically speaking 'subjects'. What the colleague said was not wrong, it is just not how ArchiMate defines the terms 'active' and 'passive'. For completeness, here quotes from ArchiMate's official underlying core definitions of active and passive:

> *Active structure elements are the subjects that can perform behavior.*
>
> *A passive structure element is a structural element that cannot perform behavior.*
>
> *Active structure elements can perform behavior on passive structure elements.*

And service and interface are defined as:

> *[...] a service [...] represents an explicitly defined exposed behavior.*
>
> *[...] an interface [..] represents a point of access where one or more services are provided to the environment.*

12.4 One View Type

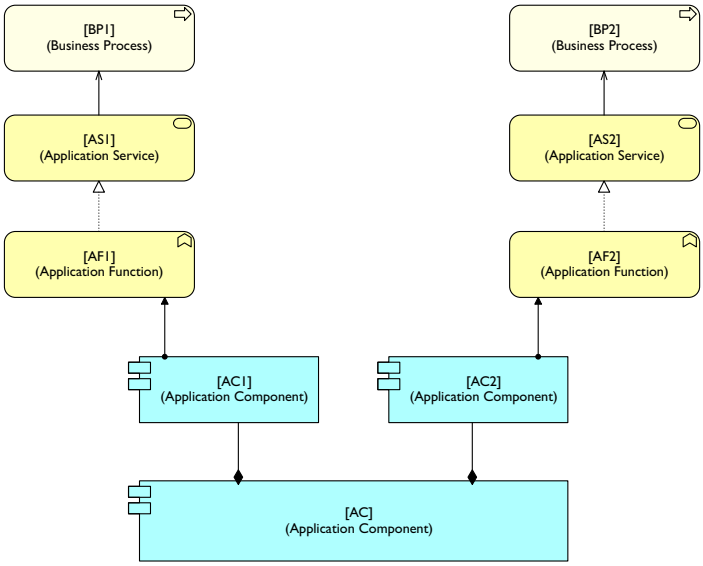
One of the beautiful aspects of ArchiMate is that it is a true Enterprise Architecture language. ArchiMate has been designed to model the interrelations and dependencies across *all* 'layers' in your Enterprise Architecture: from Business via Applications down to Infrastructure/Technology and vice versa. Though tooling often offers you all kinds of different view types (many of which are suggested optionally in the standard), ArchiMate in effect has only one single view type: the entire Enterprise Architecture. Those optional ArchiMate *Viewpoints* (which I do not use) only restrict what is in an ArchiMate view, but it is not a fundamentally different view.

At the software level, UML with all its different kind of views on the same objects and classes might be more precise. But to model the whole shebang, ArchiMate does a very good job.

12.5 On the direction of structural relations under the assumption of 'worst case'

In ArchiMate 3.1 the structural (Composition, Aggregation, Assignment, and Realization) and dependency (Serving, Access, Influence, Association) relations all have a single direction – except for Association, which exists in 'undirectional' and directional form as we have seen. This direction plays an important role in calculating the derived relations between elements that are indirectly related in a model.

The idea behind it seems logical. Take the example in View 76. If you look at it, having a derivation mechanism that



View 76. *Example used for discussing 'worst case' versus 'happy flow' assumptions about dependencies*

makes Business Process BP1 independent from Application (sub)Component AC2 is logical. After all, Business Process BP1 uses only Application Function AF1 and not Application Function AF2. But the question is, what do we mean exactly by dependency? Business Process BP1 is clearly not *meant* to depend on Application Function AF2, but does that mean it is *in*dependent? Suppose Application Function AF2 has a fault and it breaks the Application Function of Application Component AC and *as a consequence* Application Function AF1? In that case, there *is* a relation between Application Service AS1 and Application Function AF2, even if the relation is not *meant to* play a role and even if the relation from Application Component AC2 to Application Component AC has the opposite direction as the one from Application Component AC1 to Application Function AF1.

Or that interesting example of a Technology Service that does not depend on what Application Function or Technology Function uses it, except of course when the Application Function or Technology Function overwhelms the resources of the Technology Function which for instance happens when a Denial-of-Service attack hits your web servers, but it can also happen in normal cases. Suddenly, in that 'worst case' there *is* a dependency from 'served' to 'server'.

How you interpret 'dependence' has to do with the scenarios you take into account. Look at 'happy flow' only, and it makes sense that, for instance, Composition and Aggregation have a direction. But look at 'worst case' (a natural way of looking from an architectural point of view) and (officially accepted since ArchiMate 3.1) the Composition/Aggregation relations will create dependencies in the both direction.

Or: if you look at 'worst case' instead of 'happy flow', all structural relations become bidirectional. Going back to the example above: there seems no way Business Process BP1, Application Service AS1, Application Function AF1

and Application Component AC1 can depend on Business Process BP2, because whatever breaks in Business Process BP2 cannot influence Application Service AS2. But if BP2 overwhelms AS2, AS1, AC1 and thus AC, BP1 will be affected. BP1 then depends on BP2.

Maybe in a well-designed model, one could give properties to the relations about their influence strength in both directions and then calculate the degree of dependence across a model (but now I am dreaming and it is probably not feasible either).

12.6 On the limitations of derived relations

Structural and dependency relations in ArchiMate have a direction and this direction plays a role when creating valid derived relations.

The directionality of relations was required during the development of ArchiMate because of the way derived relations were set up. In that set up, bidirectional relations (Assignment and Association — then a structural relation, but the setup has changed with ArchiMate 3 and again with ArchiMate 3.1) were (for the sake of analysis) replaced by two unidirectional relations.

In ArchiMate 2.0, the bi-directionality of the Assignment relation was replaced by a unidirectionality. This was a good thing and it removed quite a few nonsense derived relations. The ones that made sense (like a Business Service Serving an Actor) were added explicitly as 'core set' relations to the meta-model (which is why my version in View 54 on page 38 has these explicitly added, often these are forgotten in views of the meta-model as they were not necessary in ArchiMate 1.0). The new meta-meta-model and the improved foundation of the full relationship table of version 3.0.1 meant that even more relations that were no longer derivable had to be added as direct relations (e.g. the dynamic relations between services and events). ArchiMate 3.1 has not removed the now derivable 'direct relations' from the metamodel (an omission, probably).

If I look at the relations and their direction and I want to have some sort of criterium to be able to decide on making summaries and which combinations should and should not be allowed, I end up choosing the concept of 'dependence'

> *(Note: I originally wrote this for ArchiMate 1 and 2. The generic use of the term 'dependency' now overlaps with ArchiMate 3's use of the term for the differentiation between structural and* dependency *relations, and there is little I can do about it. There is a sort of dependency in all of ArchiMate's relations, so the choice in ArchiMate has been unfortunate.).*

If element A depends via relation R on element B, but B does not via the same relation depend on A, then the relation from B to A is unidirectional. The clearest example is the Serving relation: if a Technology Service X Serves an Application Function Y, Y depends on X, but not the other way around. For the Technology Service X it is totally irrelevant if whether is actually used or not: it does not depend on being used (but as we saw above: this is not quite true, as real-world Denial-of-Service attacks prove). But — in a simplified design-sense — for Application Function Y, life stops when Technology Service X is not available and not the other way around.

Take also, for instance, the Access relation. It has a direction that runs from the behavioral element to the passive element. Now, I can understand that in part: a Data Object that is created or deleted by an Application Function depends for its existence on the work done by the Application Function. It also fits with the subject-verb-object like structure of ArchiMate. But a Data Object that is read by an Application Function does not depend on it at all, it is the other way around: if the Data Object is not there or it is broken, the Application Function may fail. The object may influence the subject, as in "The broken tile caused him to fall". In natural language, objects without behavior may be used as causes (become active). In ArchiMate, passive elements are not like that.

ArchiMate 3.0 has fixed some of these limitations by allowing 'top-down' dependencies (e.g. Technology being Served by the Business Layer as described in Section 7.16 "Who is in Charge?" on page 30) and seen in our example in View 73 on page 48. E.g. think of a Business Process that requires access to a certain document. The document is a Data Object that is Realized by a file Artifact and that itself Realizes a Business Object (e.g. a letter). The file Artifact resides on a server (Node). When you look at such a situation, there is nothing weird or 'wrong' with thinking that the Business Process Accesses the Artifact and your business people will probably think that way. They often think in files, not in abstract Business Objects. Since the redesign of derivation in ArchiMate 3.1 these relations are now possible.

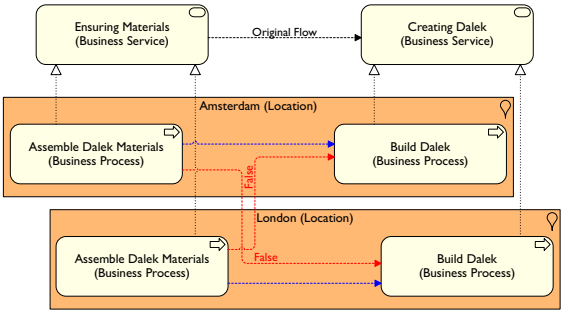
One of the things that initially attracted me to ArchiMate was not only the reasonable and holistic meta-model that is its foundation. It was the promise of derived relations. To have a model where you can have all the necessary details for a decent analysis as well as views where you have summarized the complex views in simpler ones for consumption by — for instance — management. And both views are correct and come from the same underlying structure.

I must admit, the official ArchiMate derived relations have lost much of their initial attraction for me. There are too many problems where you as an architect know that the language does not allow you a (derived) relation, but where you know it makes sense to have one because a real world dependency exists. In my practice, it has turned out that the official derived relations have a very limited use. They are too restricted in many situations. Depending on the patterns you use, you develop the need for quite different 'derived' relations.

I also have not seen a tool yet with decent support for derived relations. The original ArchiMate project created them as an extended set of relations on top of what is in the basic meta-model and they provided a mathematical proof (somebody has to get his M.Sc. or Ph.D. after all) that if they were defined as they were defined, the new set of 'basic

+ derived' relations was again a closed set. The standard now says the derived relations are as proper as the core relations. It also is being presented as such in most courses and educational material. I haven't seen much material that goes into the limitations of this approach.

Adding derived relations next to the direct ones explicitly to a model creates a lot of new analysis routes in those models. And if you use tooling, the number of possible routes quickly overwhelms your effort to keep your analysis viewpoints (often some sort of algorithmic viewpoint logic) working. This is why I advise to stick as much as possible to the basic meta-model and *only* use derived relations in the creation of your *patterns* (which we will do starting from Section 14 "Patterns"). That way you create a stable subset of all the possibilities that makes analysis of your models achievable. I advise against using too many derived relations other than those that are part of your patterns.



View 77. *Drawing false conclusions from derived dynamic relations*

Finally, it is often possible to draw false conclusions from derived relations. This is already explicit in the 'potential derivations' as we saw at the start of Section 9 on page 39. Have a look at View 77 for another example using 'valid' derivations.

We have two Locations where Daleks are being built. These are services provided to some customer, e.g. by two contractors at the same location who perform the processes. Note: the processes with the same name are separate elements in the model. It could very well be that they are slightly different because of different circumstances, regulations, etc. From the original Flow (top) we may derive all the lower ones. But the red ones (marked 'False') are not valid derivations in reality. The materials gathered in London are used in London, not Amsterdam. It is even possible to create examples where a 'potential' derivation is valid in reality, whereas a 'valid' derivation is not. Another Uncle Ludwig moment: meaning does not follow from grammatical rules.

12.7 Why two types of software?

ArchiMate has two element types for software:

- At the Technology Level (IT infrastructure), there is System Software, which is Assigned-To a Technology Function which Realizes a Technology Service;
- At the Application Level, there is the Application Component, which is Assigned-To an Application Function which Realizes an Application Service.

Now, if you know a bit about computers, you know that this distinction is engineering nonsense. After all, — maybe with the exception of the operating system ('supervisor') or virtualization software ('hypervisor') that directly interfaces with the hardware — software is software, whatever its use. That Relational Database System that you model at the Technology layer (e.g. Oracle, SQL*Server, DB2, PostgreSQL) is technically just another application that runs within the constraints of the operating system. So, an IT-engineer will probably initially balk at ArchiMate's separation of software in two types, where it is also can be a matter of taste (or confusion) how you model something. Take for instance the spreadsheet: it will be shown in Section 14.3 "Modeling Spreadsheet Applications" on page 65 that it can be modeled in two ways: either at the application layer or at the Technology layer. That seems problematic. Why should there be such confusion possible in the language?

But if you look at the reality of Enterprise Architecture, the discussion about types of software is *not technical*. It is about modeling for the enterprise how business, applications and infrastructure together (should) work. And when you take the *business perspective* and not the technical perspective, it is quite clear that such a mission-critical application that has all that specific business process oriented logic embedded in its code, is quite something different from that generic 'off the shelf' database that is needed to support it. If you look at the separation as 'business-specific' (i.e. containing business logic) versus 'generic', then the separation becomes more useful. Hence, the database is System Software, but the business-specific logic that is written inside it should be modeled as an Application Component, which is realized by that same database which, in fact, acts as an application *platform* in itself. As will be shown in the Application Deployment Patterns in Section 14 "Patterns" on page 62, it can become quite complex to create patterns for modeling modern architectures like three- (or even four-) tier application architectures and more modern inventions like SaaS. On the other hand: you have to invent the patterns only once — or take them from his book ;-) — but you can use them many times.

For sake of argument, we could try to imagine what would happen if we were to try to have only one software type in ArchiMate. The System Software of the Infrastructure would become (the equivalent of) an Application Component. Then, either the Application Function should be able to realize a Technology Service (which kind of makes the Technology Service a clone of Application Service) or the Technology Service realized by Software should become an Application Service (which means you have in fact software service versus hardware service). This leads to all kinds of problems with the business perspective on Enterprise Architecture. We either have application-layer elements in the Technology layer, which is confusing, or we have to limit infrastructure to hardware only. But the latter is also problematic because you can have many infrastructural components that can be Realized by both hardware and software. A couple of examples:

- A virtual hard disk in memory (RAM Disk);

- Software RAID;
- A network filter built from a general purpose computer with two network interfaces and dedicated software.

I think especially this latter problem is what would make a single type of software type in ArchiMate perhaps technically superior, but in terms of a business perspective (and a usable view on Enterprise Architecture) inferior. From a technical perspective the meaning comes from how it is *implemented* (is it a Device or an Application Component?), but from a business perspective, the meaning comes from its *use* (is it business-specific or generic functionality?). I think for the usability of ArchiMate for Enterprise Architecture modeling, it is a good thing ArchiMate has the (technically flawed) concept of two types of software, though I can imagine other solutions outside the constraints of ArchiMate.

12.8 There is more than one-to-one

An Artifact can Realize an Application Component. It can also Realize a Data Object. One data object can even Realize both. As will be described in more detail in Section 18 "Secondary and Tertiary Architecture" on page 104, thinking about both will lead us to better thinking about our Enterprise's Architecture. Or the same Artifact can Realize both active and passive elements, as will be shown for instance in 14.3 "Modeling Spreadsheet Applications" on page 65. ArchiMate is very powerful, because of that, but it might take some getting used to.

Sometimes, concepts from the real world you are modeling will end up in quite different ways in your model. This has nothing specifically to do with ArchiMate, but ArchiMate does have support for it. A good example is a customer. That customer is both an Actor in your Enterprise Architecture, but it might *also* be a Business Object. ArchiMate can let you do both and so catch the two quite different aspects of your customer (what he does and how he is represented inside your business) very well.

12.9 Detailed or not?

We architects love abstraction. Abstraction makes the nitty-gritty and complex details disappear. Abstraction makes the unmanageable, manageable.

Our business colleagues also love it when we produce abstractions. It makes our end products easier to digest.

In the ArchiMate language, you can do both, which means that – apart from the precision-like modeling that you do, for instance, when building 'Current State', 'Solution', or 'Project/Change' detailed models – you can use the ArchiMate language in a more 'loose' set-up; e.g. when designing a landscape top-down, even in a rough way with broad strokes.

For instance, when designing, if you have a Business Process that requires the services of an application, you generally start with defining that application service in terms of 'what the process needs' and then you design how this service will be realized by one or more applications.

The ArchiMate language has enough freedom to be used to make models with a lot of abstraction, and as such it also supports an approach to Enterprise Architecture design that sees services as abstractions defined top-down (from the needs of those that use the service). In fact, such thinking was an important aspect of the design process of the ArchiMate language in the first place.

But the ArchiMate language also supports a more specific and precise use that is useful for creating precise 'Current State' or 'Project/Change' models that can be used for documenting or designing your enterprise.

There is much more that can be said about this, but you can rest assured that the power of expression of the ArchiMate language is good for both, even if the concepts have mostly been explained so far in the 'precision approach'. Besides, the ArchiMate language has another important way of simplification: the use of derived relations (with their limitations, as we saw in section 12.6 "On the limitations of derived relations" on page 52).

Together, both ways ('more abstract use of the concepts' and 'derived relations') create a powerful set that enables very high-level/abstract modeling in the ArchiMate language. More of this in Section 33 "Why Model? (And for Whom?)" on page 198 and in other places in this book.

12.10 Layering Ambiguity

ArchiMate 3 has what I would call 'layering ambiguity'. It contains in itself two separate and irreconcilable ideas about layering:

- Service-orientation: Elements from different layers represent *different* items and can Serve each other (Section 7.1). A GUI of an application Serving a (human) Business Role. A Technology (infrastructure) Service such as a File Share Serving an Application Function.
- Abstraction: Elements from different layers represent the *same* item in a different way (Section 7.14). An Artifact Realizing a Business Object, or – since ArchiMate 3 – an Application Service Realizing a Business Service of itself to show an 'automated Business Service'. Or (see Section 7.17) a Technology Service that Realizes an Application Service representation of itself;

ArchiMate started out as what is now the core, something that mixed the service-oriented structure of 'infrastructure serves application serves humans' in the behavior and active structure aspects while at the same time using the abstraction approach between layers in the passive structure aspect. These days, the abstraction approach is available almost everywhere (just not for internal active structure).

The mix of these two disjunct approaches to layering in the language can be quite confusing. More about this in Chapter "Discussing ArchiMate" on page 205.

12.11 The Use of the Non-core Domains

As illustrated in View 4 on page 18, ArchiMate can be divided into the Core and several other domains. The Core is used to model 'the enterprise itself'. The other domains have other functions. One can wonder what the uses of these other domains are.

What does putting strategy in a model add, really? And is it realistic to assume that we can actually model something complicated as strategy realistically in a set of simple distinct concepts and relations? Doesn't a model become a mess of arguable choices as soon as it becomes more realistic? Modeling the enterprise itself does not suffer from this problem as it is founded on the reality of the enterprise itself. But strategy (and intentions) are much less easy to grasp in discrete elements. So, for strategy at least: modeling will never be more than a rough illustration. I don't expect a board of an organization to use ArchiMate to model the logical structure of their strategy. There is therefore no need to discuss patterns for modeling strategy in the rest of the book. It is all rather basic, superficial, and has little actual practical consequences.

Modeling intentions has the same problem and another one besides: ArchiMate's meta-model for modeling intentions is rich, with 10 different concepts. It is meant to model the 'why' of your *architectural* choices, but it already shows the problems of trying to model human reasoning in rational terms (related to the failed dream of digital AI — and yes, I am aware of big data analytics; and why that is probably not going to change my assessment is not something for this book).

Then there are the overlaps or missing links. The meaning of the Goal concept overlaps with the Course of Action concept, the Driver concept and even with the Requirement and Principle concepts. An Assessment is something that should be 'creatable' by a process, but it isn't a Business Object. Stakeholder and Business Role are separate, without a good reason. See also Chapter "Discussing ArchiMate".

It is not all negative, though. Most of my objections about the potential use of the non-core domains come from having some knowledge about fundamental limitations of approaches like these. But there are practical upsides too. Documenting reasons for choices with the Motivation elements can be useful. The fact that it is a *model* doesn't really add much but documenting is generally a good thing, and modeling (though practiced with restraint here) doesn't cause much harm. Sometimes, the domain can even be used for something really practical from the organization's perspective. For instance, in 19 "Modeling Risk & Security with Motivation Elements" on page 111 I use the Motivation elements to model Risk & Security in a way that allows you to create reports about these from your models (when we started to do this, our regulator immediately upped us a notch in terms of being 'in control' of our landscape).

12.12 Why model?

We will get more into reasons for modeling later in the book and how it influences how you model, but generally there are two reasons:

1. Modeling to explain something visually;
2. Modeling as a tool to work with large complexity.

The former uses the language concepts to improve the clarity of the diagram. Use ArchiMate's Access relation, and it immediately tells you something that a generic arrow from a generic drawing does not. You trust the meaning of the various element and relation *types* to help you convey a message. Of course, this does require that the receiver also understands the language.

The latter uses the fact that the language is a form of structure that enables you to set up a body of knowledge that you can analyze, either by traveling through the model or by setting up automated analyses, such as "which processes depend on this program?". Especially in large models (I know of models with 100,000 elements and relations or more), this is an important reason for modeling.

Ideally, of course, your model can do both. But there is a trade-off: using automated analysis requires a limitation in the patterns you use and a strict discipline. More in Chapter "Reflections" on page 197 and elsewhere in the book.

Here ends the Free Syntax Excerpt from Mastering ArchiMate. The full book can be purchased on paper or as PDF. See

http://masteringarchimate.com/

Mastering ArchiMate Edition 3.1 is the fourth edition of a much praised book about the ArchiMate® Enterprise Architecture Modeling Language, which is a standard and a Registered Trade Mark of The Open Group. The book gives an introduction to the language, then goes on to show you key aspects of successful modeling, and many different patterns for its use. From Business to Infrastructure, from Risk & Security to Application Exploitation and Maintenance.

While the aim of the book is to teach the language, it often also offers necessary background, so that the patterns can make sense to the reader not familiar with a subject. Thus, it also contains introductions to subjects such as virtualization, bitcoin/blockchain, infrastructure as code, processes versus functions, SOA/API, ESB, Terminal Services, etc. It also contains a short introduction to BPMN™ in order to describe a linking of both major languages.

Forewords by Marc Lankhorst and Jean-Baptiste Sarrodie.

Gerben Wierda (1961) is author of *Chess and the Art of Enterprise Architecture*, and is Lead Architect at APG, one of the largest Fiduciary Managers in the world. Before, he was Lead Architect of the Judiciary of The Netherlands, and Lead Architect of APG Asset Management. He has overseen the construction of one of the largest single ArchiMate models in the world to date. He holds an M.Sc. in Physics from the University of Groningen and an MBA from RSM Erasmus, Rotterdam.

This is an excerpt of Mastering ArchiMate. Full PDF and Hardcover available.

Published by R&A
© Gerben Wierda, 2021
info@masteringarchimate.com
ISBN PDF: 978-90-831434-0-8
ISBN hardcover: 978-90-831434-1-5